

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ



ISO 9001:2015

HUỖNH KHẢ

XÂY DỰNG HỆ THỔNG PHÂN TÍCH
VÀ GỢI Ý SỬA LỖI LOGIC MÃ NGUỒN C

ĐỒ ÁN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

VĨNH LONG, NĂM 2026

ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ

**XÂY DỰNG HỆ THỐNG PHÂN TÍCH
VÀ GỢI Ý SỬA LỖI LOGIC MÃ NGUỒN C**

ĐỒ ÁN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

Sinh viên: **HUỲNH KHẢI**

Lớp: **DA22TTB**

MSSV: **110122225**

GVHD: **ThS NGUYỄN NGỌC ĐAN THANH**

VĨNH LONG, NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan đồ án tốt nghiệp này là kết quả nghiên cứu của riêng tôi, được thực hiện dưới sự hướng dẫn của Cô ThS. Nguyễn Ngọc Đan Thanh, đảm bảo tính trung thực và tuân thủ các quy định về trích dẫn, chú thích tài liệu tham khảo. Trong quá trình thực hiện báo cáo, tôi có sử dụng một số công cụ trí tuệ nhân tạo nhằm hỗ trợ tham khảo, hệ thống hóa ý tưởng, gợi ý cấu trúc trình bày và rà soát diễn đạt. Việc sử dụng các công cụ này chỉ mang tính chất hỗ trợ học tập, không thay thế cho quá trình phân tích, thiết kế, triển khai và đánh giá hệ thống của người thực hiện. Toàn bộ nội dung trình bày, kết quả xây dựng hệ thống và các nhận định trong báo cáo đều đã được tôi kiểm tra, chọn lọc và chịu trách nhiệm.

Tôi xin chịu hoàn toàn trách nhiệm về lời cam đoan này.

Vĩnh Long, ngày tháng năm

Sinh viên thực hiện

LỜI CẢM ƠN

Trong suốt quá trình học tập, nghiên cứu và thực hiện đề tài “Xây dựng hệ thống phân tích và gợi ý sửa lỗi logic mã nguồn C”, em đã nhận được sự quan tâm, hướng dẫn và giúp đỡ quý báu từ quý thầy cô, gia đình và bạn bè. Đây là nguồn động viên to lớn giúp em hoàn thành đề tài này.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu cùng quý thầy cô Trường Kỹ thuật và Công nghệ, Đại học Trà Vinh đã tận tình giảng dạy, truyền đạt những kiến thức chuyên môn và kinh nghiệm quý báu trong suốt thời gian học tập. Những kiến thức đó là nền tảng quan trọng giúp em có đủ năng lực để nghiên cứu và thực hiện đề tài.

Đặc biệt, em xin gửi lời cảm ơn chân thành đến cô Nguyễn Ngọc Đan Thanh, người đã trực tiếp hướng dẫn, tận tình góp ý, định hướng và hỗ trợ em trong suốt quá trình thực hiện đề tài. Những ý kiến đóng góp và sự hướng dẫn của cô đã giúp em từng bước hoàn thiện hệ thống cũng như hoàn thành báo cáo này.

Em cũng xin chân thành cảm ơn gia đình và bạn bè đã luôn quan tâm, động viên và tạo điều kiện thuận lợi cả về tinh thần lẫn vật chất để em có thể yên tâm học tập và hoàn thành đề tài đúng tiến độ.

Trong quá trình thực hiện, mặc dù đã cố gắng tìm hiểu, nghiên cứu và hoàn thiện hệ thống nhưng báo cáo khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp và nhận xét từ quý thầy cô để đề tài được hoàn thiện hơn.

Một lần nữa, em xin chân thành cảm ơn!

Sinh viên thực hiện

Huỳnh Khải

BẢN NHẬN XÉT ĐỒ ÁN TỐT NGHIỆP

(Dành cho người hướng dẫn)

Họ và tên người hướng dẫn: Nguyễn Ngọc Đan Thanh

Đơn vị công tác: Khoa Công nghệ Thông tin, Trường Kỹ thuật và Công nghệ, Đại học Trà Vinh

Họ và tên sinh viên: Huỳnh Khải

MSSV: 110122225

Tên đề tài: Xây dựng hệ thống phân tích và gợi ý sửa lỗi logic mã nguồn C

1. Tinh thần, thái độ học tập, nghiên cứu của sinh viên:

Sinh viên có tinh thần học tập nghiêm túc, chủ động trong quá trình nghiên cứu và triển khai đồ án. Trong suốt thời gian thực hiện đề tài, sinh viên thực hiện đầy đủ các buổi báo cáo tiến độ, có ý thức học hỏi, tiếp thu góp ý của giảng viên hướng dẫn và tích cực tìm hiểu các công nghệ mới liên quan đến trí tuệ nhân tạo, lập trình Web và kiểm thử chương trình.

2. Khả năng nghiên cứu khoa học:

Sinh viên thể hiện khả năng tự nghiên cứu tốt thông qua việc tìm hiểu, phân tích và vận dụng nhiều công nghệ hiện đại như Flask, SQLite, cơ chế đánh giá bằng Test Case và mô hình ngôn ngữ lớn (LLM). Đề tài có định hướng nghiên cứu rõ ràng, phù hợp với xu hướng ứng dụng AI trong giáo dục, đồng thời đề xuất được mô hình kết hợp giữa kiểm thử chương trình và AI để hỗ trợ phân tích, gợi ý sửa lỗi logic mã nguồn.

3. Hình thức và nội dung của đồ án

Đồ án được trình bày tương đối đầy đủ và đúng cấu trúc của một báo cáo tốt nghiệp. Nội dung bao gồm các phần phân tích yêu cầu, thiết kế hệ thống, xây dựng và triển khai ứng dụng. Hệ thống hoạt động ổn định, giao diện thân thiện và đáp ứng các mục tiêu nghiên cứu đề ra. Tuy nhiên, đề tài hiện mới hỗ trợ ngôn ngữ C, chưa

có đánh giá định lượng về độ chính xác của AI, đồng thời cần bổ sung các nội dung về hiệu năng, khả năng mở rộng và bảo mật khi triển khai thực tế.

4. Kết luận:

Đề tài đáp ứng được mục tiêu nghiên cứu đã đề ra, có tính thực tiễn và khả năng ứng dụng cao trong hỗ trợ dạy và học lập trình C. Sinh viên có năng lực nghiên cứu, triển khai kỹ thuật và tinh thần làm việc nghiêm túc. Mặc dù vẫn còn một số hạn chế về đánh giá thực nghiệm và phạm vi triển khai, đồ án có hàm lượng kỹ thuật tốt, thể hiện sự đầu tư công sức của sinh viên và có tiềm năng tiếp tục phát triển trong tương lai.

Vĩnh Long, ngày tháng năm 2026

Người hướng dẫn

MỤC LỤC

LỜI CAM ĐOAN.....	i
LỜI CẢM ƠN	ii
MỤC LỤC	v
DANH MỤC CÁC BẢNG BIỂU	ix
TÓM TẮT	xii
CHƯƠNG MỞ ĐẦU.....	1
CHƯƠNG 1 TỔNG QUAN TÀI LIỆU	6
1.1 Tổng quan về ngôn ngữ lập trình C.....	6
1.1.1 Khái niệm ngôn ngữ lập trình C.....	6
1.1.2 Đặc điểm của ngôn ngữ lập trình C.....	6
1.1.3 Cấu trúc cơ bản của chương trình C.....	7
1.2 Lỗi logic trong chương trình C.....	8
1.2.1 Khái niệm lỗi logic	8
1.2.2 Nguyên nhân phát sinh lỗi logic.....	9
1.2.3 Đặc điểm của lỗi logic	10
1.2.4 Các dạng lỗi logic phổ biến.....	11
1.3 Mô hình ngôn ngữ lớn	15
1.3.1 Khái niệm mô hình ngôn ngữ lớn.....	15
1.3.2 Quá trình hoạt động của mô hình ngôn ngữ lớn	16
1.3.3 Một số mô hình ngôn ngữ lớn phổ biến.....	16
1.3.4 So sánh các mô hình ngôn ngữ lớn.....	17
1.3.5 Những thách thức khi sử dụng mô hình ngôn ngữ lớn.....	19
1.4 Trí tuệ nhân tạo trong phân tích mã nguồn.....	19
1.4.1 Phân tích mã nguồn truyền thống.....	19
1.4.2 Phân tích mã nguồn bằng trí tuệ nhân tạo.....	20
1.4.3 Vai trò của AI trong hệ thống	20
1.4.4 Ưu điểm và hạn chế của việc sử dụng AI trong phân tích mã nguồn	21
1.5 Flask Framework.....	22
1.5.1 Giới thiệu về Flask.....	22
1.5.2 Đặc điểm của Flask.....	22
1.5.3 Lý do lựa chọn Flask	23

1.5.4 Vai trò của Flask trong hệ thống	23
1.6 SQLite	24
1.6.1 Giới thiệu về SQLite.....	24
1.6.2 Đặc điểm của SQLite.....	25
1.6.3 Vai trò của SQLite trong hệ thống	26
1.6.4 Mối liên hệ giữa SQLite và hệ thống	27
CHƯƠNG 2 PHÂN TÍCH THIẾT KẾ HỆ THỐNG	28
2.1 Mô tả bài toán.....	28
2.2 Phân tích yêu cầu.....	29
2.2.1 Yêu cầu chức năng.....	29
2.2.2 Yêu cầu phi chức năng.....	31
2.3 Phân tích tác nhân và usecase	32
2.3.1 Danh sách các tác nhân	32
2.3.2 Use case tổng quát	32
2.3.3 Usecase của học sinh	34
2.3.4 Usecase của giáo viên	36
2.3.5 Usecase của quản trị viên.....	37
2.4 Thiết kế cơ sở dữ liệu	38
2.4.1 Mô hình quan hệ giữa các thực thể.....	38
2.4.1 Danh sách các thực thể	39
2.4.2 Thiết kế bảng người dùng (nguoi_dung)	39
2.4.3 Thiết kế bảng đề bài (de_bai).....	41
2.4.4 Thiết kế bảng bộ Test Case	42
2.4.5 Thiết kế bảng nộp bài.....	44
2.4.6 Thiết kế bảng phân tích AI.....	46
2.5 Thiết kế luồng xử lý nghiệp vụ	47
2.5.1 Mô hình hoạt động chức năng đăng nhập.....	48
2.5.2 Mô hình hoạt động chức năng làm bài lập trình	50
2.5.3 Mô hình hoạt động chức năng AI Help	50
2.5.4 Mô hình tuần tự tiến trình làm bài tập	51
2.5.5 Mô hình tuần tự tiến trình AI trợ giúp.....	52
2.6 Thiết kế kiến trúc hệ thống	53

2.6.1 Kiến trúc tổng thể của hệ thống	53
2.6.2 Tầng giao diện	54
2.6.3 Tầng xử lý nghiệp vụ	55
2.6.4 Trình biên dịch GCC	55
2.6.5 Hệ thống kiểm thử bằng Test Case	56
2.6.6 Dịch vụ AI	57
2.6.7 Cơ sở dữ liệu SQLite	57
2.7 Thiết kế API	58
2.7.1 Endpoint xác thực người dùng	58
2.7.2 Endpoint quản lý đề bài	59
2.7.3 Endpoint quản lý Test Case	59
2.7.4 Endpoint xử lý bài làm lập trình	60
2.7.5 Endpoint hỗ trợ AI	60
2.7.6 Endpoint quản trị hệ thống	61
CHƯƠNG 3 KẾT QUẢ THỰC NGHIỆM	62
3.1 Giao diện đăng nhập hệ thống	62
3.2 Chức năng dành cho quản trị viên	63
3.2.1 Quản lý tài khoản người dùng	64
3.2.2 Quản lý đề bài	65
3.2.3 Tạo đề bài mới	66
3.2.4 Quản lý Test Case	67
3.3 Chức năng dành cho giáo viên	69
3.3.1 Quản lý đề bài	69
3.3.2 Quản lý Test Case	70
3.3.3 Theo dõi bài làm của học sinh	71
3.4 Chức năng dành cho học sinh	73
3.4.1 Lựa chọn đề bài	73
3.4.2 Nhập mã nguồn chương trình	74
3.4.3 Biên dịch chương trình	75
3.4.4 Thực thi chương trình và kiểm thử bằng Test Case	76
3.4.5 Hỗ trợ phân tích bằng AI	76
3.5 Kiểm thử hệ thống	77

CHƯƠNG 4 KẾT LUẬN VÀ KIẾN NGHỊ.....	81
4.1 Kết luận.....	81
4.2 Hướng phát triển.....	82
DANH MỤC TÀI LIỆU THAM KHẢO	84
PHỤ LỤC.....	85

DANH MỤC CÁC BẢNG BIỂU

Bảng 2.1 So sánh một số mô hình ngôn ngữ lớn	17
Bảng 3.1 Danh sách các tác nhân.....	32
Bảng 3.2 Danh sách các thực thể	39
Bảng 3.3 Cấu trúc bảng nguoi_dung.....	39
Bảng 3.4 Cấu trúc bảng de_bai	41
Bảng 3.5 Cấu trúc bảng bo_testcase.....	42
Bảng 3.6 Cấu trúc bảng nop_bai.....	44
Bảng 3.7 Cấu trúc bảng mo_hinh_ai.....	46
Bảng 3.8 Endpoint xác thực người dùng.....	59
Bảng 3.9 Endpoint quản lý đề bài	59
Bảng 3.10 Endpoint quản lý Test Case	60
Bảng 3.11 Endpoint xử lý bài làm lập trình.....	60
Bảng 3.12 Endpoint hỗ trợ AI.....	61
Bảng 3.13 Endpoint quản trị hệ thống.....	61
Bảng 4.1 Các thành phần trong giao diện đăng nhập.....	63

DANH MỤC HÌNH ẢNH

Hình 3.1 Use Case Diagram của hệ thống.....	33
Hình 3.2 Usecase phân rã theo tác nhân học sinh.....	34
Hình 3.3 Usecase phân rã theo tác nhân giáo viên	36
Hình 3.4 Usecase phân rã theo tác nhân quản trị viên	37
Hình 3.5 Mô hình ERD.....	38
Hình 3.6 Activity Diagram chức năng đăng nhập	49
Hình 3.7 Activity Diagram chức năng làm bài lập trình.....	50
Hình 3.8 Activity Diagram chức năng AI Help.....	51
Hình 3.9 Mô hình tuần tự tiến trình làm bài tập	51
Hình 3.10 Mô hình tuần tự tiến trình AI trợ giúp	52
Hình 3.11 Kiến trúc hệ thống.....	54
Hình 4.1 Giao diện đăng nhập hệ thống.....	63
Hình 4.2 Giao diện quản lý tài khoản người dùng.....	64
Hình 4.3 Giao diện quản lý đề bài.....	65
Hình 4.4 . Giao diện tạo đề bài	66
Hình 4.5 Giao diện quản lý Test Case.....	67
Hình 4.6 Giao diện quản lý đề bài của giáo viên.....	69
Hình 4.7 Giao diện quản lý Test Case.....	70
Hình 4.8 Giao diện theo dõi bài làm của học sinh	72
Hình 4.9 Giao diện danh sách đề bài.....	74
Hình 4.10 Giao diện nhập mã nguồn.....	75
Hình 4.11 Chức năng biên dịch	76
Hình 4.12 Chọn đề bài.....	78
Hình 4.13 Nhập mã nguồn.....	78
Hình 4.14 Kết quả biên dịch	79

Hình 4.15 Kết quả TestCase	79
Hình 4.16 Kết quả phân tích từ AI.....	80

TÓM TẮT

Đồ án “Xây dựng hệ thống phân tích và gợi ý sửa lỗi logic mã nguồn C” được thực hiện nhằm xây dựng một hệ thống hỗ trợ phát hiện, phân tích và gợi ý sửa lỗi logic trong chương trình C. Trong quá trình học tập và phát triển phần mềm, lỗi logic là loại lỗi khó phát hiện vì chương trình vẫn có thể biên dịch và thực thi thành công nhưng cho kết quả không đúng với yêu cầu bài toán. Việc tìm nguyên nhân và khắc phục các lỗi này thường tốn nhiều thời gian và đòi hỏi kinh nghiệm của người lập trình.

Đề tài đề xuất giải pháp kết hợp giữa trình biên dịch GCC, cơ chế kiểm thử bằng Test Case và các mô hình ngôn ngữ lớn (LLM) như Gemini, OpenRouter và Groq để hỗ trợ phân tích chương trình. Hệ thống cho phép người dùng nhập mã nguồn C, khai báo Test Case, thực hiện biên dịch và chạy chương trình, sau đó gửi thông tin về mã nguồn và kết quả thực thi đến mô hình AI để phân tích nguyên nhân gây lỗi logic và đưa ra các gợi ý sửa lỗi phù hợp.

Hệ thống được xây dựng theo kiến trúc Web gồm giao diện người dùng, tầng xử lý nghiệp vụ và cơ sở dữ liệu. Các chức năng chính bao gồm quản lý bài tập, biên dịch và thực thi chương trình C, kiểm thử bằng Test Case, lưu trữ kết quả và hỗ trợ phân tích lỗi bằng AI. Kết quả triển khai cho thấy hệ thống hoạt động ổn định, giao diện thân thiện và đáp ứng được các mục tiêu nghiên cứu đề ra. Việc kết hợp giữa kiểm thử chương trình và AI giúp người học rút ngắn thời gian tìm lỗi, hiểu rõ nguyên nhân gây lỗi logic và nâng cao kỹ năng lập trình.

Bên cạnh các kết quả đạt được, đề tài vẫn còn một số hạn chế như chỉ hỗ trợ ngôn ngữ C, chất lượng gợi ý còn phụ thuộc vào mô hình AI và chưa có đánh giá định lượng đầy đủ về độ chính xác của các gợi ý. Ngoài ra, các nội dung về phân tích chất lượng mã nguồn, đánh giá hiệu năng, khả năng mở rộng và bảo mật hệ thống chưa được thực hiện đầy đủ.

Đồ án có ý nghĩa thực tiễn trong hỗ trợ giảng dạy và học tập lập trình C, đồng thời tạo nền tảng cho các nghiên cứu tiếp theo về ứng dụng trí tuệ nhân tạo trong phân tích mã nguồn và hỗ trợ học lập trình.

CHƯƠNG MỞ ĐẦU

1. GIỚI THIỆU ĐỀ TÀI

Trong những năm gần đây, công nghệ thông tin phát triển mạnh mẽ và trở thành một trong những lĩnh vực quan trọng của quá trình chuyển đổi số. Trong đó, lập trình là kỹ năng nền tảng đối với sinh viên ngành Công nghệ thông tin cũng như nhiều ngành kỹ thuật khác. Ngôn ngữ lập trình C là một trong những ngôn ngữ được giảng dạy phổ biến tại các trường đại học nhờ khả năng biểu diễn thuật toán rõ ràng, hiệu năng cao và là nền tảng của nhiều ngôn ngữ lập trình hiện đại.

Trong quá trình học tập, sinh viên thường gặp nhiều khó khăn khi xây dựng thuật toán và triển khai chương trình. Bên cạnh các lỗi cú pháp có thể được trình biên dịch phát hiện, lỗi logic là loại lỗi phổ biến nhưng khó nhận biết hơn do chương trình vẫn có thể biên dịch và thực thi bình thường nhưng kết quả đầu ra không đúng với yêu cầu của bài toán. Việc xác định nguyên nhân gây ra lỗi logic thường đòi hỏi người học phải có kiến thức về thuật toán cũng như kinh nghiệm lập trình.

Hiện nay, nhiều hệ thống chấm bài trực tuyến hỗ trợ kiểm tra tính đúng đắn của chương trình thông qua bộ Test Case. Tuy nhiên, phần lớn các hệ thống này chỉ đưa ra kết quả đúng hoặc sai mà chưa giải thích nguyên nhân hoặc hỗ trợ người học cải thiện chương trình. Trong khi đó, sự phát triển của các mô hình ngôn ngữ lớn đã mở ra khả năng phân tích mã nguồn, giải thích lỗi và đưa ra các gợi ý sửa lỗi bằng ngôn ngữ tự nhiên [1].

Xuất phát từ thực tế đó, đề tài “Xây dựng hệ thống phân tích và gợi ý sửa lỗi logic mã nguồn C” được thực hiện nhằm xây dựng một hệ thống hỗ trợ học lập trình trên nền tảng Web. Hệ thống cho phép giáo viên quản lý đề bài và Test Case, học sinh lựa chọn đề bài, nhập hoặc tải lên mã nguồn C để thực hiện bài tập. Chương trình được biên dịch bằng GCC, thực thi với các Test Case tương ứng và so sánh kết quả đầu ra để đánh giá tính đúng đắn của chương trình. Khi phát hiện bài làm chưa đạt yêu cầu hoặc người dùng cần hỗ trợ, hệ thống sẽ sử dụng các mô hình ngôn ngữ lớn như Gemini, OpenRouter hoặc Groq để phân tích nguyên nhân và đưa ra các gợi ý sửa lỗi logic, góp phần nâng cao hiệu quả học tập và rèn luyện kỹ năng lập trình.

2. LÝ DO CHỌN ĐỀ TÀI

Ngôn ngữ lập trình C là môn học nền tảng trong chương trình đào tạo của nhiều trường đại học. Tuy nhiên, trong quá trình học tập, sinh viên thường gặp khó khăn trong việc xác định và sửa các lỗi logic của chương trình. Các trình biên dịch chỉ phát hiện được lỗi cú pháp hoặc lỗi biên dịch mà không thể đánh giá thuật toán hoặc chỉ ra nguyên nhân khiến chương trình cho kết quả sai.

Bên cạnh đó, các hệ thống chấm bài trực tuyến hiện nay chủ yếu đánh giá chương trình thông qua Test Case và chỉ thông báo kết quả đạt hoặc không đạt, chưa hỗ trợ giải thích nguyên nhân hoặc đưa ra hướng sửa lỗi phù hợp. Điều này khiến người học mất nhiều thời gian trong quá trình tự tìm hiểu và cải thiện chương trình.

Sự phát triển của trí tuệ nhân tạo, đặc biệt là các mô hình ngôn ngữ lớn, tạo điều kiện để xây dựng các hệ thống có khả năng phân tích mã nguồn, giải thích lỗi và hỗ trợ người học hiệu quả hơn [1], [2]. Vì vậy, việc xây dựng một hệ thống kết hợp giữa kiểm thử bằng Test Case và AI hỗ trợ phân tích lỗi logic là cần thiết, góp phần nâng cao chất lượng học tập và hỗ trợ công tác giảng dạy.

3. MỤC TIÊU NGHIÊN CỨU

Mục tiêu tổng quát

Xây dựng hệ thống hỗ trợ học tập và thực hành lập trình ngôn ngữ C trên nền tảng Web, cho phép người dùng biên dịch, thực thi và đánh giá chương trình thông qua các bộ Test Case, đồng thời tích hợp trí tuệ nhân tạo nhằm hỗ trợ phân tích nguyên nhân và gợi ý sửa lỗi logic trong mã nguồn. Hệ thống hướng đến việc nâng cao hiệu quả học tập, giúp người học hiểu rõ bản chất của các lỗi logic và cải thiện kỹ năng lập trình thông qua các gợi ý phân tích thông minh.

Mục tiêu cụ thể

Xây dựng hệ thống quản lý người dùng với ba vai trò gồm Quản trị viên, Giáo viên và Học sinh, đáp ứng các chức năng quản lý và phân quyền phù hợp cho từng đối tượng sử dụng.

Xây dựng chức năng quản lý đề bài và bộ Test Case, cho phép giáo viên tạo, cập nhật và quản lý các bài tập lập trình cũng như các bộ dữ liệu kiểm thử phục vụ quá trình học tập và đánh giá.

Xây dựng chức năng biên dịch, thực thi và đánh giá chương trình C thông qua các Test Case, tự động so sánh kết quả thực thi với kết quả mong đợi để hỗ trợ phát hiện lỗi logic trong chương trình.

Tích hợp các mô hình trí tuệ nhân tạo nhằm phân tích nguyên nhân gây ra lỗi logic, giải thích lỗi theo ngữ cảnh của chương trình và đề xuất các hướng sửa chữa phù hợp, giúp người học nâng cao khả năng tư duy và kỹ năng lập trình.

Thiết kế và xây dựng giao diện Web trực quan, thân thiện với người dùng, hỗ trợ nhập mã nguồn, tải tệp chương trình, khai báo Test Case, lựa chọn mô hình AI và theo dõi kết quả phân tích một cách thuận tiện.

4. ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu

- Ngôn ngữ lập trình C.
- Các lỗi logic trong chương trình C.
- Phương pháp kiểm thử chương trình bằng Test Case.
- Trình biên dịch GCC.
- Mô hình ngôn ngữ lớn.
- Flask Framework.
- SQLite.

Phạm vi nghiên cứu

Đề tài tập trung xây dựng hệ thống hỗ trợ học lập trình C trên nền tảng Web. Hệ thống chỉ hỗ trợ các chương trình C dạng Console và sử dụng GCC để biên dịch, thực thi chương trình. Việc đánh giá chương trình được thực hiện thông qua bộ Test Case do giáo viên xây dựng. AI chỉ đóng vai trò hỗ trợ phân tích và gợi ý sửa lỗi logic, không thay thế quá trình đánh giá kết quả bằng Test Case.

5. PHƯƠNG PHÁP NGHIÊN CỨU

Phương pháp nghiên cứu tài liệu

Đề tài sử dụng phương pháp nghiên cứu tài liệu nhằm thu thập, phân tích và tổng hợp các kiến thức, công trình nghiên cứu và tài liệu kỹ thuật liên quan đến bài

toán phát hiện và gợi ý sửa lỗi logic trong chương trình C. Trên cơ sở đó, đề tài nghiên cứu các đặc điểm của ngôn ngữ lập trình C, các dạng lỗi logic thường gặp, phương pháp kiểm thử chương trình bằng Test Case, cũng như khả năng ứng dụng các mô hình ngôn ngữ lớn trong phân tích mã nguồn. Bên cạnh đó, đề tài cũng nghiên cứu các công nghệ phát triển hệ thống như Flask Framework, SQLite và trình biên dịch GCC để lựa chọn giải pháp phù hợp cho việc triển khai hệ thống.

Phương pháp phân tích và thiết kế hệ thống

Áp dụng phương pháp phân tích và thiết kế hướng đối tượng nhằm xác định các yêu cầu chức năng và phi chức năng của hệ thống, đồng thời mô hình hóa các thành phần và mối quan hệ giữa chúng. Trên cơ sở đó, hệ thống được thiết kế thông qua các sơ đồ UML như Use Case Diagram, Activity Diagram, Sequence Diagram và Class Diagram. Ngoài ra, đề tài tiến hành thiết kế kiến trúc hệ thống, cơ sở dữ liệu và giao diện người dùng nhằm đảm bảo tính nhất quán, khả năng mở rộng và thuận tiện trong quá trình phát triển cũng như bảo trì hệ thống.

Phương pháp lập trình và triển khai

Sử dụng phương pháp phát triển ứng dụng Web theo mô hình Client–Server với kiến trúc phân tách giữa Frontend và Backend nhằm nâng cao tính độc lập, khả năng mở rộng và bảo trì của hệ thống. Backend được xây dựng bằng Flask để xử lý các nghiệp vụ, quản lý người dùng, điều phối quá trình biên dịch và thực thi chương trình C. Frontend được phát triển bằng HTML, CSS và JavaScript nhằm xây dựng giao diện trực quan và hỗ trợ người dùng tương tác với hệ thống. Cơ sở dữ liệu SQLite được sử dụng để lưu trữ thông tin người dùng, mã nguồn và kết quả phân tích. Đồng thời, hệ thống tích hợp trình biên dịch GCC để biên dịch và thực thi chương trình C, kết hợp với các API của Gemini, OpenRouter và Groq nhằm phân tích nguyên nhân lỗi logic và đưa ra các gợi ý sửa lỗi dựa trên trí tuệ nhân tạo.

6. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN

Về mặt khoa học, đề tài góp phần nghiên cứu việc kết hợp giữa phương pháp kiểm thử bằng Test Case với các mô hình ngôn ngữ lớn nhằm hỗ trợ phân tích lỗi logic trong chương trình C.

Về mặt thực tiễn, hệ thống hỗ trợ giáo viên trong việc quản lý bài tập và đánh giá kết quả học tập của sinh viên, đồng thời giúp học sinh dễ dàng kiểm tra chương trình, xác định nguyên nhân gây lỗi logic và nhận được các gợi ý sửa lỗi từ AI, qua đó nâng cao hiệu quả học tập và rèn luyện kỹ năng lập trình [1], [3].

7. BỐ CỤC BÁO CÁO

Đồ án được tổ chức thành 5 chương với nội dung chính như sau:

Chương mở đầu

Trình bày lý do chọn đề tài, mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, phương pháp thực hiện và ý nghĩa khoa học, thực tiễn của đề tài.

Chương 1. Tổng quan tài liệu

Trình bày các kiến thức nền tảng liên quan đến ngôn ngữ lập trình C, lỗi logic, kiểm thử bằng Test Case, mô hình ngôn ngữ lớn (LLM), ứng dụng AI trong phân tích mã nguồn và các công nghệ sử dụng để phát triển hệ thống.

Chương 2. Phân tích và thiết kế hệ thống

Phân tích bài toán, xác định yêu cầu hệ thống và trình bày thiết kế kiến trúc, cơ sở dữ liệu, giao diện người dùng và các mô hình UML của hệ thống.

Chương 3. Kết quả triển khai hệ thống

Trình bày quá trình xây dựng hệ thống, các chức năng chính, giao diện người dùng và kết quả thử nghiệm khả năng biên dịch, thực thi, phân tích lỗi logic và gợi ý sửa lỗi bằng AI.

Chương 4. Kết luận và hướng phát triển

Tổng kết kết quả đạt được, nêu các hạn chế của đề tài và đề xuất hướng phát triển nhằm mở rộng chức năng và nâng cao khả năng ứng dụng của hệ thống trong tương lai.

CHƯƠNG 1 TỔNG QUAN TÀI LIỆU

1.1 TỔNG QUAN VỀ NGÔN NGỮ LẬP TRÌNH C

1.1.1 Khái niệm ngôn ngữ lập trình C

Ngôn ngữ lập trình C được phát triển bởi Dennis M. Ritchie tại Bell Laboratories vào năm 1972 nhằm phục vụ việc xây dựng hệ điều hành UNIX. Với thiết kế đơn giản, hiệu quả và khả năng truy cập trực tiếp đến tài nguyên phần cứng, C nhanh chóng trở thành một trong những ngôn ngữ lập trình có ảnh hưởng lớn nhất trong lịch sử phát triển phần mềm. Nhiều ngôn ngữ lập trình hiện đại như C++, Java, C# hay Objective-C đều chịu ảnh hưởng từ cú pháp và tư tưởng thiết kế của ngôn ngữ C [4].

C được xếp vào nhóm ngôn ngữ lập trình bậc trung, kết hợp ưu điểm của cả ngôn ngữ bậc thấp và ngôn ngữ bậc cao. Một mặt, C cho phép lập trình viên thao tác trực tiếp với bộ nhớ thông qua con trỏ, quản lý tài nguyên hệ thống và tương tác với phần cứng. Mặt khác, ngôn ngữ vẫn cung cấp đầy đủ các cấu trúc điều khiển, hàm, kiểu dữ liệu và thư viện chuẩn để xây dựng các chương trình có quy mô lớn.

Khác với các ngôn ngữ thông dịch, chương trình C cần được biên dịch thành mã máy trước khi thực thi. Quá trình biên dịch được thực hiện bởi các trình biên dịch như GCC hoặc Clang. Trong quá trình này, trình biên dịch có thể phát hiện các lỗi cú pháp, lỗi khai báo hoặc lỗi kiểu dữ liệu. Tuy nhiên, các lỗi liên quan đến thuật toán và quá trình xử lý dữ liệu thường không được phát hiện vì chương trình vẫn hợp lệ về mặt cú pháp và có thể thực thi bình thường.

Nhờ tốc độ thực thi cao, khả năng kiểm soát tài nguyên tốt và tính ổn định, ngôn ngữ C hiện vẫn được sử dụng rộng rãi trong phát triển hệ điều hành, trình điều khiển thiết bị, hệ thống nhúng, trình biên dịch và các ứng dụng yêu cầu hiệu năng cao. Đồng thời, đây cũng là ngôn ngữ nền tảng trong giảng dạy lập trình tại nhiều trường đại học, giúp sinh viên hình thành tư duy thuật toán và hiểu rõ cách thức hoạt động của máy tính.

1.1.2 Đặc điểm của ngôn ngữ lập trình C

Ngôn ngữ C sở hữu nhiều đặc điểm nổi bật, góp phần tạo nên vị trí quan trọng của nó trong lĩnh vực phát triển phần mềm.

Thứ nhất, C có hiệu năng thực thi cao. Sau khi được biên dịch, chương trình được chuyển trực tiếp thành mã máy nên có tốc độ xử lý nhanh và sử dụng tài nguyên hệ thống hiệu quả. Đây là lý do C thường được sử dụng trong các phần mềm hệ thống và các ứng dụng yêu cầu hiệu suất cao.

Thứ hai, C cho phép quản lý bộ nhớ linh hoạt thông qua con trỏ và các hàm cấp phát động như malloc(), calloc() và free(). Lập trình viên có thể chủ động kiểm soát việc cấp phát và giải phóng bộ nhớ, từ đó tối ưu hiệu năng chương trình. Tuy nhiên, nếu sử dụng không đúng cách có thể dẫn đến các lỗi như rò rỉ bộ nhớ hoặc truy cập vùng nhớ không hợp lệ [6].

Thứ ba, C có cấu trúc chương trình rõ ràng. Chương trình được tổ chức thành các hàm độc lập, giúp tăng khả năng tái sử dụng mã nguồn, giảm độ phức tạp và thuận tiện trong việc bảo trì cũng như mở rộng hệ thống.

Ngoài ra, C còn có tính di động cao. Nhờ các tiêu chuẩn ANSI C và ISO C, chương trình có thể được biên dịch trên nhiều hệ điều hành khác nhau mà chỉ cần thay đổi rất ít hoặc không cần thay đổi mã nguồn. Điều này giúp C trở thành một trong những ngôn ngữ được sử dụng phổ biến trong cả môi trường học thuật và công nghiệp.

Bên cạnh các ưu điểm trên, C cũng tồn tại một số hạn chế. Ngôn ngữ không hỗ trợ cơ chế quản lý bộ nhớ tự động, không tích hợp sẵn lập trình hướng đối tượng và đặc biệt không có khả năng phát hiện các lỗi logic trong chương trình. Vì vậy, người lập trình cần thực hiện kiểm thử, phân tích thuật toán và đánh giá kết quả đầu ra để đảm bảo chương trình hoạt động chính xác.

1.1.3 Cấu trúc cơ bản của chương trình C

Một chương trình C thông thường bao gồm các thành phần như chỉ thị tiền xử lý, khai báo thư viện, khai báo hàm, hàm chính main() và các câu lệnh xử lý.

Ví dụ một chương trình C đơn giản:

```
#include <stdio.h>
int main() {
    printf("Hello World!");
    return 0;}
```

Trong chương trình trên, chỉ thị `#include <stdio.h>` được sử dụng để khai báo thư viện chuẩn hỗ trợ các hàm nhập xuất. Hàm `main()` là điểm bắt đầu thực thi của chương trình. Lệnh `printf()` dùng để hiển thị chuỗi ký tự ra màn hình, trong khi câu lệnh `return 0` cho biết chương trình kết thúc thành công [6].

Đối với các chương trình có quy mô lớn, mã nguồn thường được chia thành nhiều hàm khác nhau để tăng tính mô-đun, giúp việc phát triển và bảo trì trở nên thuận tiện hơn. Các hàm có thể nhận tham số đầu vào, xử lý dữ liệu và trả về kết quả cho chương trình chính.

Trong đề tài này, hệ thống tiếp nhận toàn bộ mã nguồn C do người dùng nhập trực tiếp hoặc tải lên từ tệp. Sau đó, mã nguồn được gửi đến trình biên dịch GCC để kiểm tra lỗi cú pháp và tạo tệp thực thi. Nếu quá trình biên dịch thành công, chương trình sẽ tiếp tục được thực thi với các Test Case nhằm phục vụ việc phát hiện lỗi logic.

1.2 LỖI LOGIC TRONG CHƯƠNG TRÌNH C

1.2.1 Khái niệm lỗi logic

Trong quá trình phát triển phần mềm, chương trình có thể phát sinh nhiều loại lỗi khác nhau. Một trong những loại lỗi khó phát hiện nhất là lỗi logic. Đây là loại lỗi xảy ra khi thuật toán hoặc cách xử lý của chương trình không đúng với yêu cầu của bài toán, mặc dù chương trình vẫn đảm bảo đúng về mặt cú pháp và có thể biên dịch thành công [2], [7].

Khác với lỗi cú pháp, lỗi logic không được trình biên dịch phát hiện trong quá trình biên dịch. Khi chương trình được thực thi, các câu lệnh vẫn được xử lý theo đúng cú pháp đã viết, tuy nhiên kết quả đầu ra lại không đáp ứng yêu cầu hoặc sai lệch so với kết quả mong đợi. Chính vì vậy, việc phát hiện lỗi logic thường phải thông qua quá trình kiểm thử với nhiều bộ dữ liệu khác nhau hoặc dựa vào việc phân tích thuật toán của chương trình.

Ví dụ, yêu cầu của bài toán là tính tổng các số từ 1 đến n . Người lập trình viết chương trình như sau:

```
int sum = 0;
for(int i = 1; i < n; i++)
    sum += i;
```

Chương trình trên hoàn toàn hợp lệ về cú pháp và được biên dịch thành công. Tuy nhiên, điều kiện vòng lặp $i < n$ khiến chương trình chỉ tính tổng từ 1 đến $n-1$ thay vì từ 1 đến n . Kết quả thực thi vì vậy không đúng với yêu cầu của bài toán. Đây là một ví dụ điển hình của lỗi logic.

Trong đề tài này, lỗi logic là đối tượng phân tích chính của hệ thống. Sau khi chương trình được biên dịch thành công, hệ thống sẽ thực thi chương trình với các Test Case do người dùng cung cấp. Nếu kết quả đầu ra khác với kết quả mong đợi, toàn bộ thông tin về mã nguồn, dữ liệu đầu vào và kết quả thực thi sẽ được gửi đến mô hình trí tuệ nhân tạo để phân tích nguyên nhân gây lỗi và đề xuất hướng sửa chữa.

1.2.2 Nguyên nhân phát sinh lỗi logic

Lỗi logic có thể xuất phát từ nhiều nguyên nhân khác nhau trong quá trình thiết kế thuật toán và hiện thực chương trình. Không giống các lỗi cú pháp thường xuất hiện do sai quy tắc ngôn ngữ lập trình, lỗi logic chủ yếu liên quan đến tư duy giải quyết bài toán của người lập trình.

Nguyên nhân đầu tiên là hiểu chưa chính xác yêu cầu bài toán. Khi người lập trình phân tích sai yêu cầu hoặc bỏ sót một số điều kiện của bài toán, thuật toán được xây dựng sẽ không phản ánh đúng mục tiêu cần giải quyết. Điều này dẫn đến việc chương trình cho kết quả sai mặc dù quá trình thực thi vẫn diễn ra bình thường.

Nguyên nhân thứ hai là thiết kế thuật toán chưa hợp lý. Thuật toán là nền tảng quyết định tính đúng đắn của chương trình. Nếu trình tự xử lý dữ liệu hoặc các bước giải quyết bài toán không chính xác thì kết quả cuối cùng cũng sẽ sai. Đây là dạng lỗi logic thường gặp ở những chương trình có thuật toán phức tạp.

Ngoài ra, việc sử dụng sai điều kiện trong các cấu trúc điều khiển như if, switch, for hoặc while cũng là nguyên nhân phổ biến gây ra lỗi logic. Chỉ một sai sót nhỏ trong điều kiện kiểm tra cũng có thể làm chương trình thực hiện sai nhánh xử lý hoặc lặp sai số lần cần thiết.

Một nguyên nhân khác là không xét đầy đủ các trường hợp biên của dữ liệu đầu vào. Trong nhiều bài toán, chương trình có thể hoạt động đúng với các trường hợp thông thường nhưng lại cho kết quả sai khi gặp dữ liệu đặc biệt như giá trị bằng 0, số âm, tập dữ liệu rỗng hoặc giá trị cực đại của biến.

Bên cạnh đó, người lập trình cũng có thể lựa chọn toán tử hoặc công thức tính toán không phù hợp, làm thay đổi hoàn toàn kết quả xử lý của chương trình. Những lỗi này thường rất khó phát hiện nếu chỉ quan sát mã nguồn mà không tiến hành kiểm thử.

Trong thực tế, các lỗi logic thường xuất hiện đồng thời với nhau. Một chương trình có thể mắc nhiều lỗi logic ở các vị trí khác nhau, khiến quá trình tìm nguyên nhân và sửa lỗi trở nên phức tạp hơn. Vì vậy, việc kết hợp giữa kiểm thử chương trình và sử dụng trí tuệ nhân tạo để phân tích mã nguồn là một hướng tiếp cận hiệu quả nhằm hỗ trợ người lập trình phát hiện lỗi nhanh hơn.

1.2.3 Đặc điểm của lỗi logic

Lỗi logic có một số đặc điểm khác biệt so với các loại lỗi khác trong lập trình.

Đặc điểm đầu tiên là chương trình vẫn biên dịch thành công. Do các câu lệnh đều tuân thủ đúng quy tắc cú pháp của ngôn ngữ C nên trình biên dịch không phát hiện bất kỳ lỗi nào và vẫn tạo ra tệp thực thi.

Đặc điểm thứ hai là chương trình vẫn có thể thực thi bình thường. Khi chạy chương trình, hệ điều hành không phát hiện lỗi bất thường và chương trình vẫn hoàn thành quá trình xử lý theo đúng trình tự mà người lập trình xây dựng.

Tuy nhiên, kết quả đầu ra không đúng với yêu cầu của bài toán. Đây là dấu hiệu quan trọng nhất để nhận biết lỗi logic. Trong nhiều trường hợp, chương trình chỉ cho kết quả sai với một số bộ dữ liệu nhất định, trong khi các trường hợp còn lại vẫn cho kết quả đúng.

Một đặc điểm khác là khó phát hiện bằng các công cụ truyền thống. Trình biên dịch như GCC chỉ kiểm tra tính hợp lệ về cú pháp, kiểu dữ liệu hoặc liên kết thư viện mà không đánh giá tính đúng đắn của thuật toán. Do đó, lỗi logic chỉ được phát hiện thông qua quá trình kiểm thử hoặc phân tích kết quả thực thi.

Ngoài ra, lỗi logic thường phụ thuộc vào dữ liệu đầu vào. Với cùng một chương trình, một số Test Case có thể cho kết quả chính xác trong khi các Test Case khác lại cho kết quả sai. Điều này khiến việc xây dựng bộ Test Case đầy đủ trở thành một yếu tố quan trọng trong quá trình kiểm thử phần mềm.

Đối với đề tài này, các đặc điểm trên chính là cơ sở để xây dựng quy trình phân tích lỗi logic. Thay vì chỉ dựa vào kết quả biên dịch, hệ thống sẽ thực thi chương trình với nhiều Test Case khác nhau, thu thập kết quả đầu ra và so sánh với kết quả mong đợi. Các thông tin này sau đó được gửi đến mô hình AI nhằm phân tích nguyên nhân gây lỗi, giải thích quá trình xử lý của chương trình và đề xuất hướng sửa chữa phù hợp.

1.2.4 Các dạng lỗi logic phổ biến

Lỗi logic có thể xuất hiện ở nhiều vị trí khác nhau trong chương trình. Tùy theo nguyên nhân phát sinh mà lỗi logic được chia thành nhiều dạng khác nhau. Trong phạm vi đề tài, hệ thống tập trung phân tích các dạng lỗi logic thường gặp trong chương trình C, bao gồm sai điều kiện vòng lặp, sai điều kiện rẽ nhánh, sử dụng sai toán tử, sai thuật toán xử lý và không xử lý đầy đủ các trường hợp biên. Đây đều là những lỗi mà trình biên dịch GCC không thể phát hiện nhưng lại ảnh hưởng trực tiếp đến tính đúng đắn của kết quả thực thi [2].

❖ Sai điều kiện vòng lặp

Vòng lặp là cấu trúc điều khiển được sử dụng để thực hiện lặp lại một nhóm câu lệnh nhiều lần. Lỗi logic xảy ra khi người lập trình xác định không đúng điều kiện bắt đầu, điều kiện kết thúc hoặc bước tăng của biến lặp. Điều này có thể khiến chương trình thực hiện thiếu số lần lặp, lặp thừa hoặc trong một số trường hợp gây ra vòng lặp vô hạn.

Ví dụ, yêu cầu của bài toán là tính tổng các số nguyên từ 1 đến n.

```
#include <stdio.h>

int main() {
    int n = 5;
    int sum = 0;

    for(int i = 1; i < n; i++)
        sum += i;

    printf("%d", sum);
}
```

Trong ví dụ trên, điều kiện của vòng lặp được viết là $i < n$. Khi $n = 5$, chương trình chỉ cộng các giá trị từ 1 đến 4 nên kết quả nhận được là 10, trong khi kết quả đúng phải là 15.

Nguyên nhân của lỗi là người lập trình sử dụng sai điều kiện kết thúc của vòng lặp. Điều kiện đúng phải là:

```
for(int i = 1; i <= n; i++)
```

Mặc dù chương trình vẫn được biên dịch và thực thi bình thường, kết quả đầu ra không đáp ứng yêu cầu của bài toán nên đây là một lỗi logic.

Đối với hệ thống được xây dựng trong đề tài, sau khi chương trình được thực thi với các Test Case, AI sẽ so sánh kết quả đầu ra với kết quả mong đợi. Khi phát hiện kết quả nhỏ hơn dự kiến, mô hình có thể phân tích mã nguồn và nhận diện điều kiện vòng lặp chưa phù hợp, từ đó gợi ý thay đổi từ $<$ sang $<=$.

❖ Sai điều kiện rẽ nhánh

Điều kiện rẽ nhánh quyết định chương trình sẽ thực hiện nhánh xử lý nào. Nếu biểu thức điều kiện không phản ánh đúng yêu cầu của bài toán thì chương trình sẽ thực hiện sai nhánh và tạo ra kết quả không chính xác.

Ví dụ, yêu cầu là kiểm tra số lớn hơn.

```
#include <stdio.h>

int main() {
    int a = 8;
    int b = 5;

    if(a < b)
        printf("a la so lon hon");
}
```

Chương trình trên hoàn toàn hợp lệ về cú pháp nhưng điều kiện kiểm tra lại không đúng. Khi thực thi, chương trình sẽ không in ra kết quả mặc dù giá trị của a lớn hơn b .

Điều kiện đúng phải là:

```
if(a > b)
```

Đây là dạng lỗi thường gặp khi người lập trình nhầm lẫn giữa các toán tử so sánh hoặc chưa phân tích đầy đủ yêu cầu bài toán.

Trong hệ thống của đề tài, AI không chỉ phát hiện kết quả sai mà còn có thể giải thích rằng điều kiện so sánh đang bị đảo ngược và đề xuất điều kiện phù hợp hơn.

❖ Sử dụng sai toán tử

Toán tử đóng vai trò thực hiện các phép tính và so sánh trong chương trình. Việc lựa chọn sai toán tử có thể làm thay đổi hoàn toàn kết quả xử lý.

Ví dụ:

```
#include <stdio.h>

int main() {
    int a = 5;
    int b = 2;

    float c = a / b;

    printf("%.1f", c);
}
```

Kết quả hiển thị là

2.0

trong khi người lập trình mong muốn nhận được

2.5

Nguyên nhân là phép chia được thực hiện giữa hai số nguyên nên phần thập phân bị loại bỏ trước khi gán vào biến `float`.

Cách sửa:

```
float c = (float)a / b;
```

Sau khi ép kiểu, chương trình sẽ cho kết quả chính xác là 2.5.

Dạng lỗi này rất phổ biến trong các chương trình tính toán và thường không được trình biên dịch cảnh báo. AI có thể phân tích kiểu dữ liệu của các biến và giải thích nguyên nhân dẫn đến kết quả sai.

❖ Sai thuật toán xử lý

Đây là dạng lỗi logic phức tạp nhất vì nguyên nhân xuất phát từ chính tư duy giải quyết bài toán của người lập trình. Thuật toán được xây dựng không đúng sẽ khiến toàn bộ quá trình xử lý dữ liệu sai, mặc dù từng câu lệnh riêng lẻ đều hợp lệ.

Ví dụ, bài toán yêu cầu tìm số lớn nhất trong mảng.

```
int max = a[0];  
  
for(int i = 1; i < n; i++)  
{  
    if(a[i] < max)  
        max = a[i];  
}
```

Thuật toán trên thực tế lại tìm giá trị nhỏ nhất thay vì giá trị lớn nhất do sử dụng sai điều kiện so sánh.

Điều kiện đúng phải là:

```
if(a[i] > max)
```

Lỗi này không thể phát hiện thông qua quá trình biên dịch mà chỉ được nhận biết khi so sánh kết quả đầu ra với kết quả mong đợi.

Trong đề tài, đây là dạng lỗi mà AI phát huy hiệu quả rõ rệt. Thay vì chỉ thông báo kết quả sai, mô hình có thể phân tích mục đích của thuật toán, xác định nguyên nhân và giải thích vì sao điều kiện hiện tại không phù hợp.

❖ Không xử lý đầy đủ trường hợp biên

Trong nhiều bài toán, người lập trình thường chỉ kiểm thử với các dữ liệu thông thường mà bỏ qua các trường hợp đặc biệt như giá trị bằng 0, số âm, mảng rỗng hoặc dữ liệu có kích thước rất lớn. Điều này khiến chương trình chỉ hoạt động đúng trong một số trường hợp nhất định.

Ví dụ:

```
int average(int sum, int n)  
{  
    return sum / n;  
}
```

Nếu $n = 0$, chương trình sẽ phát sinh lỗi chia cho 0 trong quá trình thực thi.

Để khắc phục, cần kiểm tra điều kiện trước khi thực hiện phép chia.

```
if(n == 0)
{
    printf("Khong hop le");
}
else
{
    printf("%d", sum / n);
}
```

Việc xử lý đầy đủ các trường hợp biên giúp chương trình hoạt động ổn định hơn và hạn chế các lỗi phát sinh trong quá trình sử dụng.

Đối với hệ thống trong đề tài, nếu người dùng bổ sung các Test Case chứa dữ liệu biên, AI có thể dựa trên kết quả thực thi để nhận diện chương trình chưa xử lý đầy đủ các tình huống đặc biệt và đề xuất bổ sung các điều kiện kiểm tra phù hợp.

1.3 MÔ HÌNH NGÔN NGỮ LỚN

1.3.1 Khái niệm mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn là các mô hình trí tuệ nhân tạo được huấn luyện trên khối lượng dữ liệu văn bản rất lớn nhằm học các quy luật của ngôn ngữ tự nhiên và tạo ra văn bản có ý nghĩa. Khác với các hệ thống dựa trên tập luật, mô hình ngôn ngữ lớn có khả năng hiểu ngữ cảnh, suy luận và sinh nội dung mới dựa trên dữ liệu đầu vào mà không cần lập trình sẵn các quy tắc xử lý cho từng trường hợp cụ thể.

Sự phát triển của mô hình ngôn ngữ lớn gắn liền với kiến trúc Transformer, được giới thiệu năm 2017. Kiến trúc này cho phép mô hình xử lý đồng thời nhiều thành phần của chuỗi dữ liệu, từ đó nâng cao khả năng ghi nhớ ngữ cảnh và học được mối quan hệ giữa các từ trong văn bản. Nhờ đó, LLM có thể thực hiện nhiều nhiệm vụ như trả lời câu hỏi, dịch máy, tóm tắt văn bản, sinh mã nguồn và phân tích chương trình với độ chính xác ngày càng cao.

Hiện nay, LLM không chỉ được ứng dụng trong lĩnh vực xử lý ngôn ngữ tự nhiên mà còn được sử dụng rộng rãi trong phát triển phần mềm. Các mô hình có khả năng đọc hiểu mã nguồn, giải thích thuật toán, phát hiện lỗi lập trình và đề xuất phương án sửa chữa. Theo nghiên cứu của Macneil và cộng sự, các mô hình ngôn ngữ lớn có khả năng phát hiện nhiều dạng lỗi logic trong chương trình với hiệu quả tương đương hoặc cao hơn người học ở một số bài toán lập trình.

Trong phạm vi đề tài, LLM đóng vai trò là thành phần phân tích thông minh. Sau khi chương trình được biên dịch và thực thi, toàn bộ thông tin về mã nguồn, dữ liệu đầu vào, kết quả đầu ra và kết quả mong đợi sẽ được tổng hợp thành Prompt gửi đến mô hình AI. Dựa trên các thông tin này, mô hình sẽ phân tích nguyên nhân gây lỗi logic và đưa ra gợi ý sửa lỗi phù hợp.

1.3.2 Quá trình hoạt động của mô hình ngôn ngữ lớn

Một mô hình ngôn ngữ lớn thường trải qua ba giai đoạn chính gồm tiền huấn luyện, tinh chỉnh và suy luận.

Ở giai đoạn tiền huấn luyện, mô hình được học từ tập dữ liệu có quy mô rất lớn bao gồm sách, bài báo, tài liệu kỹ thuật và mã nguồn lập trình. Quá trình này giúp mô hình học được các quy luật của ngôn ngữ cũng như mối quan hệ giữa các từ và câu.

Sau khi hoàn thành tiền huấn luyện, mô hình tiếp tục được tinh chỉnh cho các nhiệm vụ cụ thể như trả lời câu hỏi, hỗ trợ lập trình hoặc phân tích mã nguồn. Giai đoạn Fine-tuning giúp mô hình cải thiện độ chính xác trong từng lĩnh vực chuyên biệt và giảm các phản hồi không phù hợp.

Trong quá trình sử dụng thực tế, người dùng gửi yêu cầu dưới dạng Prompt. Mô hình sẽ phân tích nội dung Prompt, kết hợp với kiến thức đã học trong quá trình huấn luyện để sinh ra câu trả lời phù hợp. Đối với đề tài này, Prompt bao gồm mã nguồn C, dữ liệu đầu vào, kết quả thực thi và kết quả mong đợi. Trên cơ sở đó, mô hình đưa ra nhận xét về nguyên nhân gây lỗi logic và đề xuất hướng khắc phục.

Nhờ khả năng hiểu ngữ cảnh và suy luận, mô hình ngôn ngữ lớn có thể giải thích lỗi bằng ngôn ngữ tự nhiên thay vì chỉ đưa ra thông báo đơn thuần như các công cụ kiểm tra truyền thống.

1.3.3 Một số mô hình ngôn ngữ lớn phổ biến

Trong những năm gần đây, nhiều mô hình ngôn ngữ lớn đã được phát triển với các đặc điểm và thế mạnh khác nhau [7]. Một số mô hình tiêu biểu được sử dụng rộng rãi hiện nay bao gồm:

GPT do OpenAI phát triển, nổi bật với khả năng xử lý ngôn ngữ tự nhiên, sinh văn bản và hỗ trợ lập trình.

Gemini do Google DeepMind phát triển, hỗ trợ xử lý đa phương thức, có khả năng phân tích văn bản, hình ảnh và mã nguồn.

Claude do Anthropic phát triển, được đánh giá cao về khả năng suy luận, phân tích tài liệu dài và giải thích chi tiết.

Llama do Meta phát triển, là mô hình mã nguồn mở được cộng đồng nghiên cứu và doanh nghiệp sử dụng rộng rãi.

DeepSeek là mô hình được tối ưu cho các bài toán lập trình và suy luận, có khả năng sinh mã nguồn với chất lượng cao.

Ngoài các mô hình trên, còn có nhiều mô hình khác như Qwen, Mistral hay Phi được phát triển nhằm phục vụ các nhu cầu chuyên biệt trong xử lý ngôn ngữ và lập trình.

1.3.4 So sánh các mô hình ngôn ngữ lớn

Mỗi mô hình đều có những ưu điểm và hạn chế riêng. Bảng 2.1 trình bày sự so sánh một số mô hình phổ biến dựa trên các tiêu chí liên quan đến đề tài.

Bảng 1.1 So sánh một số mô hình ngôn ngữ lớn

Tiêu chí	GPT	Gemini	Claude	Llama	DeepSeek
Hiểu ngôn ngữ tự nhiên	Rất tốt	Rất tốt	Rất tốt	Tốt	Tốt
Phân tích mã nguồn	Rất tốt	Tốt	Tốt	Khá	Rất tốt
Giải thích lỗi	Tốt	Tốt	Rất tốt	Khá	Tốt
Hỗ trợ API	Có	Có	Có	Phụ thuộc triển khai	Có
Khả năng triển khai	Dễ	Dễ	Dễ	Cần tự triển khai	Dễ

Qua bảng 2.1 có thể thấy, các mô hình hiện nay đều có khả năng hỗ trợ lập trình ở mức cao. Tuy nhiên, mỗi mô hình lại được tối ưu cho những mục đích khác nhau. GPT và Claude nổi bật về khả năng suy luận, DeepSeek được tối ưu cho mã nguồn, trong khi Gemini có lợi thế về khả năng tích hợp với hệ sinh thái của Google và hỗ trợ đa phương thức.

Trong quá trình xây dựng hệ thống, có nhiều mô hình và nền tảng AI có thể được sử dụng để hỗ trợ phân tích mã nguồn. Tuy nhiên, đề tài lựa chọn Gemini, OpenRouter và Groq dựa trên các tiêu chí như khả năng phân tích mã nguồn, chi phí

sử dụng, tốc độ phản hồi, khả năng tích hợp API và tính linh hoạt trong quá trình phát triển hệ thống.

Gemini là mô hình ngôn ngữ lớn do Google phát triển, có khả năng xử lý ngôn ngữ tự nhiên và phân tích mã nguồn hiệu quả. Mô hình có thể đọc hiểu cấu trúc chương trình, giải thích nguyên nhân gây lỗi và đưa ra các gợi ý sửa lỗi bằng ngôn ngữ tự nhiên. Bên cạnh đó, Google cung cấp API miễn phí với hạn mức sử dụng nhất định, đáp ứng tốt nhu cầu nghiên cứu và phát triển của đề tài mà không phát sinh chi phí trong quá trình thử nghiệm. Ngoài ra, API của Gemini có tài liệu hướng dẫn đầy đủ, dễ tích hợp với ứng dụng Web sử dụng Flask.

OpenRouter không phải là một mô hình ngôn ngữ lớn mà là một nền tảng trung gian cho phép truy cập nhiều mô hình AI khác nhau thông qua một API thống nhất. Thông qua OpenRouter, hệ thống có thể sử dụng các mô hình như DeepSeek, Qwen, Mistral hoặc nhiều mô hình khác mà không cần thay đổi cấu trúc chương trình. Một ưu điểm khác của OpenRouter là cung cấp nhiều mô hình miễn phí hoặc có chi phí thấp, giúp người phát triển dễ dàng thử nghiệm và lựa chọn mô hình phù hợp với từng nhu cầu phân tích mã nguồn. Điều này cũng tạo điều kiện thuận lợi cho việc mở rộng hệ thống trong tương lai khi cần tích hợp thêm các mô hình AI mới.

Groq là nền tảng cung cấp dịch vụ suy luận tốc độ cao cho các mô hình ngôn ngữ lớn. So với nhiều dịch vụ AI khác, Groq có ưu điểm nổi bật là thời gian phản hồi nhanh, giúp rút ngắn đáng kể thời gian phân tích mã nguồn và cải thiện trải nghiệm người dùng. Bên cạnh đó, Groq cũng cung cấp gói sử dụng miễn phí với giới hạn yêu cầu nhất định, phù hợp với các dự án nghiên cứu và đồ án tốt nghiệp. API của Groq được thiết kế đơn giản, dễ tích hợp và hỗ trợ nhiều mô hình mã nguồn mở thông dụng.

Việc kết hợp cả ba nền tảng mang lại nhiều lợi ích cho hệ thống. Gemini cung cấp khả năng phân tích và giải thích mã nguồn với chất lượng cao; OpenRouter giúp hệ thống dễ dàng chuyển đổi và thử nghiệm nhiều mô hình AI khác nhau; trong khi Groq đảm bảo tốc độ phản hồi nhanh trong quá trình suy luận. Đồng thời, cả ba nền tảng đều có gói sử dụng miễn phí, đáp ứng yêu cầu triển khai của đề tài mà không làm tăng chi phí phát triển. Nhờ đó, hệ thống vừa đảm bảo khả năng phân tích lỗi logic hiệu quả, vừa có tính linh hoạt và khả năng mở rộng trong tương lai.

1.3.5 Những thách thức khi sử dụng mô hình ngôn ngữ lớn

Bên cạnh những ưu điểm nổi bật, việc ứng dụng mô hình ngôn ngữ lớn trong phân tích mã nguồn vẫn tồn tại một số hạn chế. Mô hình có thể đưa ra các kết quả chưa chính xác hoặc giải thích chưa phù hợp nếu Prompt không đầy đủ hoặc dữ liệu đầu vào chưa rõ ràng. Hiện tượng này thường được gọi là ảo giác, tức mô hình sinh ra các thông tin có vẻ hợp lý nhưng không phản ánh đúng thực tế.

Ngoài ra, các vấn đề liên quan đến bảo mật và quyền riêng tư cũng cần được quan tâm khi sử dụng dịch vụ AI trực tuyến. Việc gửi mã nguồn lên máy chủ của nhà cung cấp có thể làm phát sinh nguy cơ lộ thông tin nếu không có cơ chế bảo vệ thích hợp. Vì vậy, trong quá trình xây dựng hệ thống, chỉ những thông tin cần thiết mới được gửi đến mô hình AI, đồng thời người dùng có thể chủ động lựa chọn dịch vụ AI phù hợp với nhu cầu sử dụng.

Mặc dù còn tồn tại một số hạn chế, các nghiên cứu gần đây cho thấy mô hình ngôn ngữ lớn vẫn là một trong những hướng tiếp cận hiệu quả trong việc hỗ trợ phân tích mã nguồn và phát hiện lỗi logic, đặc biệt khi được kết hợp với quá trình biên dịch và kiểm thử chương trình.

1.4 TRÍ TUỆ NHÂN TẠO TRONG PHÂN TÍCH MÃ NGUỒN

1.4.1 Phân tích mã nguồn truyền thống

Phân tích mã nguồn là quá trình kiểm tra chương trình nhằm phát hiện các lỗi hoặc các vấn đề có thể ảnh hưởng đến chất lượng phần mềm. Trong quá trình phát triển phần mềm, việc phân tích mã nguồn giúp lập trình viên phát hiện sớm các lỗi, từ đó giảm thời gian kiểm thử và nâng cao chất lượng sản phẩm.

Các công cụ phân tích truyền thống thường hoạt động theo hai phương pháp chính là phân tích tĩnh và phân tích động.

Phân tích tĩnh được thực hiện trực tiếp trên mã nguồn mà không cần chạy chương trình. Các công cụ như GCC, Clang hoặc Cppcheck sẽ kiểm tra cú pháp, kiểu dữ liệu, biến chưa khởi tạo, truy cập bộ nhớ không hợp lệ và một số cảnh báo về lập trình. Phương pháp này có ưu điểm là tốc độ nhanh và dễ triển khai, tuy nhiên khả năng phát hiện lỗi logic còn hạn chế do không đánh giá được kết quả thực thi của chương trình.

Ngược lại, phân tích động được thực hiện trong quá trình chạy chương trình. Hệ thống sẽ cung cấp dữ liệu đầu vào, theo dõi quá trình thực thi và thu thập kết quả đầu ra để đánh giá tính đúng đắn của chương trình. Phương pháp này có khả năng phát hiện nhiều lỗi hơn so với phân tích tĩnh, đặc biệt là các lỗi chỉ xuất hiện trong quá trình thực thi.

Đối với đề tài này, hệ thống sử dụng kết hợp cả hai phương pháp. Đầu tiên, mã nguồn được biên dịch bằng GCC để kiểm tra lỗi cú pháp và tạo tệp thực thi. Sau đó, chương trình được chạy với các Test Case do người dùng cung cấp để thu thập kết quả đầu ra. Đây là cơ sở để hệ thống tiếp tục phân tích lỗi logic bằng trí tuệ nhân tạo.

1.4.2 Phân tích mã nguồn bằng trí tuệ nhân tạo

Sự phát triển của các mô hình ngôn ngữ lớn đã mở ra hướng tiếp cận mới trong phân tích mã nguồn. Thay vì chỉ kiểm tra dựa trên các tập luật cố định, AI có khả năng hiểu ngữ cảnh chương trình, phân tích thuật toán và giải thích nguyên nhân gây lỗi bằng ngôn ngữ tự nhiên.

Khi nhận được mã nguồn và các thông tin liên quan, mô hình AI sẽ phân tích mối quan hệ giữa các biến, cấu trúc điều khiển, thuật toán và kết quả thực thi. Nếu phát hiện chương trình cho kết quả không đúng, mô hình sẽ xác định nguyên nhân có thể gây ra lỗi, chỉ ra vị trí cần sửa và đề xuất phương án khắc phục.

Ví dụ, đối với bài toán tính tổng các số từ 1 đến n , nếu chương trình sử dụng điều kiện vòng lặp $i < n$ thay vì $i \leq n$, AI không chỉ phát hiện kết quả sai mà còn có thể giải thích rằng chương trình đã bỏ qua giá trị cuối cùng của vòng lặp. Điều này giúp người dùng hiểu rõ bản chất của lỗi thay vì chỉ biết chương trình hoạt động không chính xác.

Ngoài khả năng phát hiện lỗi, AI còn có thể giải thích thuật toán, đưa ra nhận xét về cách viết mã nguồn và đề xuất các phương án cải tiến. Đây là ưu điểm nổi bật so với các công cụ phân tích truyền thống vốn chỉ đưa ra cảnh báo hoặc thông báo lỗi mà không giải thích chi tiết nguyên nhân.

1.4.3 Vai trò của AI trong hệ thống

Trong hệ thống được xây dựng, trí tuệ nhân tạo là thành phần thực hiện nhiệm vụ phân tích và gợi ý sửa lỗi logic cho chương trình C. Sau khi người dùng nhập mã

nguồn, hệ thống sẽ sử dụng trình biên dịch GCC để kiểm tra lỗi cú pháp. Nếu chương trình được biên dịch thành công, hệ thống tiếp tục thực thi chương trình với các Test Case và so sánh kết quả đầu ra với kết quả mong đợi.

Khi phát hiện kết quả không chính xác, hệ thống sẽ tổng hợp các thông tin bao gồm mã nguồn, dữ liệu đầu vào, kết quả đầu ra, kết quả mong đợi và một Prompt được xây dựng sẵn để gửi đến dịch vụ AI mà người dùng lựa chọn. Dịch vụ AI có thể là Gemini hoặc các mô hình được truy cập thông qua OpenRouter và Groq.

Sau khi nhận được Prompt, mô hình AI sẽ phân tích toàn bộ chương trình, xác định nguyên nhân gây lỗi, giải thích quá trình xử lý và đưa ra các gợi ý sửa lỗi. Kết quả phân tích sau đó được trả về Backend, lưu vào cơ sở dữ liệu SQLite và hiển thị trên giao diện Web để người dùng tham khảo.

Việc tích hợp AI vào hệ thống giúp nâng cao khả năng hỗ trợ người dùng trong quá trình học tập và phát triển chương trình. Thay vì chỉ thông báo chương trình cho kết quả sai, hệ thống có thể giải thích nguyên nhân, chỉ ra vị trí cần chỉnh sửa và đề xuất hướng khắc phục một cách trực quan. Điều này góp phần rút ngắn thời gian tìm lỗi, nâng cao hiệu quả học tập và hỗ trợ người dùng cải thiện kỹ năng lập trình.

1.4.4 Ưu điểm và hạn chế của việc sử dụng AI trong phân tích mã nguồn

Việc ứng dụng trí tuệ nhân tạo trong phân tích mã nguồn mang lại nhiều lợi ích. AI có khả năng hiểu ngữ cảnh của chương trình, phân tích thuật toán, phát hiện các lỗi logic khó nhận biết và giải thích kết quả bằng ngôn ngữ tự nhiên. Ngoài ra, AI còn giúp giảm thời gian tìm lỗi, hỗ trợ người dùng tiếp cận nhiều hướng giải quyết khác nhau và nâng cao hiệu quả học tập.

Tuy nhiên, AI vẫn tồn tại một số hạn chế. Chất lượng kết quả phụ thuộc vào Prompt và thông tin đầu vào mà hệ thống cung cấp. Trong một số trường hợp, AI có thể đưa ra các nhận định chưa chính xác hoặc đề xuất chưa tối ưu. Vì vậy, kết quả phân tích của AI chỉ mang tính chất hỗ trợ và người lập trình vẫn cần kiểm tra, đánh giá lại trước khi áp dụng vào chương trình thực tế.

Trong phạm vi đề tài, AI được sử dụng như một công cụ hỗ trợ phân tích và gợi ý sửa lỗi logic, kết hợp với quá trình biên dịch và kiểm thử chương trình nhằm nâng cao độ chính xác của kết quả phân tích. Cách tiếp cận này giúp tận dụng được

ưu điểm của cả phương pháp phân tích truyền thống và trí tuệ nhân tạo, từ đó nâng cao hiệu quả của hệ thống.

1.5 FLASK FRAMEWORK

1.5.1 Giới thiệu về Flask

Flask là một micro web framework được phát triển bằng ngôn ngữ lập trình Python, ra mắt lần đầu vào năm 2010 bởi Armin Ronacher. Khác với các framework có cấu trúc lớn như Django, Flask được thiết kế theo hướng tối giản, chỉ cung cấp những thành phần cần thiết để xây dựng ứng dụng Web. Điều này giúp lập trình viên dễ dàng tùy chỉnh, mở rộng và tích hợp các thư viện bên ngoài phù hợp với từng yêu cầu cụ thể của dự án.

Flask hoạt động theo mô hình Client–Server. Khi người dùng gửi yêu cầu từ trình duyệt Web, Flask sẽ tiếp nhận yêu cầu, xử lý dữ liệu, tương tác với cơ sở dữ liệu hoặc các dịch vụ bên ngoài, sau đó trả kết quả về cho người dùng dưới dạng trang HTML hoặc dữ liệu JSON.

Nhờ đặc điểm nhẹ, dễ học và có cộng đồng phát triển lớn, Flask hiện được sử dụng rộng rãi trong việc xây dựng các hệ thống Web, RESTful API, ứng dụng học máy và các hệ thống tích hợp trí tuệ nhân tạo.

1.5.2 Đặc điểm của Flask

Flask sở hữu nhiều đặc điểm phù hợp với việc phát triển các ứng dụng Web quy mô vừa và nhỏ.

Thứ nhất, Flask có kiến trúc đơn giản và linh hoạt. Framework không áp đặt cấu trúc thư mục cố định, giúp lập trình viên chủ động tổ chức mã nguồn theo yêu cầu của dự án. Điều này đặc biệt phù hợp với các đề tài nghiên cứu khi hệ thống cần được mở rộng hoặc thay đổi trong quá trình phát triển.

Thứ hai, Flask hỗ trợ xây dựng RESTful API một cách thuận tiện. Các API được định nghĩa thông qua cơ chế Route, cho phép ánh xạ giữa đường dẫn URL và các hàm xử lý tương ứng. Nhờ đó, việc xây dựng các chức năng như đăng nhập, tải tệp mã nguồn, phân tích chương trình hoặc lưu kết quả trở nên đơn giản và dễ quản lý.

Thứ ba, Flask có khả năng tích hợp tốt với nhiều thư viện của Python. Trong đề tài này, Flask được kết hợp với thư viện sqlite3 để làm việc với cơ sở dữ liệu SQLite, thư viện subprocess để gọi trình biên dịch GCC và các thư viện HTTP để gửi yêu cầu đến các dịch vụ AI như Gemini, OpenRouter và Groq.

Ngoài ra, Flask còn hỗ trợ nhiều tiện ích như quản lý phiên đăng nhập, xử lý biểu mẫu, tải tệp lên máy chủ và trả dữ liệu dưới dạng JSON, đáp ứng đầy đủ các yêu cầu của hệ thống được xây dựng trong đề tài.

1.5.3 Lý do lựa chọn Flask

Có nhiều framework Python có thể được sử dụng để phát triển ứng dụng Web như Django, FastAPI hay Flask. Tuy nhiên, sau khi phân tích yêu cầu của đề tài, Flask được lựa chọn vì những lý do sau:

Thứ nhất, Flask có cấu trúc đơn giản, dễ học và dễ triển khai. Điều này giúp rút ngắn thời gian phát triển hệ thống và phù hợp với phạm vi của đề án tốt nghiệp.

Thứ hai, Flask hỗ trợ xây dựng RESTful API hiệu quả. Hầu hết các chức năng của hệ thống như đăng nhập, tải mã nguồn, thêm Test Case, phân tích chương trình và lưu kết quả đều được triển khai dưới dạng API.

Thứ ba, Flask dễ dàng tích hợp với các thư viện của Python cũng như các dịch vụ AI thông qua HTTP API. Đây là yêu cầu quan trọng của đề tài khi hệ thống cần kết nối với Gemini, OpenRouter và Groq để thực hiện phân tích lỗi logic.

Thứ tư, Flask hoạt động tốt với SQLite mà không cần cài đặt thêm máy chủ cơ sở dữ liệu, giúp việc triển khai hệ thống trở nên đơn giản và tiết kiệm tài nguyên.

Cuối cùng, Flask có cộng đồng sử dụng lớn, tài liệu phong phú và được cập nhật thường xuyên, giúp việc phát triển và bảo trì hệ thống thuận lợi hơn.

Từ những ưu điểm trên, Flask đáp ứng tốt các yêu cầu về hiệu năng, khả năng mở rộng và tích hợp của hệ thống phân tích và gợi ý sửa lỗi logic mã nguồn C.

1.5.4 Vai trò của Flask trong hệ thống

Trong hệ thống được xây dựng, Flask đóng vai trò là Backend, chịu trách nhiệm xử lý toàn bộ các yêu cầu từ phía người dùng.

Khi người dùng gửi yêu cầu từ giao diện Web, Flask tiếp nhận dữ liệu và thực hiện các bước xử lý tương ứng. Đối với chức năng đăng nhập, Flask kiểm tra thông tin tài khoản trong cơ sở dữ liệu SQLite trước khi cho phép người dùng truy cập hệ thống.

Khi người dùng thực hiện phân tích mã nguồn, Flask tiếp nhận mã nguồn C, Test Case và lựa chọn mô hình AI. Sau đó, Flask sử dụng thư viện subprocess để gọi trình biên dịch GCC nhằm biên dịch và thực thi chương trình. Kết quả thực thi được so sánh với kết quả mong đợi để xác định chương trình có xuất hiện lỗi logic hay không.

Nếu phát hiện kết quả sai, Flask sẽ tổng hợp mã nguồn, Test Case, kết quả đầu ra và Prompt rồi gửi đến dịch vụ AI thông qua API. Sau khi nhận được kết quả phân tích từ AI, Flask tiếp tục xử lý dữ liệu, lưu kết quả vào cơ sở dữ liệu SQLite và trả về giao diện Web dưới dạng JSON để hiển thị cho người dùng.

Có thể thấy rằng Flask là thành phần trung tâm của toàn bộ hệ thống, đóng vai trò kết nối giữa giao diện người dùng, cơ sở dữ liệu, trình biên dịch GCC và các dịch vụ trí tuệ nhân tạo. Mọi luồng xử lý đều được điều phối thông qua Flask, đảm bảo hệ thống hoạt động ổn định và đáp ứng đúng các yêu cầu chức năng đã đề ra.

1.6 SQLITE

1.6.1 Giới thiệu về SQLite

SQLite là hệ quản trị cơ sở dữ liệu quan hệ được phát triển theo mô hình nhúng. Khác với các hệ quản trị cơ sở dữ liệu như MySQL hay PostgreSQL, SQLite không hoạt động dưới dạng một máy chủ độc lập mà được tích hợp trực tiếp vào ứng dụng thông qua một thư viện. Toàn bộ dữ liệu được lưu trữ trong một tệp duy nhất trên hệ thống, giúp việc triển khai và quản lý cơ sở dữ liệu trở nên đơn giản hơn [8].

SQLite hỗ trợ hầu hết các tính năng cơ bản của một hệ quản trị cơ sở dữ liệu quan hệ như tạo bảng, truy vấn dữ liệu, cập nhật, xóa dữ liệu, tạo chỉ mục, khóa ngoại và quản lý giao dịch. Nhờ đáp ứng đầy đủ các chuẩn SQL thông dụng, SQLite được sử dụng rộng rãi trong các ứng dụng máy tính, thiết bị di động, trình duyệt Web và các hệ thống có quy mô nhỏ hoặc trung bình.

Đối với các dự án nghiên cứu và đồ án tốt nghiệp, SQLite là một lựa chọn phù hợp nhờ ưu điểm dễ triển khai, không cần cài đặt máy chủ riêng và sử dụng ít tài nguyên hệ thống.

1.6.2 Đặc điểm của SQLite

SQLite có nhiều đặc điểm nổi bật giúp nó trở thành một trong những hệ quản trị cơ sở dữ liệu phổ biến hiện nay.

Thứ nhất, SQLite không yêu cầu cài đặt máy chủ cơ sở dữ liệu. Toàn bộ dữ liệu được quản lý thông qua một tệp có phần mở rộng .db hoặc .sqlite, giúp giảm đáng kể chi phí triển khai và cấu hình hệ thống.

Thứ hai, SQLite có dung lượng nhỏ và tiêu thụ ít tài nguyên. Thư viện SQLite chỉ chiếm vài MB nhưng vẫn cung cấp đầy đủ các chức năng quản lý dữ liệu cần thiết. Điều này giúp hệ thống hoạt động ổn định ngay cả trên các máy tính có cấu hình không cao.

Thứ ba, SQLite hỗ trợ đầy đủ các thao tác SQL như SELECT, INSERT, UPDATE, DELETE, JOIN, ORDER BY, GROUP BY và nhiều tính năng khác. Nhờ đó, việc xây dựng các chức năng quản lý dữ liệu trong hệ thống được thực hiện thuận tiện.

Ngoài ra, SQLite còn hỗ trợ cơ chế giao dịch (Transaction), giúp đảm bảo tính toàn vẹn của dữ liệu trong quá trình thêm, sửa hoặc xóa thông tin.

Bên cạnh những ưu điểm trên, SQLite cũng tồn tại một số hạn chế. Do không hoạt động theo mô hình máy chủ nên SQLite không phù hợp với các hệ thống có số lượng lớn người dùng truy cập và ghi dữ liệu đồng thời. Tuy nhiên, hạn chế này không ảnh hưởng nhiều đến phạm vi của đề tài vì hệ thống chủ yếu phục vụ mục đích nghiên cứu và có số lượng người dùng không lớn.

Có nhiều hệ quản trị cơ sở dữ liệu có thể được sử dụng trong hệ thống như MySQL, PostgreSQL hoặc MongoDB. Tuy nhiên, đề tài lựa chọn SQLite vì những lý do sau:

Thứ nhất, SQLite có quy trình triển khai đơn giản, không yêu cầu cài đặt hoặc cấu hình máy chủ cơ sở dữ liệu. Điều này giúp giảm thời gian phát triển và thuận tiện trong quá trình triển khai hệ thống.

Thứ hai, SQLite tích hợp sẵn với Python thông qua thư viện sqlite3, cho phép Flask dễ dàng thực hiện các thao tác truy vấn và quản lý dữ liệu mà không cần cài đặt thêm thư viện bên ngoài.

Thứ ba, dữ liệu được lưu trữ trong một tệp duy nhất, giúp việc sao lưu, di chuyển và phục hồi cơ sở dữ liệu trở nên đơn giản hơn. Người phát triển chỉ cần sao chép tệp cơ sở dữ liệu là có thể triển khai hệ thống trên một môi trường khác.

Thứ tư, quy mô của hệ thống trong đề tài không lớn, số lượng người dùng đồng thời thấp và dữ liệu lưu trữ chủ yếu bao gồm thông tin tài khoản, Test Case và lịch sử phân tích. Vì vậy, SQLite hoàn toàn đáp ứng được yêu cầu về hiệu năng và khả năng lưu trữ.

Ngoài ra, SQLite là phần mềm mã nguồn mở và được sử dụng miễn phí, phù hợp với các dự án nghiên cứu, học tập và phát triển thử nghiệm.

1.6.3 Vai trò của SQLite trong hệ thống

Trong hệ thống được xây dựng, SQLite đóng vai trò lưu trữ toàn bộ dữ liệu phục vụ quá trình hoạt động của ứng dụng.

Đối với chức năng đăng nhập, SQLite lưu trữ thông tin tài khoản người dùng, bao gồm tên đăng nhập và mật khẩu đã được mã hóa. Khi người dùng đăng nhập vào hệ thống, Flask sẽ truy vấn cơ sở dữ liệu để xác thực thông tin trước khi cấp quyền truy cập.

Bên cạnh đó, SQLite còn lưu trữ các Test Case mà người dùng tạo trong quá trình kiểm thử chương trình. Các Test Case này bao gồm dữ liệu đầu vào và kết quả mong đợi, là cơ sở để hệ thống đánh giá tính đúng đắn của chương trình sau khi thực thi.

Sau mỗi lần phân tích mã nguồn, hệ thống sẽ lưu các thông tin như mã nguồn C, kết quả thực thi, kết quả phân tích của AI, thời gian thực hiện và mô hình AI được sử dụng vào cơ sở dữ liệu. Việc lưu trữ này giúp người dùng có thể theo dõi và quản lý các kết quả phân tích trong những lần sử dụng tiếp theo.

Thông qua SQLite, toàn bộ dữ liệu của hệ thống được quản lý tập trung, đảm bảo tính nhất quán và hỗ trợ các chức năng truy xuất thông tin nhanh chóng.

1.6.4 Môi liên hệ giữa SQLite và hệ thống

Trong kiến trúc của hệ thống, SQLite được tích hợp trực tiếp với Flask thông qua thư viện sqlite3. Mọi thao tác thêm, sửa, xóa và truy vấn dữ liệu đều được thực hiện bởi Backend.

Khi người dùng gửi yêu cầu từ giao diện Web, Flask sẽ xử lý dữ liệu và thực hiện các truy vấn tương ứng đến SQLite. Sau khi nhận được kết quả, Backend tiếp tục xử lý và trả dữ liệu về giao diện để hiển thị.

Quá trình này diễn ra trong hầu hết các chức năng của hệ thống như đăng nhập, quản lý Test Case, lưu kết quả phân tích và hiển thị lịch sử phân tích. Việc sử dụng SQLite giúp hệ thống hoạt động ổn định, dễ triển khai và đáp ứng đầy đủ các yêu cầu về lưu trữ dữ liệu trong phạm vi của đề tài.

CHƯƠNG 2 PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1 MÔ TẢ BÀI TOÁN

Trong quá trình học tập môn lập trình C, học sinh thường được giao các bài tập nhằm rèn luyện tư duy thuật toán và kỹ năng lập trình. Sau khi hoàn thành chương trình, học sinh phải tự kiểm tra kết quả hoặc nhờ giáo viên đánh giá. Tuy nhiên, việc xác định nguyên nhân gây ra lỗi logic trong chương trình thường gặp nhiều khó khăn do trình biên dịch chỉ phát hiện được lỗi cú pháp, trong khi các lỗi logic chỉ được bộc lộ khi chương trình được thực thi với các bộ dữ liệu kiểm thử.

Bên cạnh đó, các hệ thống chấm bài trực tuyến hiện nay chủ yếu đánh giá kết quả thông qua Test Case và chỉ trả về trạng thái đúng hoặc sai. Người học thường không biết nguyên nhân vì sao chương trình chưa đạt yêu cầu, dẫn đến mất nhiều thời gian trong quá trình tìm lỗi và sửa chương trình. Đối với giáo viên, việc xây dựng đề bài, quản lý Test Case và hỗ trợ giải đáp cho nhiều học sinh cũng đòi hỏi nhiều thời gian và công sức.

Xuất phát từ thực tế trên, đề tài xây dựng một hệ thống hỗ trợ học lập trình ngôn ngữ C trên nền tảng Web, kết hợp giữa phương pháp đánh giá bằng Test Case và trí tuệ nhân tạo nhằm hỗ trợ phát hiện và gợi ý sửa lỗi logic trong chương trình. Hệ thống được xây dựng với ba nhóm người dùng gồm Quản trị viên (Admin), Giáo viên và Học sinh, trong đó mỗi nhóm người dùng được cấp các quyền hạn và chức năng phù hợp với vai trò của mình.

Quản trị viên có nhiệm vụ quản lý tài khoản người dùng, bao gồm tạo mới, cập nhật và phân quyền cho giáo viên và học sinh, đồng thời quản lý các thông tin chung của hệ thống nhằm đảm bảo hệ thống hoạt động ổn định.

Giáo viên có chức năng tạo và quản lý các đề bài lập trình, xây dựng bộ Test Case tương ứng với từng đề bài và theo dõi kết quả thực hiện của học sinh. Mỗi đề bài sẽ bao gồm phần mô tả yêu cầu, dữ liệu đầu vào, dữ liệu đầu ra và bộ Test Case được sử dụng để đánh giá chương trình.

Học sinh sau khi đăng nhập vào hệ thống sẽ lựa chọn đề bài cần thực hiện. Người dùng có thể nhập trực tiếp mã nguồn C trên trình soạn thảo của hệ thống hoặc tải lên tệp mã nguồn từ máy tính. Sau khi hoàn thành chương trình, học sinh sử dụng

chức năng Biên dịch để kiểm tra lỗi cú pháp của chương trình thông qua trình biên dịch GCC. Nếu chương trình được biên dịch thành công, người dùng tiếp tục sử dụng chức năng Chạy để thực thi chương trình với bộ Test Case của đề bài.

Trong quá trình thực thi, hệ thống sẽ lần lượt đưa từng Test Case vào chương trình, thu nhận kết quả đầu ra và so sánh với kết quả mong đợi đã được giáo viên thiết lập. Nếu tất cả các Test Case đều đạt yêu cầu, chương trình được đánh giá là đúng. Ngược lại, nếu có Test Case không đạt, hệ thống sẽ xác định chương trình còn tồn tại lỗi logic hoặc chưa đáp ứng đầy đủ yêu cầu của bài toán.

Để hỗ trợ người học trong quá trình sửa lỗi, hệ thống tích hợp các mô hình trí tuệ nhân tạo thông qua API như Gemini, OpenRouter và Groq. Khi người dùng sử dụng chức năng AI Help, hệ thống sẽ gửi mã nguồn chương trình cùng với thông tin về đề bài và kết quả thực thi đến mô hình AI để phân tích. AI sẽ đánh giá thuật toán, xác định nguyên nhân có thể dẫn đến lỗi logic và đưa ra các gợi ý sửa lỗi phù hợp mà không làm thay toàn bộ lời giải của người học.

Toàn bộ kết quả thực hiện, bao gồm thông tin bài làm, kết quả biên dịch, kết quả chạy Test Case và lịch sử sử dụng AI hỗ trợ đều được lưu trữ trong cơ sở dữ liệu SQLite. Điều này giúp giáo viên dễ dàng theo dõi quá trình học tập của học sinh, đồng thời tạo cơ sở để đánh giá mức độ hoàn thành bài tập cũng như hiệu quả sử dụng hệ thống.

Thông qua việc kết hợp giữa biên dịch chương trình, kiểm thử bằng Test Case và khả năng phân tích của trí tuệ nhân tạo, hệ thống không chỉ giúp đánh giá tính đúng đắn của chương trình mà còn hỗ trợ người học hiểu rõ nguyên nhân gây ra lỗi logic và từng bước cải thiện kỹ năng lập trình.

2.2 PHÂN TÍCH YÊU CẦU

2.2.1 Yêu cầu chức năng

Hệ thống được xây dựng nhằm hỗ trợ quá trình học tập và giảng dạy lập trình ngôn ngữ C thông qua việc kiểm tra chương trình bằng Test Case kết hợp với trí tuệ nhân tạo hỗ trợ phân tích lỗi logic. Hệ thống gồm ba nhóm người dùng chính là Quản trị viên, Giáo viên và Học sinh. Mỗi nhóm người dùng được cấp quyền truy cập và sử dụng các chức năng khác nhau.

❖ Chức năng quản lý người dùng

Hệ thống cho phép người dùng đăng nhập bằng tài khoản đã được cấp. Sau khi đăng nhập thành công, hệ thống sẽ phân quyền tương ứng với vai trò của từng người dùng.

Quản trị viên có quyền quản lý toàn bộ tài khoản trong hệ thống, bao gồm thêm mới, chỉnh sửa, khóa hoặc xóa tài khoản giáo viên và học sinh. Đồng thời, quản trị viên có thể theo dõi hoạt động của hệ thống nhằm đảm bảo hệ thống hoạt động ổn định.

❖ Chức năng quản lý đề bài

Giáo viên có quyền tạo mới đề bài lập trình, chỉnh sửa nội dung đề bài hoặc xóa đề bài không còn sử dụng. Mỗi đề bài bao gồm tên đề bài, nội dung mô tả, yêu cầu đầu vào, yêu cầu đầu ra và các thông tin cần thiết phục vụ quá trình làm bài.

Các đề bài sau khi được tạo sẽ được hiển thị cho học sinh để lựa chọn và thực hiện.

❖ Chức năng quản lý Test Case

Đối với mỗi đề bài, giáo viên có thể xây dựng một hoặc nhiều Test Case nhằm kiểm tra tính chính xác của chương trình.

Mỗi Test Case bao gồm dữ liệu đầu vào và kết quả đầu ra mong đợi. Bộ Test Case được sử dụng trong quá trình đánh giá kết quả thực hiện của học sinh.

❖ Chức năng thực hiện bài tập

Học sinh có thể lựa chọn đề bài từ danh sách đề bài được cung cấp trên hệ thống. Sau khi chọn đề bài, người dùng có thể nhập trực tiếp mã nguồn C vào trình soạn thảo hoặc tải lên tệp mã nguồn từ máy tính.

Hệ thống cung cấp môi trường làm bài trực tuyến giúp người dùng dễ dàng chỉnh sửa và cập nhật chương trình trong quá trình thực hiện.

❖ Chức năng biên dịch chương trình

Sau khi hoàn thành chương trình, học sinh có thể sử dụng chức năng biên dịch để kiểm tra lỗi cú pháp của mã nguồn.

Hệ thống sử dụng trình biên dịch GCC để thực hiện quá trình biên dịch. Nếu chương trình chứa lỗi cú pháp hoặc lỗi biên dịch, hệ thống sẽ hiển thị thông báo lỗi để người dùng chỉnh sửa.

❖ Chức năng thực thi chương trình và đánh giá bằng Test Case

Khi chương trình được biên dịch thành công, người dùng có thể sử dụng chức năng chạy chương trình.

Hệ thống sẽ thực thi chương trình với từng Test Case tương ứng của đề bài, thu nhận kết quả đầu ra và so sánh với kết quả mong đợi.

Kết quả đánh giá được hiển thị dưới dạng đạt hoặc không đạt đối với từng Test Case, giúp người dùng xác định mức độ chính xác của chương trình.

❖ Chức năng hỗ trợ phân tích bằng AI

Trong trường hợp chương trình chưa vượt qua các Test Case hoặc người dùng cần hỗ trợ, hệ thống cung cấp chức năng AI Help.

Hệ thống gửi mã nguồn, thông tin đề bài và kết quả thực thi đến mô hình trí tuệ nhân tạo để phân tích. AI sẽ đưa ra nhận xét về nguyên nhân có thể gây ra lỗi logic và đề xuất hướng sửa lỗi nhằm hỗ trợ người học cải thiện chương trình.

❖ Chức năng lưu trữ kết quả

Kết quả biên dịch, kết quả chạy chương trình, trạng thái Test Case và lịch sử sử dụng AI được lưu trữ trong cơ sở dữ liệu.

Thông tin này giúp giáo viên theo dõi quá trình học tập của học sinh và hỗ trợ công tác đánh giá kết quả học tập.

2.2.2 Yêu cầu phi chức năng

❖ Tính bảo mật

Hệ thống phải đảm bảo chỉ những người dùng đã được cấp tài khoản mới có thể truy cập vào hệ thống. Mỗi nhóm người dùng chỉ được phép sử dụng các chức năng tương ứng với quyền hạn được phân công.

❖ Hiệu năng

Quá trình biên dịch và thực thi chương trình cần được thực hiện trong thời gian ngắn nhằm đảm bảo trải nghiệm sử dụng của người dùng. Hệ thống phải phản hồi nhanh đối với các thao tác truy cập và xử lý dữ liệu.

❖ Tính ổn định

Hệ thống phải hoạt động ổn định trong quá trình sử dụng, hạn chế tối đa các lỗi gây gián đoạn quá trình làm bài hoặc quản lý dữ liệu.

❖ Khả năng mở rộng

Hệ thống cần được thiết kế theo hướng dễ dàng bổ sung thêm đề bài, Test Case, chức năng mới hoặc tích hợp thêm các mô hình AI khác trong tương lai.

❖ Tính dễ sử dụng

Giao diện hệ thống cần được thiết kế trực quan, thân thiện và dễ thao tác đối với cả giáo viên và học sinh. Các chức năng chính phải được bố trí rõ ràng nhằm hỗ trợ người dùng sử dụng hệ thống một cách thuận tiện.

❖ Khả năng bảo trì

Mã nguồn hệ thống cần được tổ chức hợp lý để thuận tiện cho việc bảo trì, sửa lỗi và nâng cấp trong quá trình vận hành sau này.

2.3 PHÂN TÍCH TÁC NHÂN VÀ USECASE

2.3.1 Danh sách các tác nhân

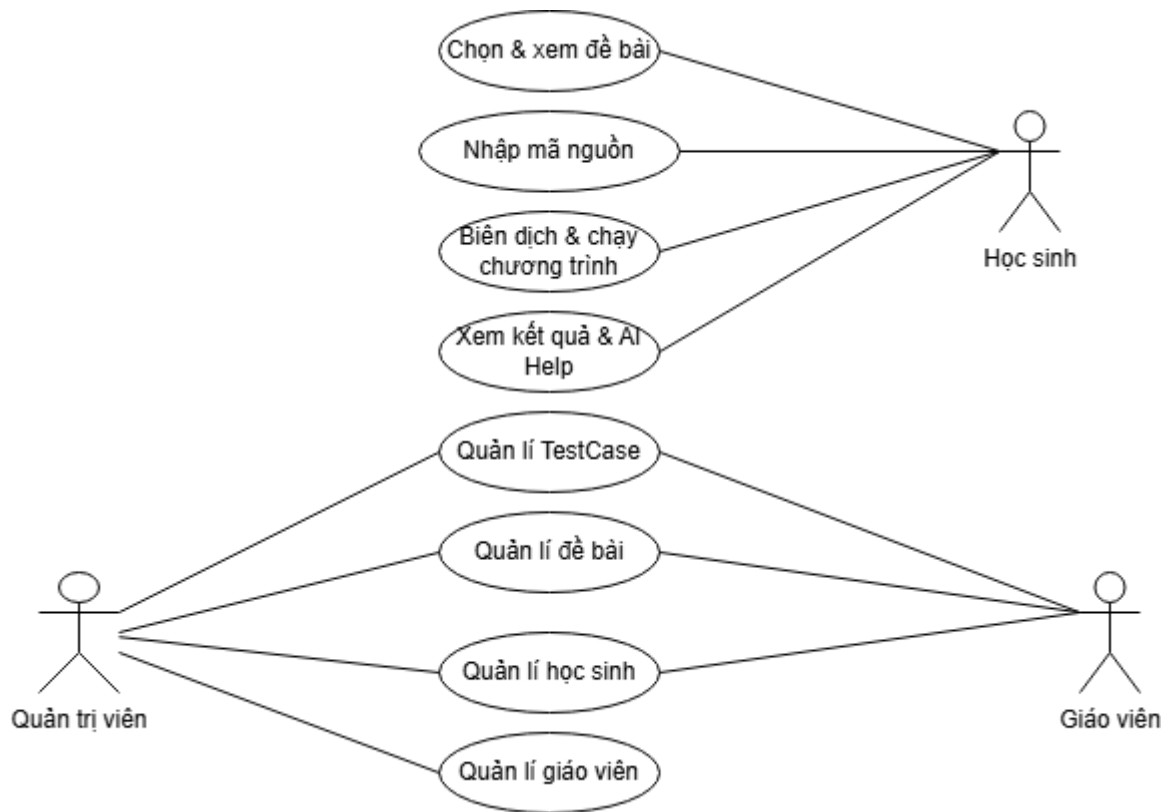
Bảng 2.1 Danh sách các tác nhân

Tác nhân	Vai trò
Học sinh	Làm bài, nộp mã nguồn, xem kết quả và nhận gợi ý sửa lỗi
Giáo viên	Quản lý đề bài, test case và xem bài làm học sinh
Quản trị viên	Quản lý tài khoản, người dùng, phân quyền và hệ thống

2.3.2 Use case tổng quát

Use Case Diagram được sử dụng nhằm mô tả các chức năng của hệ thống và sự tương tác giữa người sử dụng với hệ thống. Thông qua sơ đồ Use Case, có thể xác định rõ phạm vi chức năng của hệ thống cũng như quyền hạn của từng nhóm người

dùng. Trong hệ thống "Xây dựng hệ thống phân tích và gợi ý sửa lỗi logic mã nguồn C", có ba nhóm người dùng chính gồm Quản trị viên, Giáo viên và Học sinh. Mỗi nhóm người dùng được cấp những quyền sử dụng khác nhau nhằm đảm bảo tính bảo mật và phù hợp với chức năng của từng đối tượng.



Hình 2.1 Use Case Diagram của hệ thống

Qua sơ đồ Use Case có thể thấy hệ thống được chia thành ba nhóm chức năng chính: nhóm chức năng dành cho học sinh, nhóm chức năng dành cho giáo viên và nhóm chức năng quản trị hệ thống.

Học sinh: là đối tượng sử dụng chính của hệ thống, thực hiện các bài tập lập trình C do giáo viên cung cấp sau khi đăng nhập. Học sinh có thể chọn và xem đề bài, nhập hoặc tải lên mã nguồn C, tiến hành biên dịch và chạy chương trình để kiểm tra kết quả. Hệ thống cũng cho phép học sinh xem kết quả thực thi và sử dụng chức năng AI Help để hỗ trợ phân tích lỗi và gợi ý sửa lỗi trong quá trình học tập.

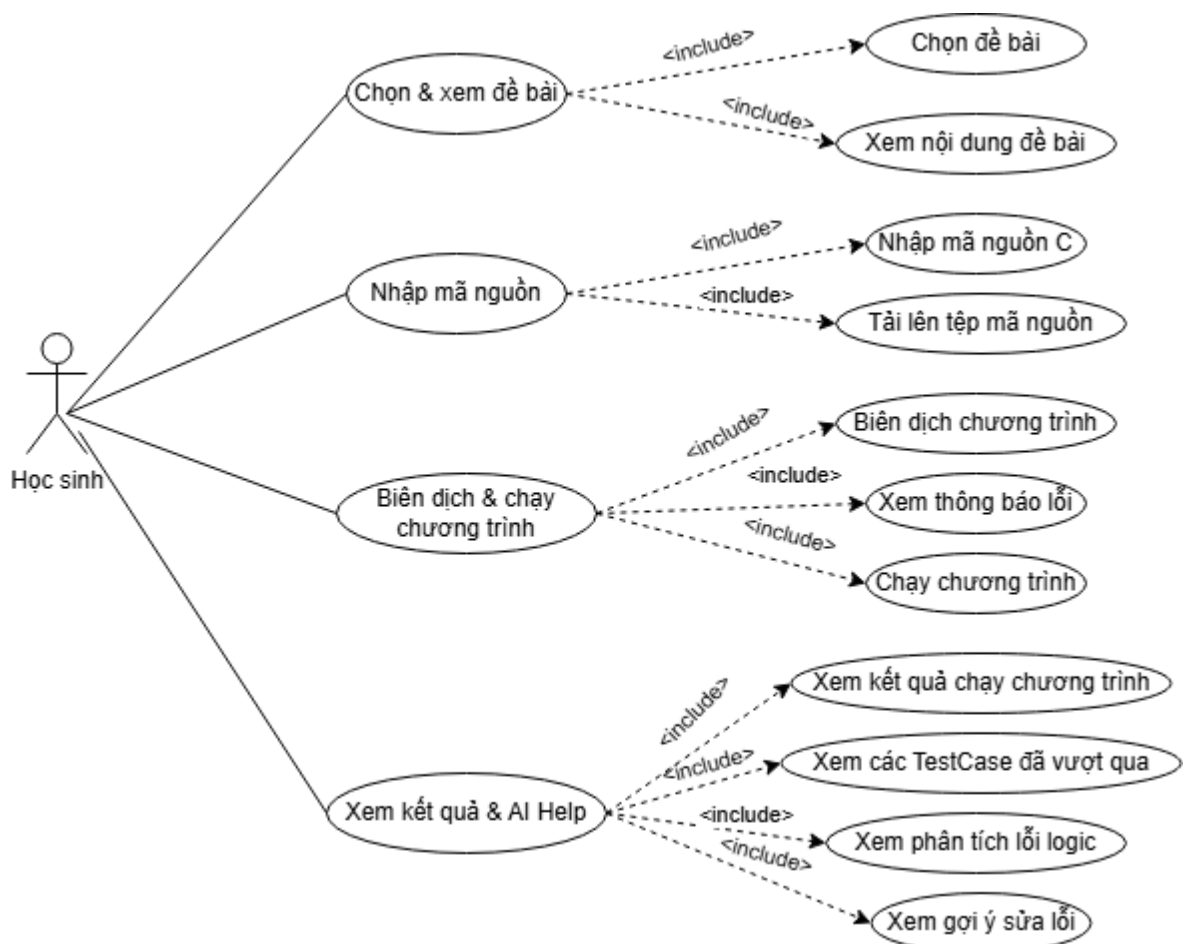
Giáo viên: là người chịu trách nhiệm xây dựng hệ thống bài tập và theo dõi kết quả học tập của học sinh, đồng thời quản lý nội dung giảng dạy và quá trình đánh giá bài làm. Các chức năng chính của giáo viên bao gồm quản lý đề bài, quản lý Test

Case và xem bài làm của học sinh nhằm phục vụ việc kiểm tra và đánh giá kết quả học tập.

Quản trị viên: là người có quyền cao nhất trong hệ thống, chịu trách nhiệm quản lý toàn bộ người dùng và đảm bảo hệ thống hoạt động ổn định. Bên cạnh các chức năng tương tự giáo viên, quản trị viên còn có quyền kiểm soát và quản lý toàn bộ tài khoản trong hệ thống nhằm đảm bảo tính an toàn và phân quyền hợp lý. Cụ thể, các chức năng của quản trị viên bao gồm quản lý học sinh, quản lý giáo viên, quản lý đề bài và quản lý Test Case.

2.3.3 Usecase của học sinh

Học sinh là đối tượng sử dụng chính của hệ thống. Sau khi đăng nhập thành công, học sinh có thể lựa chọn đề bài lập trình do giáo viên cung cấp và thực hiện các thao tác để hoàn thành bài tập.



Hình 2.2 Usecase phân rã theo tác nhân học sinh

Chọn và xem đề bài: Học sinh có thể xem danh sách các đề bài hiện có trong hệ thống và lựa chọn bài tập phù hợp. Sau khi chọn đề bài, hệ thống sẽ hiển thị đầy

đủ thông tin gồm tiêu đề, mô tả bài toán, yêu cầu đầu vào, đầu ra, mức độ khó và đoạn mã khởi động (mã chuẩn) nếu có. Sau khi lựa chọn đề bài, học sinh được phép xem chi tiết nội dung của bài tập để hiểu rõ yêu cầu trước khi tiến hành lập trình. Nội dung đề bài bao gồm mô tả bài toán, dữ liệu đầu vào, dữ liệu đầu ra và các yêu cầu cần đạt của chương trình.

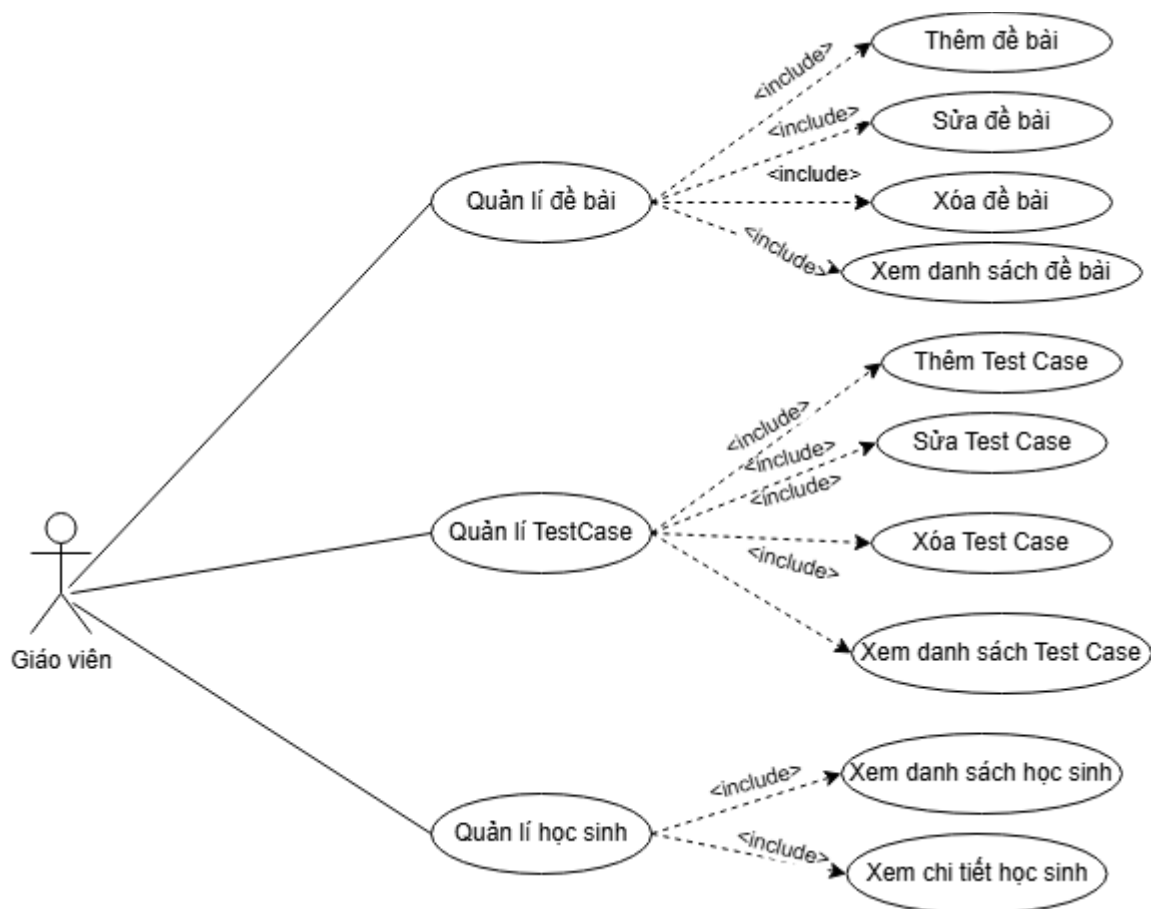
Nhập mã nguồn: Hệ thống cung cấp trình soạn thảo mã nguồn trực tiếp trên giao diện Web, cho phép học sinh viết chương trình C mà không cần sử dụng phần mềm bên ngoài. Trình soạn thảo hỗ trợ hiển thị mã nguồn rõ ràng, giúp người học dễ dàng chỉnh sửa và theo dõi chương trình. Ngoài việc nhập trực tiếp, học sinh còn có thể tải lên tệp chương trình C đã được viết sẵn trên máy tính. Hệ thống sẽ đọc nội dung tệp và hiển thị trong trình soạn thảo để người dùng tiếp tục chỉnh sửa hoặc thực hiện các chức năng khác.

Biên dịch và chạy chương trình: Sau khi hoàn thành chương trình, học sinh sử dụng chức năng “Biên dịch” để kiểm tra lỗi cú pháp. Hệ thống sẽ sử dụng trình biên dịch GCC để biên dịch mã nguồn. Nếu chương trình có lỗi, hệ thống sẽ hiển thị thông báo lỗi để học sinh chỉnh sửa. Nếu biên dịch thành công, chương trình sẽ sẵn sàng cho bước thực thi. Sau khi ấn nút chạy chương trình, hệ thống sẽ tự động lấy bộ Test Case tương ứng với đề bài và lần lượt truyền dữ liệu đầu vào vào chương trình. Kết quả đầu ra của chương trình sẽ được so sánh với kết quả mong đợi nhằm đánh giá tính chính xác của thuật toán.

Xem kết quả và AI Help: Sau khi chương trình được thực thi, học sinh có thể xem kết quả biên dịch, trạng thái từng Test Case, số lượng Test Case đạt và không đạt cũng như kết quả đầu ra của chương trình. Thông tin này giúp học sinh xác định chương trình còn tồn tại lỗi ở những trường hợp nào để tiếp tục hoàn thiện. Khi học sinh lựa chọn “AI Help”, hệ thống sẽ gửi nội dung đề bài, mã nguồn chương trình và kết quả thực hiện Test Case đến mô hình trí tuệ nhân tạo. AI sẽ phân tích chương trình, xác định nguyên nhân gây lỗi logic, giải thích các lỗi phát hiện được và đưa ra các gợi ý giúp học sinh sửa chương trình mà không cung cấp trực tiếp lời giải hoàn chỉnh.

2.3.4 Usecase của giáo viên

Các chức năng của giáo viên tập trung vào việc quản lý nội dung giảng dạy và đánh giá kết quả làm bài.



Hình 2.3 Usecase phân rã theo tác nhân giáo viên

Quản lý đề bài

Giáo viên có quyền tạo mới, chỉnh sửa hoặc cập nhật các đề bài lập trình trong hệ thống. Đối với mỗi đề bài, giáo viên nhập các thông tin như tiêu đề, mô tả bài toán, yêu cầu đầu vào, đầu ra, mã chuẩn và mức độ khó.

Quản lý Test Case

Sau khi tạo đề bài, giáo viên xây dựng bộ Test Case tương ứng. Mỗi Test Case bao gồm dữ liệu đầu vào và kết quả đầu ra mong đợi. Bộ Test Case được hệ thống sử dụng để kiểm tra tính đúng đắn của chương trình khi học sinh thực hiện chức năng chạy chương trình.

Xem bài làm học sinh

Giáo viên có thể xem danh sách các bài nộp của học sinh, theo dõi trạng thái biên dịch, kết quả thực hiện Test Case và thời gian nộp bài. Thông qua đó, giáo viên có thể đánh giá mức độ hoàn thành của từng học sinh cũng như xác định các lỗi mà học sinh thường gặp trong quá trình học tập.

2.3.5 Usecase của quản trị viên

Quản trị viên có thể tạo mới, cập nhật, khóa hoặc kích hoạt các tài khoản người dùng trong hệ thống. Đồng thời, quản trị viên có quyền phân quyền tài khoản theo các vai trò như quản trị viên, giáo viên hoặc học sinh.



Hình 2.4 Usecase phân rã theo tác nhân quản trị viên

Quản lý học sinh

Quản trị viên thực hiện việc chuyển đổi vai trò giữa học sinh và giáo viên, xóa thông tin học sinh khi cần thiết. Đồng thời có thể theo dõi danh sách học sinh đã đăng ký sử dụng hệ thống.

Quản lý giáo viên

Quản trị viên có quyền quản lý thông tin của các giáo viên, bao gồm chuyển đổi vai trò giữa học sinh và giáo viên, xóa tài khoản

Quản lý đề bài

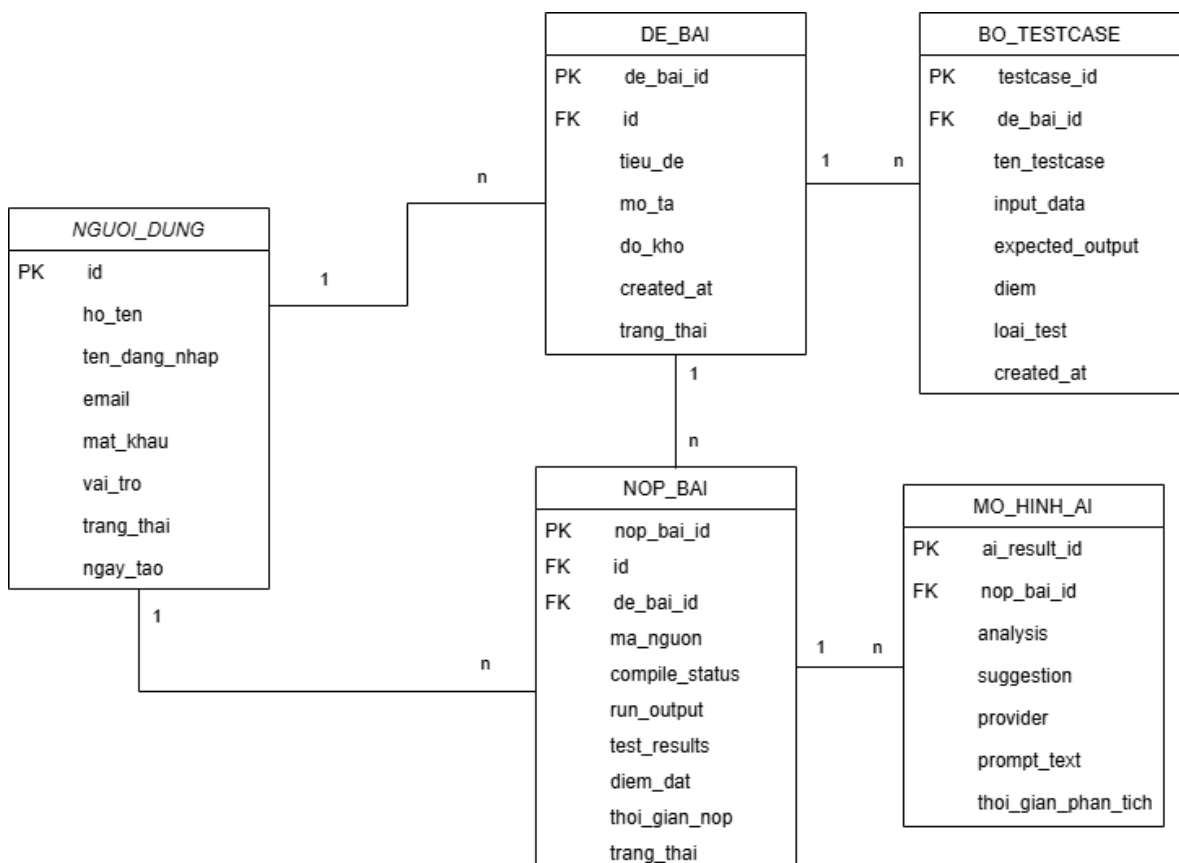
Quản trị viên có toàn quyền quản lý các đề bài trong hệ thống nhằm đảm bảo nội dung phù hợp và chính xác trước khi được sử dụng trong quá trình giảng dạy.

Quản lý Test Case

Quản trị viên có thể thêm mới, chỉnh sửa hoặc xóa các Test Case của từng đề bài. Chức năng này giúp đảm bảo bộ Test Case luôn đầy đủ và phản ánh chính xác yêu cầu của bài toán.

2.4 THIẾT KẾ CƠ SỞ DỮ LIỆU

2.4.1 Mô hình quan hệ giữa các thực thể



Hình 2.5 Mô hình ERD

2.4.1 Danh sách các thực thể

Bảng 2.2 Danh sách các thực thể

Bảng	Chức năng
nguoi_dung	Lưu thông tin tài khoản, vai trò và trạng thái người dùng
de_bai	Lưu thông tin đề bài lập trình
bo_testcase	Lưu các test case thuộc từng đề bài
nop_bai	Lưu mã nguồn, kết quả biên dịch, kết quả chạy và điểm của học sinh
mo_hinh_ai	Lưu kết quả phân tích, gợi ý sửa lỗi và prompt gửi đến AI

2.4.2 Thiết kế bảng người dùng (nguoi_dung)

Bảng 2.3 Cấu trúc bảng nguoi_dung

STT	Tên trường	Kiểu dữ liệu	Khóa	Mô tả
1	id	INTEGER	PK	Mã người dùng
2	ten_dang_nhap	TEXT		Tên đăng nhập
3	email	TEXT		Địa chỉ email
4	mat_khau_hash	TEXT		Mật khẩu đã mã hóa
5	ho_ten	TEXT		Họ và tên
6	vai_tro	TEXT		admin, giaovien hoặc hoc_sinh
7	metadata	TEXT		Thông tin bổ sung
8	last_login	DATETIME		Lần đăng nhập gần nhất
9	is_active	INTEGER		Trạng thái hoạt động
10	created_at	DATETIME		Ngày tạo tài khoản

Bảng nguoi_dung là bảng trung tâm của hệ thống, được sử dụng để lưu trữ toàn bộ thông tin tài khoản của những người tham gia sử dụng hệ thống, bao gồm quản trị viên (Admin), giáo viên và học sinh. Đây là bảng đầu tiên được truy cập khi người dùng thực hiện chức năng đăng nhập và cũng là cơ sở để xác định quyền hạn của từng đối tượng trong hệ thống.

Mỗi tài khoản được cấp một mã định danh duy nhất (id) đóng vai trò là khóa chính của bảng. Giá trị này được sử dụng để liên kết với các bảng khác như bảng

nop_bai nhằm xác định học sinh nào đã nộp bài hoặc giáo viên nào đã tạo đề bài. Bên cạnh đó, hệ thống sử dụng trường ten_dang_nhap kết hợp với mat_khau_hash để thực hiện quá trình xác thực người dùng trước khi cho phép truy cập vào hệ thống.

Để đảm bảo an toàn thông tin, hệ thống không lưu mật khẩu dưới dạng văn bản thuần (Plain Text) mà sử dụng trường mat_khau_hash để lưu giá trị mật khẩu sau khi đã được mã hóa bằng thuật toán băm. Cách lưu trữ này giúp hạn chế nguy cơ rò rỉ thông tin tài khoản trong trường hợp cơ sở dữ liệu bị truy cập trái phép.

Ngoài các thông tin cơ bản như họ tên (ho_ten) và địa chỉ thư điện tử (email), bảng còn sử dụng trường vai_tro để phân quyền người dùng. Trường này quyết định các chức năng mà người dùng được phép thực hiện sau khi đăng nhập. Cụ thể:

Quản trị viên có toàn quyền quản lý tài khoản, giáo viên, học sinh và giám sát toàn bộ hoạt động của hệ thống.

Giáo viên có quyền tạo đề bài, xây dựng bộ Test Case và theo dõi kết quả làm bài của học sinh.

Học sinh chỉ được phép lựa chọn đề bài, nộp mã nguồn, thực hiện biên dịch, chạy chương trình và sử dụng chức năng AI Help.

Ngoài ra, bảng còn bổ sung trường metadata nhằm lưu trữ các thông tin mở rộng, chẳng hạn như dữ liệu tài khoản khi người dùng đăng nhập thông qua Google OAuth. Việc sử dụng trường này giúp hệ thống có khả năng mở rộng mà không cần thay đổi cấu trúc cơ sở dữ liệu trong tương lai.

Hai trường last_login và created_at được sử dụng để lưu thời điểm người dùng đăng nhập gần nhất và thời gian tạo tài khoản. Thông tin này hỗ trợ quản trị viên theo dõi hoạt động của người dùng cũng như phục vụ việc thống kê và quản lý hệ thống. Trường is_active được sử dụng để xác định trạng thái của tài khoản, cho phép quản trị viên khóa hoặc mở tài khoản mà không cần xóa dữ liệu khỏi cơ sở dữ liệu.

Với cấu trúc trên, bảng nguoi_dung không chỉ đảm nhận chức năng xác thực và phân quyền mà còn đóng vai trò là nền tảng liên kết với các bảng dữ liệu khác trong toàn bộ hệ thống.

2.4.3 Thiết kế bảng đề bài (de_bai)

Bảng 2.4 Cấu trúc bảng de_bai

STT	Tên trường	Kiểu dữ liệu	Khóa	Mô tả
1	de_bai_id	INTEGER	PK	Mã đề bài
2	tieu_de	TEXT		Tiêu đề đề bài
3	mo_ta	TEXT		Mô tả bài toán
4	yeu_cau	TEXT		Yêu cầu thực hiện
5	ma_chuan	TEXT		Đoạn mã C mẫu
6	do_kho	TEXT		Độ khó
7	created_by	INTEGER	FK	Giáo viên tạo đề
8	is_active	INTEGER		Trạng thái
9	created_at	DATETIME		Ngày tạo

Bảng de_bai được xây dựng nhằm quản lý toàn bộ thông tin liên quan đến các bài tập lập trình được sử dụng trong hệ thống. Đây là bảng dữ liệu quan trọng, đóng vai trò là nguồn dữ liệu chính để học sinh lựa chọn bài làm, giáo viên quản lý nội dung giảng dạy và hệ thống thực hiện quá trình chấm bài thông qua bộ Test Case.

Mỗi đề bài được cấp một mã định danh duy nhất đóng vai trò là khóa chính của bảng. Giá trị này được sử dụng để liên kết với bảng bo_testcase và bảng nop_bai, từ đó xác định bộ Test Case tương ứng với từng đề bài cũng như lưu lại các bài nộp của học sinh đối với đề bài đó.

Để giúp người học dễ dàng nhận biết nội dung bài tập, mỗi đề bài đều có trường tieu_de, lưu tên của bài toán lập trình. Tiêu đề cần ngắn gọn nhưng thể hiện rõ yêu cầu của bài toán, ví dụ như "Tính tổng từ 1 đến N", "Kiểm tra số nguyên tố" hoặc "Sắp xếp mảng tăng dần". Trường này được hiển thị tại danh sách đề bài trên giao diện của học sinh và giáo viên.

Bên cạnh tiêu đề, trường mo_ta được sử dụng để trình bày nội dung tổng quan của bài toán. Nội dung mô tả giúp người học hiểu được mục đích của bài tập, dữ liệu cần xử lý cũng như kết quả cần đạt được trước khi bắt đầu viết chương trình. Ngoài ra, trường yeu_cau được thiết kế để mô tả chi tiết các yêu cầu mà chương trình cần đáp ứng, bao gồm dữ liệu đầu vào, dữ liệu đầu ra, điều kiện ràng buộc hoặc những

lưu ý khi xây dựng thuật toán. Việc tách riêng phần mô tả và phần yêu cầu giúp thông tin được trình bày rõ ràng hơn, tạo thuận lợi cho người học trong quá trình thực hiện.

Một thành phần quan trọng của bảng là trường `ma_chuan`. Đây là đoạn mã nguồn C mẫu được hệ thống hiển thị khi học sinh mở đề bài. Mã chuẩn thường bao gồm các thư viện cần thiết, hàm `main()` và phần khai báo biến cơ bản. Người học chỉ cần bổ sung thuật toán vào phần còn thiếu thay vì phải tự viết toàn bộ cấu trúc chương trình từ đầu. Điều này giúp giảm thời gian chuẩn bị, hạn chế các lỗi cú pháp cơ bản và tạo sự thống nhất giữa các bài làm.

Ngoài ra, trường `do_kho` được sử dụng để phân loại mức độ của đề bài theo các cấp độ như Easy, Medium hoặc Hard. Việc phân loại này giúp giáo viên xây dựng lộ trình học tập phù hợp với từng nhóm đối tượng, đồng thời hỗ trợ học sinh lựa chọn bài tập tương ứng với năng lực của mình.

Để xác định người tạo đề bài, bảng sử dụng trường `created_by`, liên kết đến tài khoản giáo viên trong bảng `nguoi_dung`. Thông qua liên kết này, hệ thống có thể quản lý quyền chỉnh sửa hoặc xóa đề bài, đồng thời hỗ trợ thống kê số lượng bài tập do mỗi giáo viên xây dựng.

Hai trường `is_active` và `created_at` được sử dụng để quản lý trạng thái hoạt động và thời gian tạo của đề bài. Trường `is_active` cho phép giáo viên hoặc quản trị viên tạm ẩn một đề bài khỏi danh sách mà không cần xóa dữ liệu khỏi cơ sở dữ liệu. Trong khi đó, `created_at` hỗ trợ việc theo dõi thời điểm tạo đề, phục vụ cho công tác quản lý và thống kê.

Nhìn chung, bảng `de_bai` là trung tâm của quá trình học tập trong hệ thống. Mọi hoạt động như lựa chọn bài tập, tải mã chuẩn, thực hiện kiểm thử, chấm bài và phân tích bằng AI đều được thực hiện dựa trên dữ liệu lưu trữ trong bảng này.

2.4.4 Thiết kế bảng bộ Test Case

Bảng 2.5 Cấu trúc bảng `bo_testcase`

STT	Tên trường	Kiểu dữ liệu	Khóa	Mô tả
1	<code>id</code>	INTEGER	PK	Mã Test Case
2	<code>de_bai_id</code>	INTEGER	FK	Mã đề bài
3	<code>created_by</code>	INTEGER	FK	Giáo viên tạo Test Case

4	ten_testcase	TEXT		Tên Test Case
5	input_data	TEXT		Dữ liệu đầu vào
6	expected_output	TEXT		Kết quả mong đợi
7	is_active	INTEGER		Trạng thái hoạt động
8	created_at	DATETIME		Thời gian tạo Test Case

Bảng bo_testcase được sử dụng để lưu trữ toàn bộ dữ liệu kiểm thử (Test Case) tương ứng với từng đề bài lập trình. Đây là một trong những bảng quan trọng của hệ thống, đóng vai trò là cơ sở để thực hiện chức năng chấm bài tự động. Thông qua các Test Case được xây dựng sẵn, hệ thống có thể đánh giá tính đúng đắn của chương trình do học sinh nộp mà không cần giáo viên phải kiểm tra thủ công.

Mỗi Test Case được tạo ra sẽ gắn với một đề bài cụ thể và một giáo viên. Giáo viên là người chịu trách nhiệm xây dựng dữ liệu đầu vào, kết quả mong đợi và quản lý các Test Case trong quá trình giảng dạy. Việc lưu thông tin giáo viên tạo Test Case giúp hệ thống xác định quyền chỉnh sửa, cập nhật hoặc xóa Test Case, đồng thời hỗ trợ thống kê số lượng Test Case do từng giáo viên xây dựng.

Mỗi Test Case được cấp một mã định danh duy nhất đóng vai trò là khóa chính của bảng. Trường de_bai_id là khóa ngoại liên kết với bảng de_bai, giúp xác định Test Case thuộc đề bài nào. Ngoài ra, trường created_by là khóa ngoại liên kết với bảng nguoi_dung, tham chiếu đến tài khoản giáo viên đã tạo Test Case.

Để thuận tiện trong việc quản lý, bảng sử dụng trường ten_testcase để đặt tên cho từng bộ kiểm thử. Tên Test Case có thể phản ánh mục đích kiểm tra, chẳng hạn như "Trường hợp cơ bản", "Kiểm tra giá trị biên", hoặc "Kiểm tra dữ liệu lớn". Điều này giúp giáo viên dễ dàng quản lý và hỗ trợ học sinh xác định trường hợp kiểm thử chưa đạt khi xem kết quả.

Hai trường quan trọng nhất của bảng là input_data và expected_output. Trường input_data lưu dữ liệu đầu vào được truyền vào chương trình khi thực hiện chức năng Run. Sau khi chương trình kết thúc, hệ thống sẽ thu nhận kết quả đầu ra và so sánh với dữ liệu lưu trong trường expected_output. Nếu hai kết quả giống nhau, Test Case được đánh dấu là đạt; ngược lại, hệ thống sẽ xác định chương trình chưa đáp ứng yêu cầu của bài toán và ghi nhận Test Case không đạt.

Việc xây dựng nhiều Test Case với các bộ dữ liệu khác nhau giúp kiểm tra chương trình trong nhiều tình huống, bao gồm các trường hợp thông thường, trường hợp biên và các trường hợp đặc biệt. Nhờ đó, hệ thống có thể phát hiện các lỗi logic mà chương trình vẫn có thể biên dịch thành công nhưng cho kết quả không chính xác. Đây cũng là cơ sở để chức năng AI Help phân tích nguyên nhân lỗi và đưa ra các gợi ý sửa lỗi phù hợp.

Ngoài ra, bảng còn sử dụng trường `is_active` để quản lý trạng thái hoạt động của Test Case và trường `created_at` để lưu thời điểm tạo dữ liệu kiểm thử. Thiết kế này giúp giáo viên có thể bổ sung hoặc cập nhật Test Case mà không ảnh hưởng đến các dữ liệu đã tồn tại, đồng thời tạo điều kiện thuận lợi cho việc mở rộng hệ thống trong tương lai.

2.4.5 Thiết kế bảng nộp bài

Bảng 2.6 Cấu trúc bảng `nop_bai`

STT	Tên trường	Kiểu dữ liệu	Khóa	Mô tả
1	<code>nop-bai-id</code>	INTEGER	PK	Mã bài nộp
2	<code>id</code>	INTEGER	FK	Mã học sinh thực hiện bài nộp
3	<code>de_bai_id</code>	INTEGER	FK	Mã đề bài
4	<code>ma_nguon</code>	TEXT		Mã nguồn C của học sinh
5	<code>compile_status</code>	TEXT		Kết quả biên dịch
6	<code>test_results</code>	TEXT		Kết quả thực hiện Test Case
7	<code>run_output</code>	TEXT		Kết quả đầu ra của chương trình
8	<code>created_at</code>	DATETIME		Thời gian nộp bài

Bảng `nop_bai` được thiết kế nhằm lưu trữ toàn bộ thông tin liên quan đến các lần nộp bài của học sinh trong quá trình thực hiện các bài tập lập trình. Đây là bảng phản ánh trực tiếp kết quả học tập của học sinh và đóng vai trò trung tâm trong quá trình chấm bài tự động, phân tích kết quả cũng như hỗ trợ giáo viên theo dõi tiến độ học tập.

Mỗi khi học sinh hoàn thành chương trình và lựa chọn chức năng Nộp bài, hệ thống sẽ tạo một bản ghi mới trong bảng `nop_bai`. Bản ghi này lưu lại toàn bộ thông tin về bài làm, bao gồm người thực hiện, đề bài được chọn, mã nguồn chương trình,

kết quả biên dịch, kết quả thực hiện Test Case cũng như thời gian nộp bài. Việc lưu trữ lịch sử các lần nộp giúp người học có thể theo dõi quá trình cải thiện bài làm của mình, đồng thời tạo điều kiện để giáo viên đánh giá quá trình học tập thay vì chỉ đánh giá kết quả cuối cùng.

Trường id là khóa chính của bảng, được sử dụng để định danh duy nhất cho mỗi lần nộp bài. Trường id là khóa ngoại liên kết với bảng nguoi_dung, xác định học sinh đã thực hiện bài nộp. Thông qua liên kết này, hệ thống có thể thống kê số lượng bài nộp của từng học sinh, theo dõi tiến độ học tập và hỗ trợ giáo viên đánh giá kết quả thực hành.

Bên cạnh đó, trường de_bai_id là khóa ngoại liên kết với bảng de_bai, giúp xác định bài nộp thuộc đề bài nào. Nhờ mối quan hệ này, hệ thống có thể truy xuất bộ Test Case tương ứng để thực hiện kiểm thử cũng như hỗ trợ thống kê tỷ lệ hoàn thành của từng đề bài.

Toàn bộ chương trình do học sinh xây dựng được lưu trong trường ma_nguon. Đây là dữ liệu quan trọng nhất của bảng vì được sử dụng cho nhiều chức năng khác nhau như biên dịch chương trình, thực hiện kiểm thử, lưu lịch sử bài làm và gửi đến mô hình AI để phân tích khi người học sử dụng chức năng AI Help. Việc lưu mã nguồn trực tiếp trong cơ sở dữ liệu giúp người học có thể xem lại bài làm của mình ở các lần nộp trước mà không cần lưu trữ thủ công trên máy tính.

Sau khi người học thực hiện chức năng Biên dịch, hệ thống sẽ sử dụng trình biên dịch GCC để kiểm tra chương trình. Kết quả biên dịch được lưu trong trường compile_status. Nếu chương trình có lỗi cú pháp hoặc lỗi biên dịch, hệ thống sẽ lưu thông báo lỗi để người học có thể chỉnh sửa trước khi tiếp tục thực hiện các bước tiếp theo. Chỉ khi chương trình được biên dịch thành công, hệ thống mới cho phép thực hiện kiểm thử bằng Test Case.

Đối với chức năng Chạy chương trình, hệ thống sẽ tự động lấy toàn bộ Test Case của đề bài, truyền dữ liệu đầu vào vào chương trình và so sánh kết quả đầu ra với kết quả mong đợi. Kết quả của từng Test Case được tổng hợp và lưu trong trường test_results. Thông tin này có thể bao gồm trạng thái đạt hoặc không đạt của từng Test Case, số lượng Test Case vượt qua cũng như thông tin chi tiết về sự khác biệt

giữa kết quả thực tế và kết quả mong đợi. Đây là cơ sở quan trọng để xác định chương trình có đáp ứng đúng yêu cầu của đề bài hay không.

Ngoài kết quả kiểm thử, trường `run_output` được sử dụng để lưu toàn bộ kết quả đầu ra của chương trình trong lần thực thi gần nhất. Trường dữ liệu này hỗ trợ người học kiểm tra kết quả chương trình và giúp AI có thêm thông tin khi phân tích nguyên nhân gây ra lỗi logic.

Cuối cùng, trường `created_at` lưu thời điểm học sinh thực hiện nộp bài. Dữ liệu này phục vụ cho việc thống kê, sắp xếp lịch sử nộp bài theo thời gian và hỗ trợ giáo viên đánh giá mức độ hoàn thành của học sinh trong từng giai đoạn học tập.

Có thể thấy, bảng `nop_bai` không chỉ đóng vai trò lưu trữ bài làm mà còn là cầu nối giữa chức năng chấm bài tự động và chức năng phân tích bằng trí tuệ nhân tạo. Hầu hết các chức năng quan trọng của hệ thống như xem lịch sử bài nộp, đánh giá kết quả học tập, thống kê tiến độ và AI Help đều sử dụng dữ liệu được lưu trong bảng này.

2.4.6 Thiết kế bảng phân tích AI

Bảng 2.7 Cấu trúc bảng `mo_hinh_ai`

STT	Tên trường	Kiểu dữ liệu	Khóa	Mô tả
1	<code>id_ai</code>	INTEGER	PK	Mã kết quả phân tích
2	<code>de_bai_id</code>	INTEGER	FK	Mã đề bài
3	<code>analysis</code>	TEXT		Nội dung phân tích của AI
4	<code>suggestion</code>	TEXT		Gợi ý sửa lỗi
5	<code>provider</code>	TEXT		Mô hình AI sử dụng
6	<code>created_at</code>	DATETIME		Thời gian phân tích

Bảng `phan_tich_ai` được sử dụng để lưu trữ kết quả phân tích và gợi ý sửa lỗi do mô hình trí tuệ nhân tạo tạo ra trong quá trình hỗ trợ người học. Đây là bảng dữ liệu đặc trưng của hệ thống, góp phần tạo nên sự khác biệt so với các hệ thống chấm bài lập trình truyền thống. Thay vì chỉ thông báo chương trình đúng hoặc sai, hệ thống còn sử dụng AI để phân tích mã nguồn, xác định nguyên nhân dẫn đến lỗi và đưa ra các gợi ý giúp người học cải thiện chương trình.

Chức năng AI Help được kích hoạt sau khi học sinh hoàn thành quá trình biên dịch hoặc chạy chương trình. Khi người học lựa chọn chức năng này, hệ thống sẽ thu thập các thông tin cần thiết như nội dung đề bài, mã nguồn C do học sinh viết, kết quả biên dịch và kết quả thực hiện Test Case. Những thông tin này được tổng hợp thành một yêu cầu và gửi đến mô hình ngôn ngữ lớn thông qua API. Sau khi xử lý, AI sẽ trả về nội dung phân tích và các gợi ý sửa lỗi để hỗ trợ người học. Trong thiết kế hiện tại, bảng `phan_tich_ai` được liên kết trực tiếp với bảng `de_bai` thông qua trường `de_bai_id`. Trường này đóng vai trò khóa ngoại, giúp xác định kết quả phân tích thuộc về đề bài nào. Việc liên kết với bảng đề bài giúp hệ thống dễ dàng truy xuất nội dung bài toán và hiển thị kết quả phân tích tương ứng khi người học thực hiện chức năng AI Help.

Trường `analysis` được sử dụng để lưu nội dung phân tích do AI trả về. Nội dung này thường bao gồm nhận xét về thuật toán, nguyên nhân gây ra lỗi logic hoặc các điểm chưa hợp lý trong chương trình. Bên cạnh đó, trường `suggestion` lưu các gợi ý chỉnh sửa, giúp học sinh xác định hướng khắc phục mà không làm mất đi tính chủ động trong quá trình học tập. Để thuận tiện cho việc quản lý, bảng còn sử dụng trường `provider` nhằm lưu tên mô hình AI hoặc nhà cung cấp dịch vụ được sử dụng, chẳng hạn như Gemini, OpenRouter hoặc Groq. Điều này giúp hệ thống có thể thống kê hiệu quả của từng mô hình AI và dễ dàng thay đổi hoặc mở rộng khi tích hợp các mô hình mới trong tương lai. Trường `created_at` lưu thời điểm AI thực hiện phân tích, hỗ trợ việc theo dõi lịch sử sử dụng chức năng AI Help và phục vụ công tác thống kê.

Việc xây dựng bảng `phan_tich_ai` giúp toàn bộ kết quả phân tích được lưu trữ tập trung trong cơ sở dữ liệu. Điều này không chỉ giúp người học xem lại các lần phân tích trước mà còn tạo điều kiện để giáo viên đánh giá những lỗi mà học sinh thường gặp, từ đó điều chỉnh nội dung giảng dạy cho phù hợp.

2.5 THIẾT KẾ LƯỠNG XỬ LÝ NGHIỆP VỤ

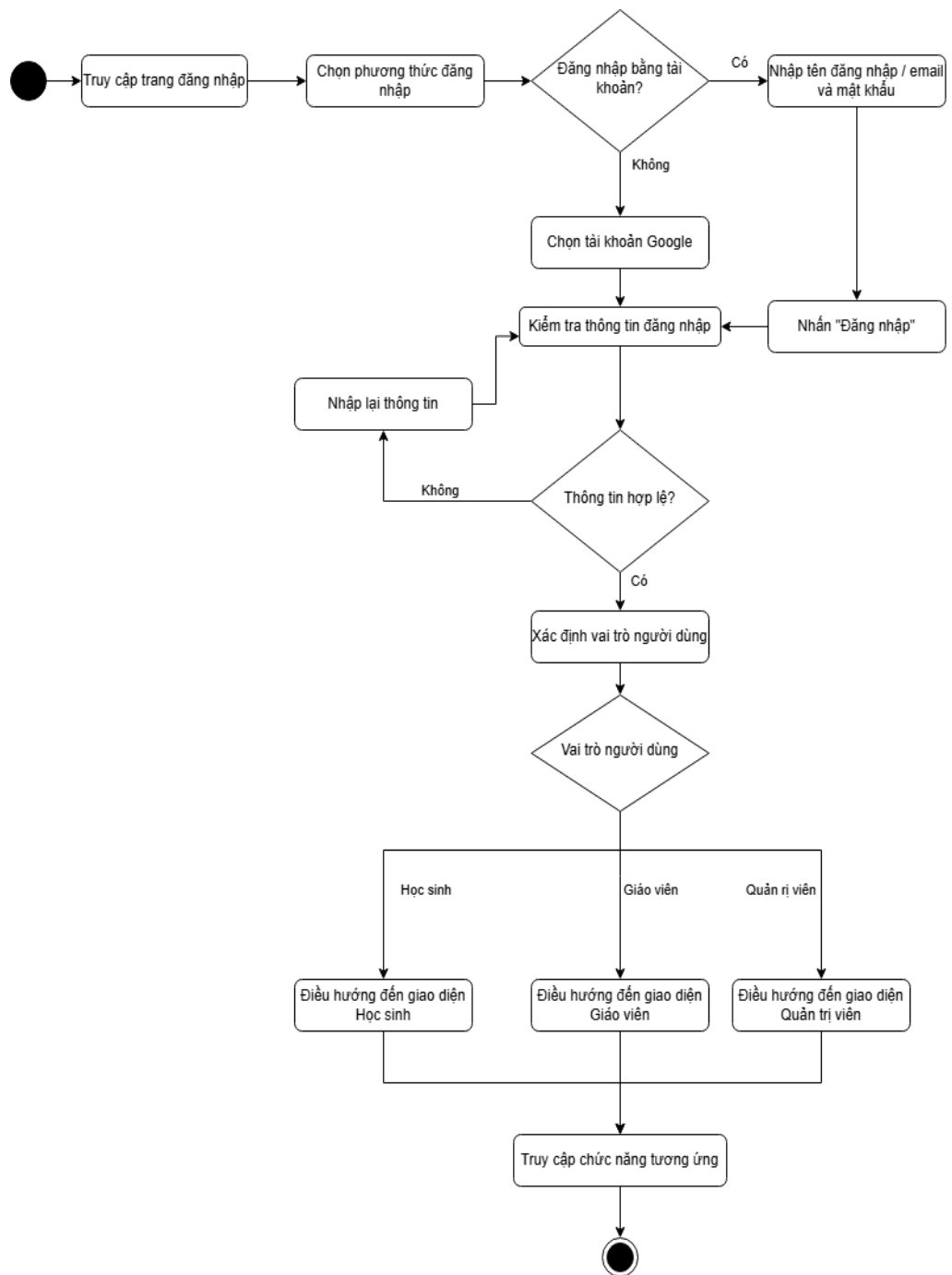
Activity Diagram được sử dụng để mô tả luồng xử lý của các chức năng chính trong hệ thống từ thời điểm người dùng gửi yêu cầu đến khi hệ thống trả về kết quả. Khác với Use Case chỉ mô tả mối quan hệ giữa người dùng và hệ thống, Activity Diagram thể hiện chi tiết trình tự các bước xử lý, các điều kiện rẽ nhánh cũng như kết quả của từng thao tác.

Việc xây dựng Activity Diagram giúp làm rõ quy trình hoạt động của hệ thống, hỗ trợ quá trình thiết kế và hiện thực các chức năng trong giai đoạn phát triển. Đồng thời, sơ đồ còn giúp người đọc dễ dàng hình dung cách các thành phần trong hệ thống phối hợp với nhau để hoàn thành một chức năng cụ thể. Trong đề tài này, Activity Diagram được xây dựng cho các chức năng quan trọng của hệ thống gồm: đăng nhập, thực hiện bài lập trình, hỗ trợ AI và quản lý đề bài.

2.5.1 Mô hình hoạt động chức năng đăng nhập

Activity Diagram chức năng đăng nhập mô tả quy trình xác thực người dùng trước khi sử dụng hệ thống. Người dùng có thể đăng nhập bằng tài khoản được cấp hoặc thông qua Google OAuth. Sau khi nhận yêu cầu, hệ thống kiểm tra thông tin đăng nhập, xác định vai trò của người dùng và điều hướng đến giao diện phù hợp.

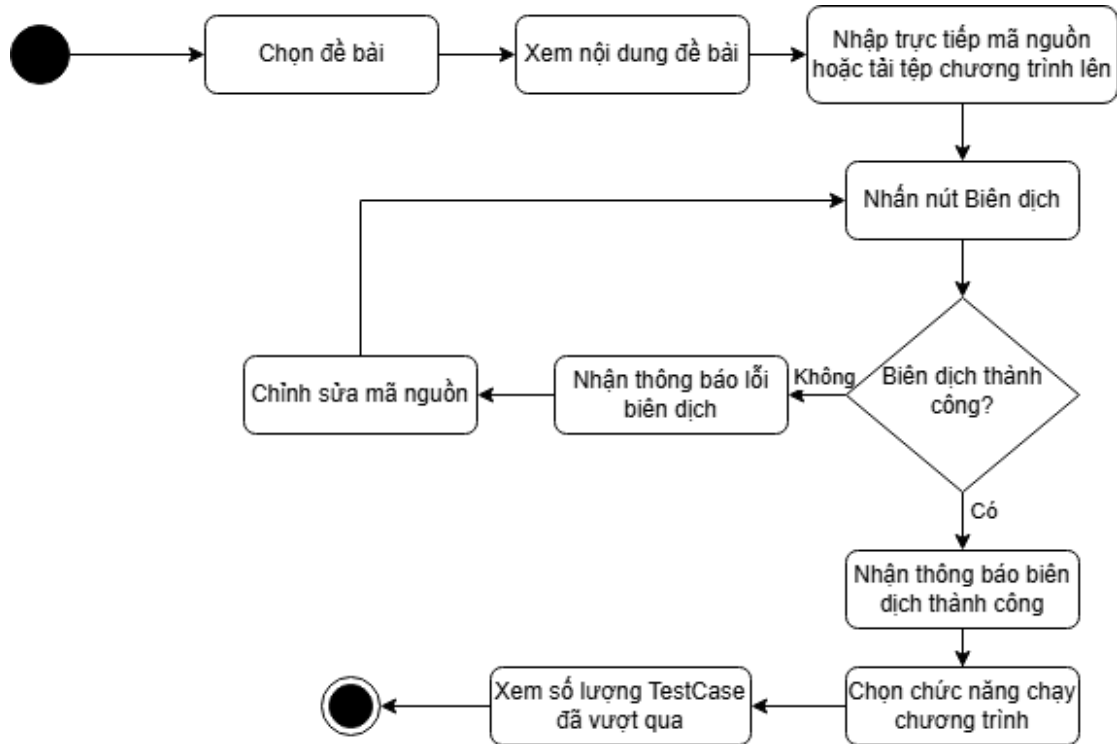
Nếu thông tin đăng nhập hợp lệ, hệ thống tạo phiên làm việc và cho phép truy cập các chức năng tương ứng. Ngược lại, hệ thống sẽ hiển thị thông báo lỗi và yêu cầu người dùng đăng nhập lại.



Hình 2.6 Activity Diagram chức năng đăng nhập

2.5.2 Mô hình hoạt động chức năng làm bài lập trình

Đây là Activity Diagram quan trọng nhất của hệ thống vì mô tả toàn bộ quy trình thực hiện bài lập trình của học sinh. Quá trình bắt đầu khi học sinh đăng nhập thành công và lựa chọn một đề bài từ danh sách. Hệ thống hiển thị nội dung đề bài, mã nguồn khởi động và các thông tin cần thiết để học sinh bắt đầu lập trình.



Hình 2.7 Activity Diagram chức năng làm bài lập trình

Sau khi hoàn thành chương trình, học sinh có thể nhập trực tiếp mã nguồn hoặc tải tệp chương trình lên hệ thống. Khi nhấn nút Biên dịch, Backend kiểm tra lỗi cú pháp. Nếu quá trình biên dịch thất bại, hệ thống sẽ trả về thông báo lỗi để học sinh chỉnh sửa.

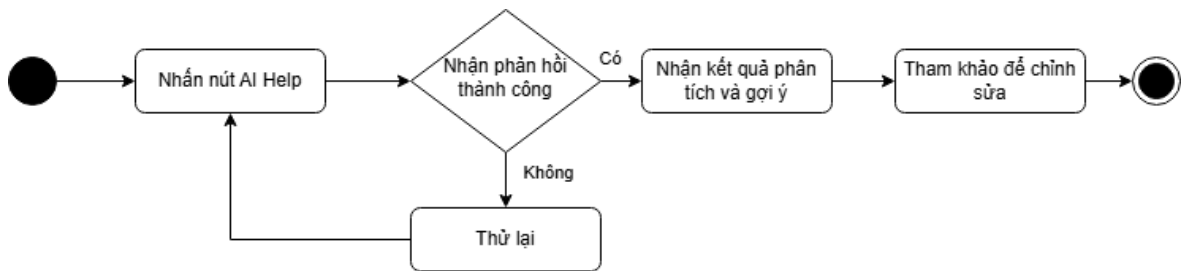
Nếu chương trình được biên dịch thành công, học sinh tiếp tục lựa chọn chức năng Chạy chương trình. Backend sẽ lấy toàn bộ Test Case của đề bài, lần lượt truyền dữ liệu đầu vào vào chương trình và thu nhận kết quả đầu ra. Sau đó, hệ thống so sánh kết quả thực tế với kết quả mong đợi của từng Test Case để xác định trạng thái đạt hoặc không đạt và hiển thị kết quả trên giao diện.

2.5.3 Mô hình hoạt động chức năng AI Help

Sau khi thực hiện chương trình, học sinh có thể sử dụng chức năng AI Help để nhận gợi ý sửa lỗi.

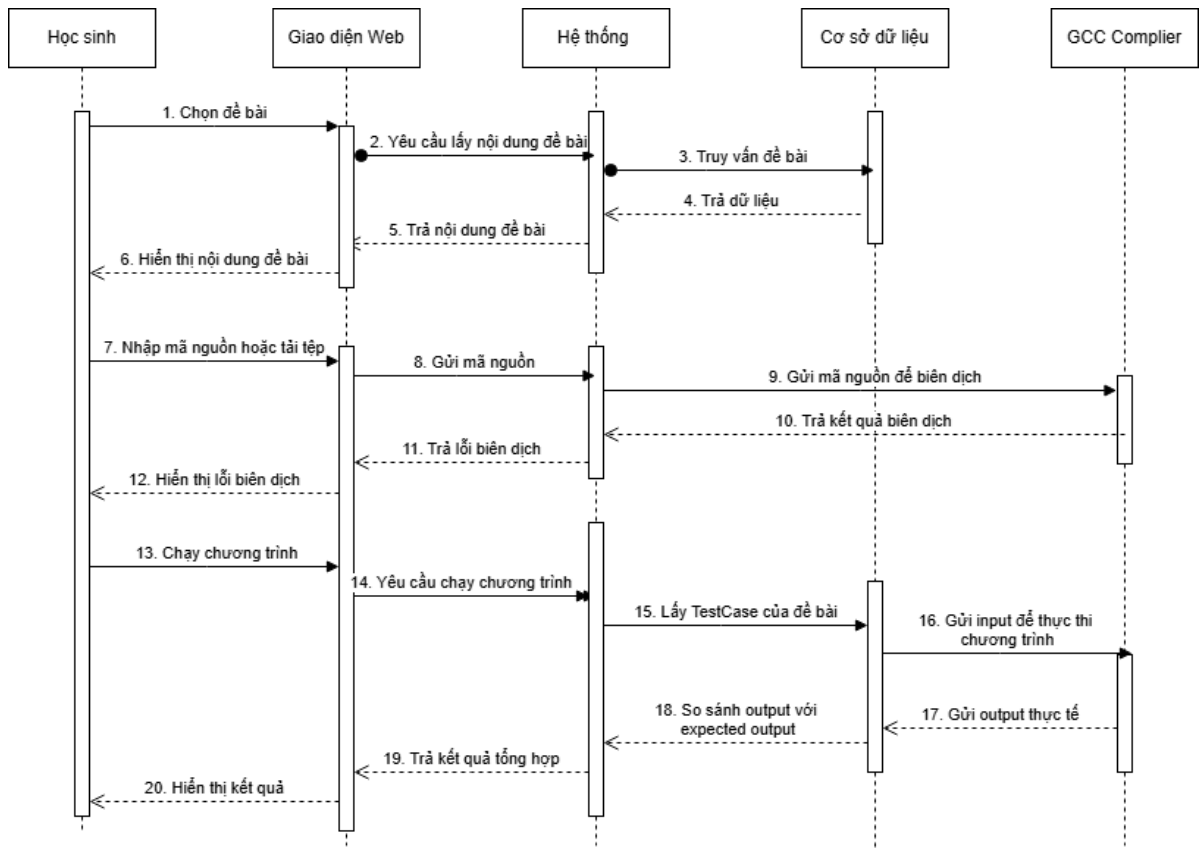
Khi người dùng nhấn nút AI Help, Backend sẽ thu thập các thông tin cần thiết như nội dung đề bài, mã nguồn, kết quả biên dịch và kết quả thực hiện Test Case. Các thông tin này được tổng hợp thành Prompt và gửi đến mô hình AI thông qua API.

Sau khi nhận được phản hồi từ AI, hệ thống hiển thị nội dung phân tích, nguyên nhân gây lỗi logic và các gợi ý sửa lỗi để học sinh tham khảo. Quá trình này chỉ mang tính chất hỗ trợ học tập, giúp người học hiểu nguyên nhân của lỗi thay vì cung cấp trực tiếp lời giải hoàn chỉnh.



Hình 2.8 Activity Diagram chức năng AI Help

2.5.4 Mô hình tuần tự tiến trình làm bài tập



Hình 2.9 Mô hình tuần tự tiến trình làm bài tập

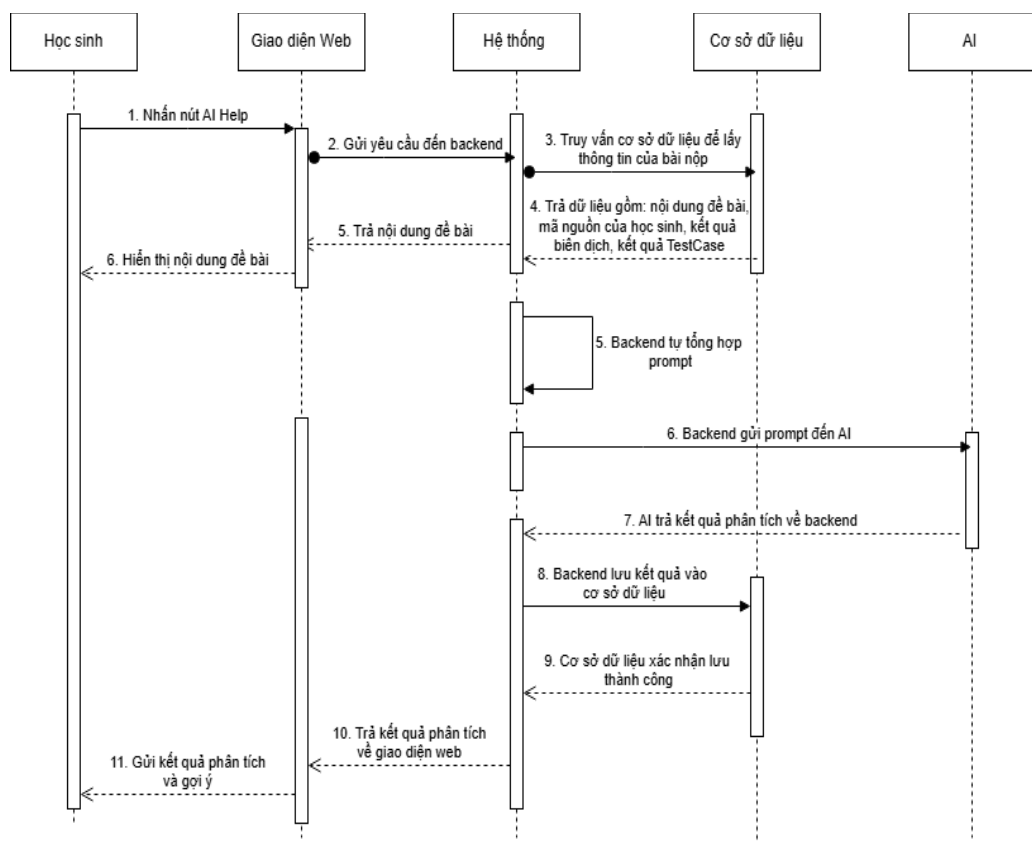
Mô hình tuần tự chức năng làm bài lập trình mô tả quá trình tương tác giữa học sinh, giao diện Web, hệ thống, cơ sở dữ liệu SQLite, trình biên dịch GCC và bộ kiểm

thử. Quy trình bắt đầu khi học sinh chọn đề bài, hệ thống truy xuất và hiển thị nội dung đề bài cùng các thông tin cần thiết. Sau đó, học sinh nhập trực tiếp mã nguồn hoặc tải chương trình lên và gửi yêu cầu biên dịch.

Hệ thống tiếp nhận mã nguồn và gọi trình biên dịch GCC để kiểm tra lỗi cú pháp. Nếu biên dịch thất bại, hệ thống trả thông báo lỗi về giao diện để học sinh chỉnh sửa và biên dịch lại. Nếu biên dịch thành công, học sinh có thể chạy chương trình. Khi đó, hệ thống truy xuất bộ kiểm thử từ cơ sở dữ liệu, lần lượt truyền dữ liệu đầu vào vào chương trình, thu nhận kết quả đầu ra và so sánh với kết quả mong đợi.

Sau khi thực hiện toàn bộ trường hợp kiểm thử, hệ thống tổng hợp kết quả chấm, lưu thông tin bài nộp và kết quả kiểm thử vào cơ sở dữ liệu, sau đó hiển thị kết quả cho học sinh. Sơ đồ này làm rõ vai trò trung tâm của hệ thống trong việc điều phối quá trình biên dịch, thực thi, kiểm thử và trả kết quả bài làm lập trình.

2.5.5 Mô hình tuần tự tiến trình AI trợ giúp



Hình 2.10 Mô hình tuần tự tiến trình AI trợ giúp

Mô hình tuần tự chức năng AI trợ giúp mô tả quá trình hệ thống hỗ trợ học sinh phân tích lỗi logic và gợi ý sửa lỗi thông qua mô hình AI. Sau khi học sinh đã thực hiện chương trình và có kết quả kiểm thử, học sinh có thể nhấn nút “AI Help”

trên giao diện. Giao diện Web gửi yêu cầu đến hệ thống để bắt đầu quá trình phân tích.

Hệ thống truy xuất các thông tin cần thiết từ cơ sở dữ liệu, bao gồm nội dung đề bài, mã nguồn học sinh, kết quả biên dịch và kết quả kiểm thử. Các thông tin này được tổng hợp thành lệnh prompt và gửi đến mô hình AI thông qua API. Sau khi phân tích, AI trả về nội dung phản hồi gồm nhận định lỗi logic, nguyên nhân có thể gây lỗi và gợi ý sửa lỗi.

Nếu quá trình gọi AI thất bại, hệ thống hiển thị thông báo lỗi để học sinh có thể thử lại. Nếu nhận phản hồi thành công, hệ thống lưu kết quả phân tích vào bảng `mo_hinh_ai` và trả nội dung về giao diện. Chức năng AI trợ giúp chỉ đóng vai trò hỗ trợ học tập, giúp học sinh hiểu nguyên nhân lỗi và định hướng sửa mã nguồn, không cung cấp trực tiếp lời giải hoàn chỉnh.

2.6 THIẾT KẾ KIẾN TRÚC HỆ THỐNG

2.6.1 Kiến trúc tổng thể của hệ thống

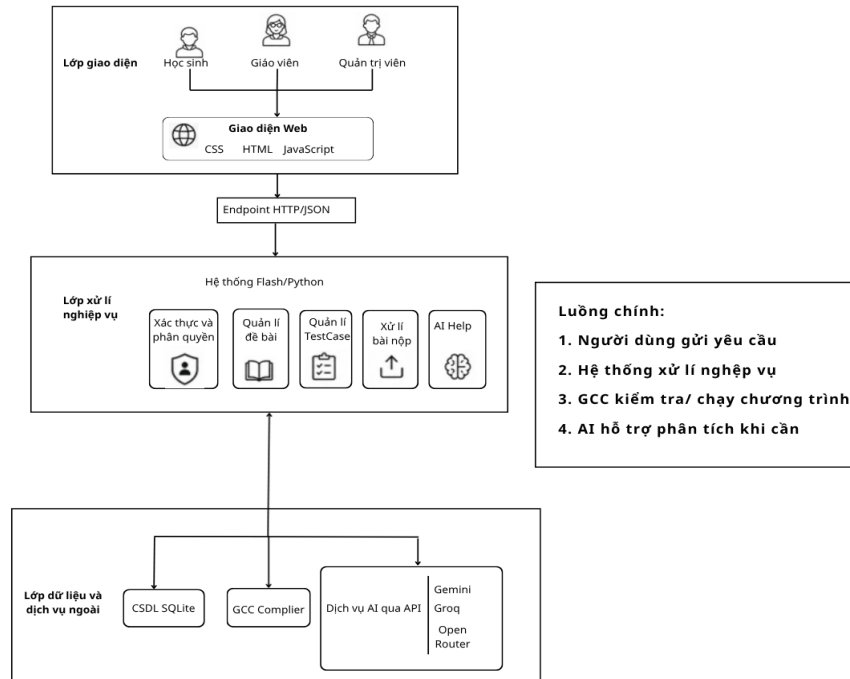
Hệ thống phân tích và gợi ý sửa lỗi logic mã nguồn C được xây dựng theo mô hình kiến trúc ba tầng, bao gồm tầng giao diện, tầng xử lý nghiệp vụ và tầng dữ liệu. Việc áp dụng mô hình kiến trúc này giúp các thành phần của hệ thống được tách biệt rõ ràng, tạo điều kiện thuận lợi cho việc phát triển, bảo trì và mở rộng trong tương lai.

Trong mô hình này, người dùng chỉ tương tác với giao diện Web. Mọi yêu cầu từ giao diện sẽ được gửi đến máy chủ Flask để xử lý. Sau khi nhận yêu cầu, Flask Backend sẽ thực hiện các nghiệp vụ như xác thực người dùng, quản lý đề bài, biên dịch chương trình C, thực hiện kiểm thử bằng Test Case, kết nối với mô hình AI và truy xuất dữ liệu từ cơ sở dữ liệu SQLite. Kết quả xử lý sau đó sẽ được trả về giao diện để hiển thị cho người dùng.

Kiến trúc của hệ thống bao gồm các thành phần chính sau:

- Người dùng
- Giao diện Web
- Máy chủ xử lý Flask Backend

- Trình biên dịch GCC
- Cơ sở dữ liệu SQLite
- Dịch vụ AI thông qua API



Hình 2.11 Kiến trúc hệ thống

2.6.2 Tầng giao diện

Tầng giao diện là nơi người dùng trực tiếp tương tác với hệ thống. Giao diện được xây dựng bằng HTML, CSS, Bootstrap và JavaScript nhằm đảm bảo tính trực quan, dễ sử dụng và có khả năng hoạt động trên nhiều trình duyệt khác nhau.

Tùy theo vai trò của người dùng, hệ thống sẽ hiển thị các chức năng tương ứng.

Đối với học sinh, giao diện cung cấp các chức năng lựa chọn đề bài, xem nội dung bài tập, nhập hoặc tải lên mã nguồn C, biên dịch chương trình, chạy chương trình, xem kết quả và sử dụng chức năng AI Help.

Đối với giáo viên, giao diện hỗ trợ quản lý đề bài, quản lý bộ Test Case và theo dõi kết quả làm bài của học sinh.

Đối với quản trị viên, giao diện bổ sung thêm các chức năng quản lý tài khoản, phân quyền người dùng và quản trị toàn bộ hệ thống.

Frontend không thực hiện bất kỳ nghiệp vụ xử lý nào mà chỉ đóng vai trò gửi yêu cầu đến Flask Backend thông qua các API và hiển thị kết quả trả về.

2.6.3 Tầng xử lý nghiệp vụ

Flask Backend là thành phần trung tâm của hệ thống, chịu trách nhiệm tiếp nhận tất cả các yêu cầu từ giao diện người dùng và thực hiện các nghiệp vụ xử lý.

Sau khi nhận được yêu cầu từ Frontend, Backend sẽ kiểm tra thông tin xác thực của người dùng, xác định quyền truy cập và thực hiện chức năng tương ứng.

Các nghiệp vụ chính của Flask Backend bao gồm:

Xác thực đăng nhập và phân quyền người dùng.

- Quản lý tài khoản.
- Quản lý đề bài.
- Quản lý Test Case.
- Quản lý bài nộp.
- Biên dịch chương trình C.
- Thực thi chương trình.
- So sánh kết quả với Test Case.
- Gửi yêu cầu đến mô hình AI.
- Lưu kết quả phân tích.
- Trả kết quả về giao diện.

Backend được xây dựng bằng Python sử dụng Flask Framework. Flask được lựa chọn do có kiến trúc đơn giản, dễ mở rộng, hỗ trợ xây dựng REST API và dễ tích hợp với các thư viện của Python cũng như các dịch vụ AI hiện nay.

2.6.4 Trình biên dịch GCC

Trình biên dịch GCC là thành phần chịu trách nhiệm biên dịch và thực thi các chương trình C do học sinh viết.

Khi người dùng lựa chọn chức năng Biên dịch, Flask Backend sẽ tạo một tệp nguồn tạm thời từ mã nguồn do học sinh nhập, sau đó sử dụng thư viện subprocess của Python để gọi GCC tiến hành biên dịch chương trình.

Nếu quá trình biên dịch xảy ra lỗi, GCC sẽ trả về thông báo lỗi để Backend gửi đến giao diện người dùng. Trong trường hợp chương trình được biên dịch thành công, hệ thống sẽ sinh ra tệp thực thi và sẵn sàng cho bước chạy chương trình.

Khi người dùng lựa chọn chức năng “Chạy chương trình”, Backend sẽ tiếp tục sử dụng subprocess để thực thi chương trình vừa được biên dịch, đồng thời truyền dữ liệu đầu vào của từng Test Case vào chương trình. Kết quả đầu ra sẽ được thu thập để phục vụ quá trình so sánh với đáp án chuẩn.

Việc sử dụng GCC giúp hệ thống đảm bảo chương trình được biên dịch đúng theo chuẩn của ngôn ngữ lập trình C và tạo ra môi trường thực thi gần giống với các hệ thống chấm bài trực tuyến hiện nay.

2.6.5 Hệ thống kiểm thử bằng Test Case

Sau khi chương trình được biên dịch thành công, Flask Backend sẽ thực hiện chức năng kiểm thử tự động dựa trên bộ Test Case do giáo viên xây dựng.

Đầu tiên, hệ thống truy xuất toàn bộ Test Case tương ứng với đề bài từ cơ sở dữ liệu SQLite. Mỗi Test Case bao gồm dữ liệu đầu vào và kết quả đầu ra mong đợi.

Backend lần lượt truyền dữ liệu đầu vào của từng Test Case vào chương trình đang thực thi. Sau khi chương trình kết thúc, kết quả đầu ra sẽ được thu thập và so sánh với kết quả chuẩn đã lưu trong cơ sở dữ liệu.

Nếu kết quả của chương trình giống hoàn toàn với đáp án mong đợi thì Test Case được đánh dấu là đạt. Ngược lại, hệ thống sẽ ghi nhận Test Case không đạt và lưu lại thông tin để hiển thị cho người học.

Kết quả của toàn bộ quá trình kiểm thử sẽ được tổng hợp thành số lượng Test Case đạt, số lượng Test Case chưa đạt và danh sách các Test Case bị sai. Đây là cơ sở quan trọng để đánh giá tính đúng đắn của chương trình cũng như cung cấp dữ liệu đầu vào cho chức năng AI Help.

2.6.6 Dịch vụ AI

Một trong những điểm nổi bật của hệ thống là chức năng hỗ trợ phân tích lỗi logic bằng trí tuệ nhân tạo.

Khi học sinh lựa chọn chức năng AI Help, Flask Backend sẽ tổng hợp các thông tin gồm:

- Nội dung đề bài.
- Mã nguồn chương trình.
- Kết quả biên dịch.
- Kết quả thực hiện Test Case.
- Các Test Case chưa đạt.

Các thông tin trên được xây dựng thành một Prompt và gửi đến mô hình ngôn ngữ lớn thông qua API. Trong phạm vi đề tài, hệ thống hỗ trợ các dịch vụ như Gemini, Groq hoặc OpenRouter.

Sau khi nhận được phản hồi từ AI, Backend sẽ xử lý nội dung, lưu kết quả vào cơ sở dữ liệu và trả về giao diện người dùng.

- AI không thay thế quá trình chấm bài mà chỉ đóng vai trò hỗ trợ người học bằng cách:
- Phân tích nguyên nhân gây ra lỗi logic.
- Giải thích vì sao chương trình chưa đạt yêu cầu.
- Đưa ra hướng sửa lỗi.
- Gợi ý cải thiện thuật toán.

Việc chỉ đưa ra gợi ý thay vì cung cấp lời giải hoàn chỉnh giúp người học phát triển tư duy giải quyết vấn đề và nâng cao kỹ năng lập trình.

2.6.7 Cơ sở dữ liệu SQLite

Hệ thống sử dụng SQLite làm hệ quản trị cơ sở dữ liệu để lưu trữ toàn bộ thông tin của hệ thống. SQLite được lựa chọn vì có cấu trúc đơn giản, không yêu cầu cài đặt máy chủ riêng, dễ triển khai và phù hợp với quy mô của đề tài.

Các bảng dữ liệu chính của hệ thống bao gồm:

- `nguoi_dung`: Lưu thông tin tài khoản và phân quyền người dùng.
- `de_bai`: Lưu thông tin các bài tập lập trình.
- `bo_testcase`: Lưu dữ liệu kiểm thử của từng đề bài.
- `nop_bai`: Lưu mã nguồn và kết quả làm bài của học sinh.
- `mo_hinh_ai`: Lưu kết quả phân tích và gợi ý sửa lỗi từ AI.

Các bảng dữ liệu được liên kết với nhau thông qua các khóa chính và khóa ngoại nhằm đảm bảo tính toàn vẹn của dữ liệu cũng như hỗ trợ việc truy xuất thông tin trong quá trình xử lý nghiệp vụ.

2.7 THIẾT KẾ API

Hệ thống được xây dựng theo mô hình Client–Server, trong đó giao diện người dùng và hệ thống xử lý giao tiếp với nhau thông qua các Endpoint HTTP. Mỗi Endpoint đảm nhận một chức năng cụ thể, cho phép giao diện gửi yêu cầu đến hệ thống để thực hiện các thao tác như đăng nhập, lấy danh sách đề bài, quản lý Test Case, biên dịch chương trình, chạy chương trình, xem kết quả hoặc yêu cầu AI hỗ trợ phân tích lỗi.

Các Endpoint được xây dựng bằng Flask và trả về dữ liệu ở định dạng JSON để giao diện Web có thể xử lý và hiển thị kết quả cho người dùng. Việc tổ chức Endpoint theo từng nhóm chức năng giúp hệ thống dễ bảo trì, dễ mở rộng và tách biệt rõ giữa phần giao diện với phần xử lý nghiệp vụ. Dựa trên các chức năng đã cài đặt, hệ thống gồm các nhóm Endpoint chính: xác thực người dùng, quản lý đề bài, quản lý Test Case, xử lý bài làm lập trình, hỗ trợ AI và quản trị hệ thống.

2.7.1 Endpoint xác thực người dùng

Nhóm Endpoint này chịu trách nhiệm xác thực danh tính người dùng trước khi sử dụng hệ thống. Khi người dùng gửi thông tin đăng nhập, hệ thống kiểm tra tài khoản trong cơ sở dữ liệu, xác thực mật khẩu và xác định vai trò của người dùng. Nếu đăng nhập thành công, hệ thống tạo phiên làm việc để người dùng có thể truy cập các chức năng tương ứng với quyền hạn của mình.

Đối với đăng nhập bằng Google, hệ thống sử dụng Google OAuth 2.0 để xác thực tài khoản. Sau khi Google xác nhận thành công, hệ thống lấy thông tin người dùng và thực hiện đăng nhập mà không yêu cầu nhập mật khẩu hệ thống.

Bảng 2.8 Endpoint xác thực người dùng

Endpoint	Phương thức	Chức năng
/login	POST	Đăng nhập bằng tài khoản hệ thống
/logout	GET	Đăng xuất khỏi hệ thống
/auth/google	GET	Đăng nhập bằng Google OAuth

2.7.2 Endpoint quản lý đề bài

Nhóm Endpoint này được sử dụng để quản lý các đề bài lập trình trong hệ thống. Giáo viên hoặc quản trị viên có thể thêm mới, cập nhật hoặc xóa đề bài phục vụ quá trình học tập và kiểm tra. Mỗi đề bài bao gồm các thông tin như tiêu đề, mô tả bài toán, yêu cầu đầu vào, mã nguồn khởi động và mức độ khó.

Đối với học sinh, hệ thống cung cấp các Endpoint cho phép xem danh sách đề bài và xem chi tiết nội dung đề bài trước khi thực hiện bài lập trình.

Bảng 2.9 Endpoint quản lý đề bài

Endpoint	Phương thức	Chức năng
/problems	GET	Lấy danh sách đề bài
/problem/<id>	GET	Xem chi tiết đề bài
/admin/problem/add	POST	Thêm đề bài
/admin/problem/update/<id>	PUT	Cập nhật đề bài
/admin/problem/delete/<id>	DELETE	Xóa đề bài

2.7.3 Endpoint quản lý Test Case

Nhóm Endpoint này cho phép quản lý các Test Case tương ứng với từng đề bài. Mỗi Test Case bao gồm dữ liệu đầu vào, kết quả mong đợi và tên Test Case. Khi học sinh chạy chương trình, hệ thống sử dụng các Test Case này để kiểm tra tính đúng đắn của chương trình.

Giáo viên hoặc quản trị viên có thể thêm, sửa hoặc xóa Test Case. Hệ thống cũng cung cấp Endpoint để lấy danh sách Test Case của một đề bài cụ thể phục vụ quá trình kiểm thử chương trình.

Bảng 2.10 Endpoint quản lý Test Case

Endpoint	Phương thức	Chức năng
/testcase/<problem_id>	GET	Lấy danh sách Test Case của đề bài
/admin/testcase/add	POST	Thêm Test Case
/admin/testcase/update/<id>	PUT	Cập nhật Test Case
/admin/testcase/delete/<id>	DELETE	Xóa Test Case

2.7.4 Endpoint xử lý bài làm lập trình

Đây là nhóm Endpoint quan trọng nhất của hệ thống, trực tiếp xử lý mã nguồn do học sinh gửi lên. Sau khi học sinh nhập hoặc tải tệp mã nguồn C, hệ thống tiếp nhận mã nguồn và sử dụng GCC để biên dịch, kiểm tra lỗi cú pháp và tạo tệp thực thi.

Nếu biên dịch thành công, hệ thống tiếp tục chạy chương trình với dữ liệu đầu vào của từng Test Case. Kết quả đầu ra thực tế được so sánh với kết quả mong đợi để xác định trạng thái đạt hoặc không đạt. Kết quả kiểm thử sau đó được tổng hợp và hiển thị cho học sinh.

Bảng 2.11 Endpoint xử lý bài làm lập trình

Endpoint	Phương thức	Chức năng
/compile	POST	Biên dịch mã nguồn C
/run	POST	Thực thi chương trình
/run-testcase	POST	Chạy toàn bộ Test Case và so sánh kết quả

2.7.5 Endpoint hỗ trợ AI

Sau khi chương trình được biên dịch và kiểm tra bằng Test Case, học sinh có thể sử dụng chức năng AI Help để yêu cầu hệ thống hỗ trợ phân tích lỗi. Endpoint này tiếp nhận các thông tin cần thiết như nội dung đề bài, mã nguồn, kết quả biên dịch và kết quả kiểm thử để xây dựng Prompt gửi đến mô hình AI.

Phản hồi từ AI bao gồm phân tích lỗi logic, nguyên nhân có thể gây lỗi và gợi ý sửa lỗi. Các gợi ý này chỉ đóng vai trò hỗ trợ học tập, giúp học sinh hiểu nguyên nhân lỗi và định hướng chỉnh sửa chương trình, không cung cấp trực tiếp lời giải hoàn chỉnh.

Bảng 2.12 Endpoint hỗ trợ AI

Endpoint	Phương thức	Chức năng
/ai-help	POST	Phân tích mã nguồn và gợi ý sửa lỗi bằng AI

2.7.6 Endpoint quản trị hệ thống

Nhóm Endpoint này dành cho quản trị viên hệ thống, cho phép quản lý tài khoản người dùng, bao gồm thêm mới, cập nhật hoặc xóa tài khoản. Bên cạnh đó, hệ thống cũng sử dụng thông tin vai trò người dùng để kiểm soát quyền truy cập, đảm bảo mỗi nhóm người dùng chỉ được sử dụng các chức năng phù hợp với quyền hạn được cấp.

Bảng 2.13 Endpoint quản trị hệ thống

Endpoint	Phương thức	Chức năng
/admin/users	GET	Xem danh sách người dùng
/admin/user/add	POST	Thêm tài khoản
/admin/user/update/<id>	PUT	Cập nhật tài khoản
/admin/user/delete/<id>	DELETE	Xóa tài khoản

CHƯƠNG 3 KẾT QUẢ THỰC NGHIỆM

3.1 GIAO DIỆN ĐĂNG NHẬP HỆ THỐNG

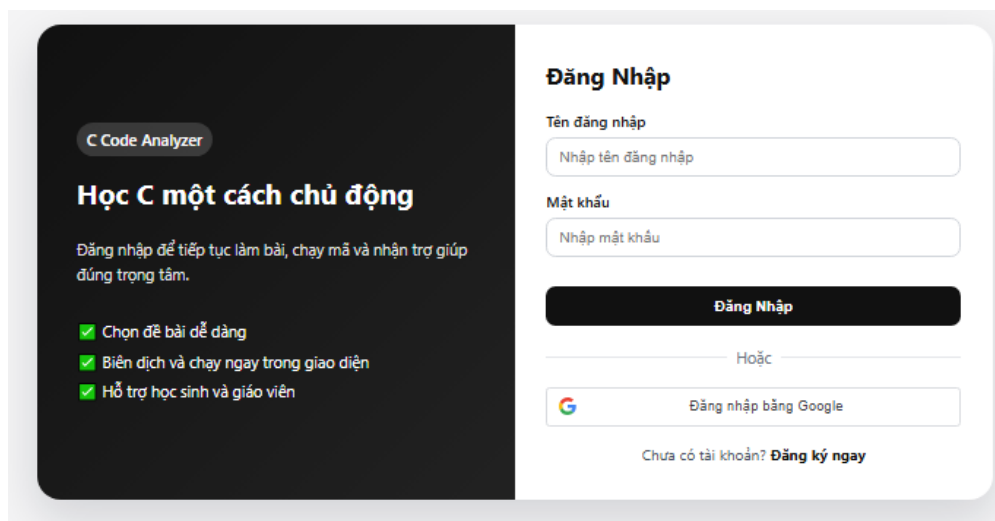
Chức năng đăng nhập là bước đầu tiên trong quá trình sử dụng hệ thống, có nhiệm vụ xác thực thông tin người dùng trước khi cho phép truy cập vào các chức năng bên trong. Việc xây dựng cơ chế đăng nhập giúp đảm bảo tính bảo mật của hệ thống, đồng thời hỗ trợ phân quyền giữa các nhóm người dùng khác nhau gồm quản trị viên, giáo viên và học sinh.

Hệ thống hỗ trợ hai hình thức đăng nhập là đăng nhập bằng tài khoản được tạo trong hệ thống và đăng nhập bằng tài khoản Google thông qua giao thức Google OAuth 2.0. Đối với hình thức đăng nhập bằng tài khoản, người dùng cần nhập tên đăng nhập và mật khẩu đã được đăng ký trước đó. Thông tin này sẽ được gửi đến máy chủ Flask để tiến hành kiểm tra với dữ liệu được lưu trong cơ sở dữ liệu SQLite. Mật khẩu được lưu dưới dạng mã hóa nhằm đảm bảo an toàn cho dữ liệu người dùng.

Bên cạnh đó, hệ thống còn tích hợp chức năng đăng nhập bằng Google OAuth 2.0 nhằm giúp người dùng có thể truy cập nhanh mà không cần tạo tài khoản mới. Sau khi người dùng xác thực thành công với Google, hệ thống sẽ nhận các thông tin cơ bản như tên hiển thị và địa chỉ thư điện tử để tạo hoặc cập nhật tài khoản trong cơ sở dữ liệu. Việc sử dụng Google OAuth giúp đơn giản hóa quá trình đăng nhập, đồng thời tăng cường tính bảo mật và nâng cao trải nghiệm sử dụng.

Sau khi quá trình xác thực hoàn tất, hệ thống sẽ kiểm tra vai trò của người dùng và điều hướng đến giao diện tương ứng. Nếu người dùng có vai trò quản trị viên, hệ thống sẽ chuyển đến trang quản lý người dùng và đề bài. Nếu là giáo viên, người dùng sẽ được chuyển đến giao diện quản lý đề bài, quản lý Test Case và theo dõi kết quả học tập của học sinh. Trong trường hợp người dùng là học sinh, hệ thống sẽ hiển thị giao diện lựa chọn đề bài và thực hiện bài lập trình.

Nếu thông tin đăng nhập không chính xác hoặc tài khoản chưa tồn tại, hệ thống sẽ hiển thị thông báo lỗi để người dùng kiểm tra và thực hiện đăng nhập lại. Đối với trường hợp đăng nhập bằng Google nhưng tài khoản chưa được cấp quyền sử dụng, hệ thống cũng sẽ từ chối truy cập và yêu cầu liên hệ quản trị viên.



Hình 3.1 Giao diện đăng nhập hệ thống

Quan sát Hình 4.1 có thể thấy giao diện được thiết kế theo hướng tối giản và trực quan. Bên trái là khu vực giới thiệu chức năng của hệ thống, trong khi bên phải là khu vực đăng nhập. Giao diện bao gồm các thành phần chính sau:

Bảng 3.1 Các thành phần trong giao diện đăng nhập

STT	Tên thành phần
1	Ô nhập tên đăng nhập.
2	Ô nhập mật khẩu.
3	Nút Đăng nhập.
4	Nút Đăng nhập bằng Google.
5	Liên kết chuyển sang trang đăng ký tài khoản.

Việc bố trí các thành phần theo dạng biểu mẫu giúp người dùng dễ dàng thao tác ngay từ lần đầu sử dụng. Đồng thời, việc kết hợp hai phương thức đăng nhập giúp hệ thống đáp ứng được nhiều nhu cầu sử dụng khác nhau.

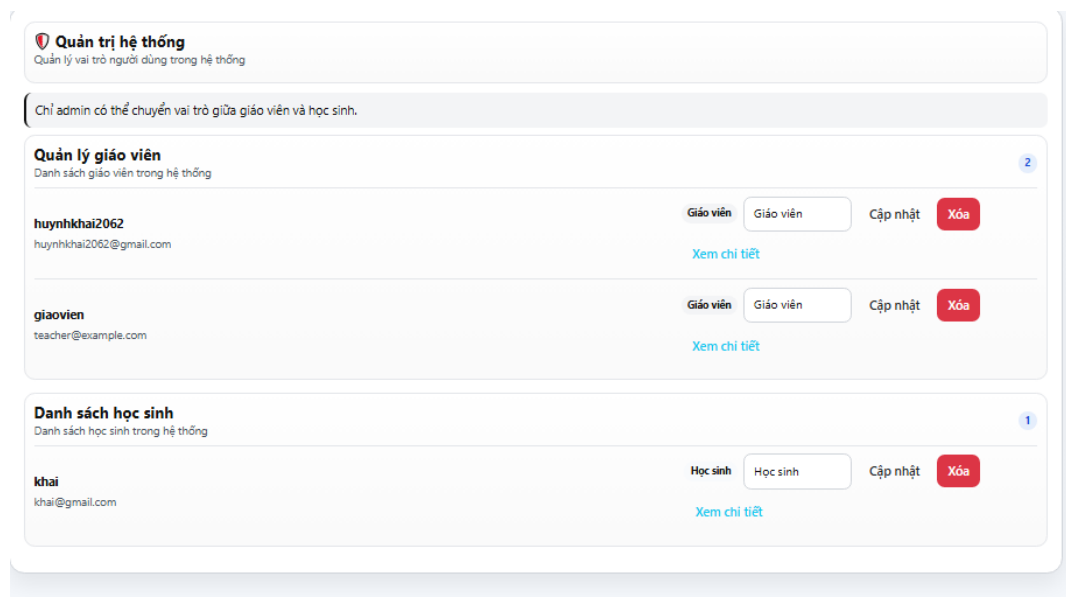
3.2 CHỨC NĂNG DÀNH CHO QUẢN TRỊ VIÊN

Quản trị viên (Administrator) là người có quyền cao nhất trong hệ thống, chịu trách nhiệm quản lý toàn bộ dữ liệu cũng như giám sát hoạt động của hệ thống. Bên cạnh việc quản lý tài khoản người dùng, quản trị viên còn có quyền tạo và cập nhật ngân hàng đề bài, xây dựng bộ Test Case phục vụ quá trình kiểm thử chương trình, đồng thời phân quyền giữa giáo viên và học sinh.

Việc phân chia quyền quản trị giúp đảm bảo dữ liệu được quản lý tập trung, hạn chế các thao tác ngoài phạm vi cho phép của từng nhóm người dùng. Điều này góp phần nâng cao tính an toàn, ổn định và thuận tiện trong quá trình vận hành hệ thống.

3.2.1 Quản lý tài khoản người dùng

Sau khi đăng nhập thành công với quyền quản trị viên, người dùng có thể truy cập chức năng Quản lý hệ thống để xem toàn bộ danh sách tài khoản hiện có. Chức năng này cho phép quản trị viên quản lý hai nhóm người dùng chính là giáo viên và học sinh.



Hình 3.2 Giao diện quản lý tài khoản người dùng

Quan sát Hình 4.2 có thể thấy giao diện được chia thành hai khu vực chính gồm danh sách giáo viên và danh sách học sinh. Mỗi tài khoản được hiển thị các thông tin cơ bản như họ tên, địa chỉ thư điện tử và vai trò hiện tại trong hệ thống.

Đối với mỗi tài khoản, quản trị viên có thể thực hiện các thao tác:

- Xem thông tin chi tiết của người dùng.
- Thay đổi vai trò giữa giáo viên và học sinh.
- Cập nhật thông tin tài khoản.
- Xóa tài khoản khỏi hệ thống.

Khi quản trị viên thay đổi vai trò của một tài khoản, hệ thống sẽ cập nhật trực tiếp vào cơ sở dữ liệu. Sau lần đăng nhập tiếp theo, người dùng sẽ được chuyển đến giao diện tương ứng với vai trò mới. Cơ chế này giúp việc phân công quyền hạn được thực hiện nhanh chóng mà không cần tạo thêm tài khoản mới.

Trong trường hợp xóa tài khoản, hệ thống sẽ kiểm tra các ràng buộc dữ liệu trước khi thực hiện thao tác nhằm đảm bảo không làm mất tính toàn vẹn của cơ sở dữ liệu. Những dữ liệu liên quan đến bài làm hoặc lịch sử hoạt động sẽ được xử lý theo quy định của hệ thống.

Thông qua chức năng quản lý tài khoản, quản trị viên có thể dễ dàng kiểm soát số lượng người dùng cũng như phân quyền phù hợp với từng đối tượng sử dụng.

3.2.2 Quản lý đề bài

Một trong những chức năng quan trọng của quản trị viên là xây dựng và quản lý ngân hàng đề bài lập trình C. Toàn bộ đề bài được lưu trữ trong cơ sở dữ liệu và được sử dụng làm cơ sở để học sinh thực hiện bài tập cũng như để hệ thống tiến hành đánh giá kết quả.

Hình 3.3 Giao diện quản lý đề bài

Giao diện quản lý đề bài bao gồm hai khu vực chính. Khu vực bên trái hiển thị danh sách các chức năng như xem danh sách đề và tạo đề mới. Khu vực bên phải hiển thị danh sách các đề bài hiện có cùng với thông tin chi tiết của đề được lựa chọn.

Khi người dùng lựa chọn một đề bài trong danh sách, toàn bộ thông tin sẽ được tải lên giao diện để phục vụ việc chỉnh sửa hoặc cập nhật. Quản trị viên có thể thay

đổi nội dung đề bài, cập nhật mã nguồn chuẩn hoặc điều chỉnh mức độ khó tùy theo yêu cầu của môn học.

Ngoài ra, hệ thống còn hỗ trợ chức năng xóa đề bài. Khi thực hiện thao tác này, hệ thống sẽ đồng thời kiểm tra các Test Case liên quan để đảm bảo tính nhất quán của dữ liệu. Việc xây dựng ngân hàng đề bài ngay trên giao diện Web giúp quá trình bổ sung nội dung học tập diễn ra nhanh chóng mà không cần can thiệp trực tiếp vào cơ sở dữ liệu.

3.2.3 Tạo đề bài mới

Để bổ sung một bài tập mới vào hệ thống, quản trị viên lựa chọn chức năng Tạo đề. Hệ thống sẽ hiển thị biểu mẫu cho phép nhập đầy đủ thông tin của đề bài.

Hình 3.4 . Giao diện tạo đề bài

Trong biểu mẫu này, quản trị viên cần nhập các thông tin sau:

- Tiêu đề đề bài.
- Mô tả bài toán.
- Yêu cầu của bài toán.
- Mã nguồn chuẩn dùng để đối chiếu kết quả.

Trong đó, mã nguồn chuẩn đóng vai trò là lời giải tham khảo do giáo viên hoặc quản trị viên xây dựng. Phần mã này không được hiển thị cho học sinh mà chỉ được sử dụng trong quá trình quản lý và hỗ trợ kiểm tra tính đúng đắn của đề bài.

Sau khi hoàn tất việc nhập dữ liệu, quản trị viên nhấn nút Tạo đề bài. Hệ thống sẽ kiểm tra tính hợp lệ của dữ liệu trước khi lưu vào cơ sở dữ liệu SQLite. Nếu quá trình lưu thành công, đề bài mới sẽ xuất hiện trong danh sách đề hiện có và sẵn sàng để bổ sung Test Case.

Việc tách riêng bước tạo đề bài và bước xây dựng Test Case giúp người quản lý dễ dàng cập nhật nội dung của từng phần mà không ảnh hưởng đến dữ liệu đã có.

3.2.4 Quản lý Test Case

Sau khi tạo thành công một đề bài, quản trị viên tiếp tục xây dựng bộ Test Case tương ứng với đề bài đó.

The screenshot displays a web interface titled "Test Cases". It contains two test case entries, each with a title, an input field, and an expected output field. The first entry is "Testcase 1: n=5" with input "5" and expected output "15". The second entry is "Testcase 2: n=3" with input "3" and expected output "6". Each entry has a red "Xóa" (Delete) button in the top right corner. Below these entries is a dashed blue box containing a form to add a new test case. The form has a title "Thêm testcase" with a plus icon, followed by three input fields: "Tên testcase", "Input", and "Expected output". Below the "Expected output" field is a green button labeled "Thêm testcase". At the bottom of the interface is a grey button with a plus icon and the text "Thêm testcase".

Hình 3.5 Giao diện quản lý Test Case

Mỗi đề bài có thể bao gồm một hoặc nhiều Test Case nhằm kiểm tra chương trình của học sinh trong nhiều trường hợp khác nhau. Việc sử dụng nhiều Test Case giúp đánh giá tính đúng đắn của chương trình một cách toàn diện hơn, đồng thời hạn chế trường hợp chương trình chỉ đúng với một số dữ liệu đầu vào cụ thể.

Bộ Test Case được lưu trữ trong cơ sở dữ liệu và liên kết trực tiếp với đề bài thông qua khóa ngoại `de_bai_id`. Khi học sinh thực hiện chương trình, hệ thống sẽ tự động truy xuất toàn bộ Test Case thuộc đề bài đã chọn để tiến hành kiểm thử.

Quan sát Hình 4.5 có thể thấy mỗi Test Case được hiển thị dưới dạng một khối thông tin riêng biệt, bao gồm tên Test Case, dữ liệu đầu vào (Input) và kết quả mong đợi (Expected Output). Cách trình bày này giúp người quản trị dễ dàng theo dõi và quản lý từng trường hợp kiểm thử.

Mỗi Test Case bao gồm các thành phần chính sau:

- Tên Test Case: dùng để phân biệt giữa các trường hợp kiểm thử khác nhau.
- Input: dữ liệu đầu vào được truyền vào chương trình khi thực thi. Nội dung Input được xây dựng dựa trên yêu cầu của đề bài và mô phỏng các tình huống thực tế mà chương trình cần xử lý.
- Expected Output: kết quả chính xác mà chương trình cần trả về tương ứng với dữ liệu đầu vào. Đây là cơ sở để hệ thống so sánh với kết quả thực tế sau khi chạy chương trình.

Ngoài việc hiển thị thông tin, hệ thống còn hỗ trợ các chức năng quản lý như:

- Thêm Test Case mới
- Chỉnh sửa nội dung Test Case
- Xóa Test Case
- Cập nhật dữ liệu đầu vào và kết quả mong đợi

Khi quản trị viên lựa chọn chức năng Thêm Test Case, hệ thống sẽ hiển thị biểu mẫu cho phép nhập đầy đủ các thông tin cần thiết. Sau khi dữ liệu được kiểm tra hợp lệ, Test Case sẽ được lưu vào bảng `bo_testcase` trong cơ sở dữ liệu và tự động liên kết với đề bài hiện tại.

Trong quá trình chỉnh sửa, quản trị viên có thể thay đổi dữ liệu đầu vào hoặc kết quả mong đợi mà không cần tạo lại toàn bộ Test Case. Điều này giúp việc cập nhật nội dung bài tập trở nên thuận tiện hơn khi phát hiện sai sót hoặc muốn mở rộng khả năng kiểm thử của đề bài.

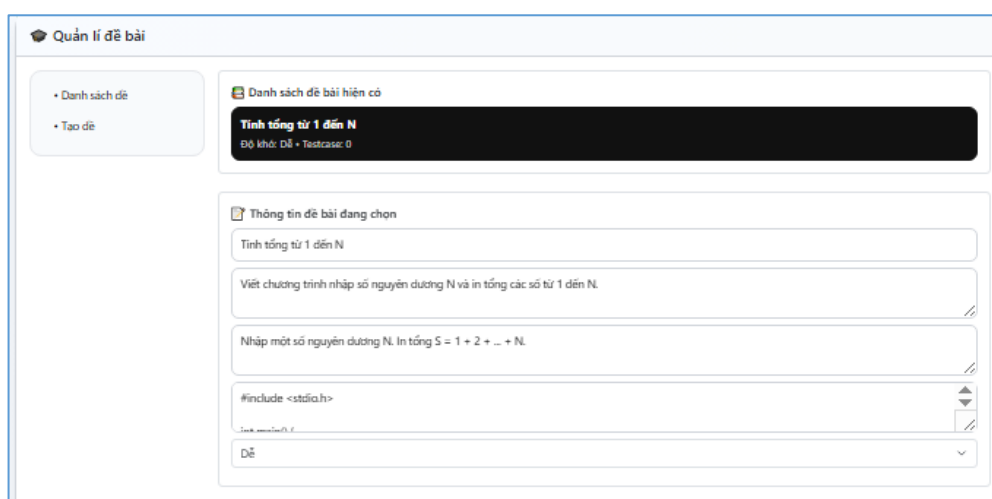
Chức năng xóa Test Case cũng được hỗ trợ nhằm loại bỏ các trường hợp kiểm thử không còn phù hợp. Sau khi xóa, Test Case sẽ không còn được sử dụng trong quá trình đánh giá chương trình của học sinh.

3.3 CHỨC NĂNG DÀNH CHO GIÁO VIÊN

Bên cạnh quản trị viên, hệ thống còn cung cấp nhóm chức năng dành riêng cho giáo viên nhằm phục vụ công tác xây dựng nội dung giảng dạy và theo dõi kết quả học tập của học sinh. Giáo viên có quyền tạo và quản lý đề bài lập trình, xây dựng bộ Test Case tương ứng với từng đề bài cũng như theo dõi bài làm của học sinh sau khi nộp lên hệ thống.

Khác với quản trị viên, giáo viên không có quyền quản lý tài khoản hoặc phân quyền người dùng. Việc giới hạn quyền truy cập giúp đảm bảo tính bảo mật của hệ thống, đồng thời phân chia rõ trách nhiệm giữa các nhóm người dùng.

3.3.1 Quản lý đề bài



Hình 3.6 Giao diện quản lý đề bài của giáo viên

Sau khi đăng nhập thành công với vai trò giáo viên, người dùng có thể truy cập chức năng Quản lý đề bài để xây dựng và cập nhật các bài tập lập trình C phục vụ cho quá trình giảng dạy.

Giao diện quản lý đề bài bao gồm danh sách các đề hiện có và khu vực hiển thị thông tin chi tiết của đề bài đang được lựa chọn. Mỗi đề bài lưu trữ các thông tin như tiêu đề, mô tả, yêu cầu, mã nguồn chuẩn và mức độ khó.

Giáo viên có thể thực hiện các chức năng sau:

- Xem danh sách đề bài
- Thêm đề bài mới
- Chính sửa nội dung đề bài
- Cập nhật mã nguồn chuẩn
- Thay đổi mức độ khó của đề bài
- Xóa đề bài khi không còn sử dụng

Trong quá trình tạo đề bài mới, giáo viên nhập đầy đủ thông tin theo biểu mẫu của hệ thống. Sau khi hoàn tất, dữ liệu sẽ được gửi đến Backend Flask để kiểm tra tính hợp lệ trước khi lưu vào cơ sở dữ liệu SQLite. Việc xây dựng đề bài trực tiếp trên hệ thống giúp giáo viên chủ động cập nhật nội dung học tập mà không cần thao tác trực tiếp với cơ sở dữ liệu, đồng thời giúp học sinh luôn được tiếp cận với phiên bản mới nhất của từng bài tập.

3.3.2 Quản lý Test Case

The screenshot displays a web interface titled "Test Cases". It contains two existing test cases and a form to add a new one.

Testcase ID	Input	Expected	Action
Testcase 1: n=5	5	15	Xóa
Testcase 2: n=3	3	6	Xóa

Below the list is a dashed box containing a form to "Thêm testcase" (Add test case). The form includes:

- A header "Thêm testcase" with a plus icon.
- A text input field labeled "Tên testcase".
- A text input field labeled "Input".
- A text input field labeled "Expected output".
- A green button labeled "Thêm testcase".
- A grey button labeled "Thêm testcase" at the bottom.

Hình 3.7 Giao diện quản lý Test Case

Sau khi tạo đề bài, giáo viên tiếp tục xây dựng bộ Test Case nhằm phục vụ việc đánh giá chương trình của học sinh.

Mỗi Test Case bao gồm ba thành phần chính:

- Tên Test Case.
- Dữ liệu đầu vào.
- Kết quả mong đợi.

Giáo viên có thể thêm nhiều Test Case cho cùng một đề bài nhằm kiểm tra chương trình trong nhiều trường hợp khác nhau. Các Test Case được lưu trong bảng `bo_testcase` và liên kết với bảng `de_bai` thông qua khóa ngoại `de_bai_id`.

Hệ thống cho phép giáo viên thực hiện các thao tác:

- Thêm Test Case mới.
- Chỉnh sửa dữ liệu Input.
- Chỉnh sửa Expected Output.
- Xóa Test Case.

Trong quá trình học sinh thực hiện bài tập, toàn bộ Test Case sẽ được hệ thống tự động lấy từ cơ sở dữ liệu để kiểm thử chương trình. Nhờ đó giáo viên không cần trực tiếp chấm bài mà vẫn đảm bảo kết quả đánh giá được thực hiện khách quan và nhất quán.

3.3.3 Theo dõi bài làm của học sinh

Ngoài chức năng xây dựng đề bài, giáo viên còn có thể theo dõi kết quả làm bài của học sinh thông qua chức năng **Xem bài làm học sinh**.



Hình 3.8 Giao diện theo dõi bài làm của học sinh

Giao diện hiển thị danh sách học sinh đã tham gia làm bài trên hệ thống. Đối với mỗi học sinh, giáo viên có thể xem các thông tin như:

- Họ và tên học sinh
- Địa chỉ thư điện tử
- Danh sách bài đã thực hiện
- Kết quả biên dịch chương trình
- Kết quả thực hiện các Test Case
- Nội dung mã nguồn đã nộp

Thông qua các thông tin này, giáo viên có thể đánh giá mức độ hoàn thành của từng học sinh, xác định những lỗi thường gặp cũng như theo dõi quá trình học tập của người học theo từng đề bài.

Việc lưu trữ toàn bộ bài làm trên hệ thống còn giúp giáo viên dễ dàng tra cứu lại kết quả ở những lần thực hành trước, phục vụ cho quá trình đánh giá và hỗ trợ học sinh trong các buổi học tiếp theo.

Nhìn chung, nhóm chức năng dành cho giáo viên giúp đơn giản hóa quá trình xây dựng bài tập và theo dõi kết quả học tập. Thay vì phải chuẩn bị dữ liệu kiểm thử và chấm bài thủ công, giáo viên chỉ cần xây dựng đề bài và Test Case một lần, hệ thống sẽ tự động thực hiện việc kiểm thử, lưu trữ kết quả và hỗ trợ đánh giá trong suốt quá trình sử dụng. Điều này góp phần giảm đáng kể thời gian chấm bài, đồng thời nâng cao hiệu quả giảng dạy và quản lý học tập.

3.4 CHỨC NĂNG DÀNH CHO HỌC SINH

Chức năng dành cho học sinh là chức năng trung tâm của hệ thống, được xây dựng nhằm hỗ trợ người học thực hành lập trình ngôn ngữ C một cách trực quan và hiệu quả. Thay vì chỉ viết chương trình và kiểm tra kết quả bằng các công cụ biên dịch truyền thống, học sinh có thể thực hiện toàn bộ quá trình lập trình ngay trên hệ thống, từ lựa chọn đề bài, viết mã nguồn, biên dịch, thực thi chương trình đến nhận các gợi ý sửa lỗi từ trí tuệ nhân tạo.

Toàn bộ quá trình thực hiện bài tập được tự động hóa. Sau khi học sinh hoàn thành chương trình, hệ thống sẽ sử dụng trình biên dịch GCC để kiểm tra lỗi cú pháp, thực thi chương trình với bộ Test Case của đề bài và so sánh kết quả đầu ra với kết quả mong đợi. Nếu chương trình chưa đạt yêu cầu, học sinh có thể sử dụng chức năng AI Help để được hỗ trợ phân tích nguyên nhân và gợi ý hướng sửa lỗi.

3.4.1 Lựa chọn đề bài

Sau khi đăng nhập thành công, học sinh sẽ được chuyển đến giao diện danh sách đề bài. Tại đây, hệ thống hiển thị toàn bộ các bài tập do giáo viên xây dựng. Mỗi đề bài bao gồm tiêu đề, mô tả ngắn và mức độ khó, giúp học sinh dễ dàng lựa chọn bài tập phù hợp với khả năng của mình.

Bảng điều khiển học tập

Chọn đề bài, viết mã và nhận kết quả ngay trong một nơi

Giao diện được sắp xếp theo hướng dẫn thực hành: editor ở trung tâm và kết quả ở dưới để bạn đọc dễ hơn.

Chọn đề bài

Chọn đề bài để làm

3 Đã sẵn sàng

ĐỘ KHÓ: **Dễ**

TESTCASE: 0 mẫu

TIÊU ĐỀ: 3

MÔ TẢ: 3

YÊU CẦU: 3

Hình 3.9 Giao diện danh sách đề bài

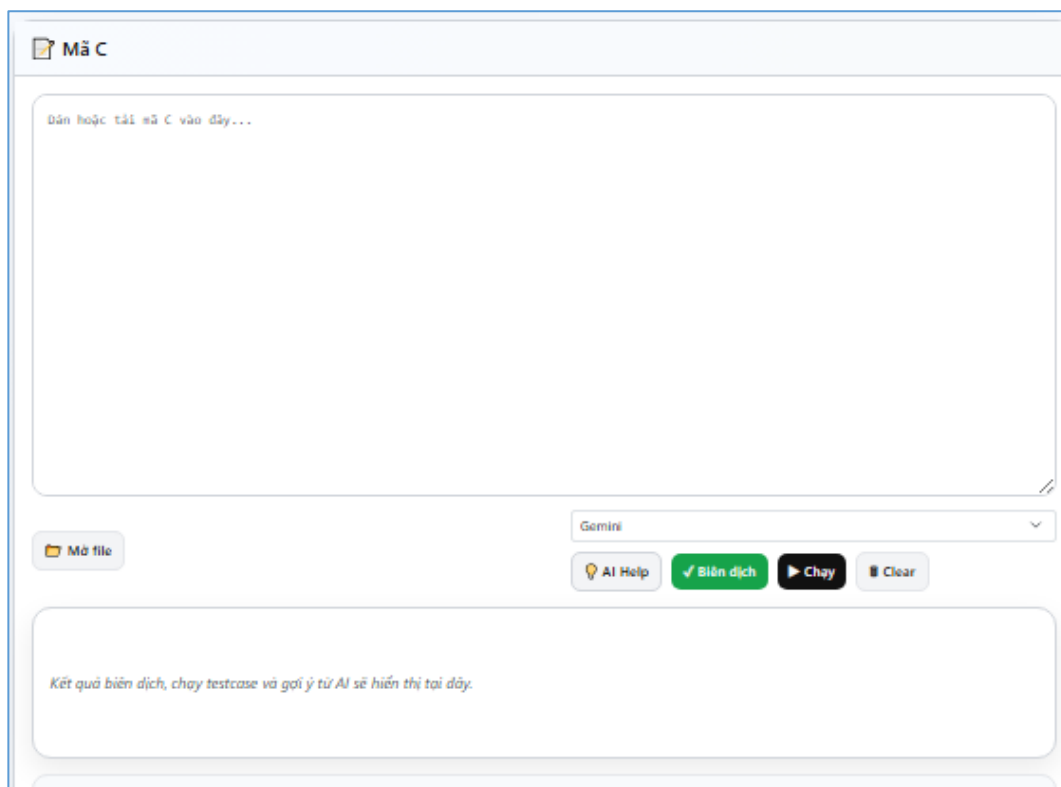
Khi lựa chọn một đề bài, hệ thống sẽ hiển thị đầy đủ các thông tin gồm:

- Tiêu đề đề bài
- Nội dung mô tả bài toán
- Yêu cầu cần thực hiện
- Mức độ khó
- Mã nguồn khởi động

Mã nguồn khởi động được cung cấp nhằm giúp học sinh nhanh chóng bắt đầu bài tập mà không phải xây dựng toàn bộ cấu trúc chương trình từ đầu. Đối với những đề bài không yêu cầu mã khởi động, học sinh có thể tự xây dựng chương trình theo yêu cầu. Việc tách riêng danh sách đề bài và giao diện làm bài giúp học sinh dễ dàng chuyển đổi giữa các bài tập cũng như theo dõi tiến độ học tập của mình.

3.4.2 Nhập mã nguồn chương trình

Sau khi lựa chọn đề bài, học sinh tiến hành xây dựng chương trình bằng ngôn ngữ C.



Hình 3.10 Giao diện nhập mã nguồn

- Hệ thống hỗ trợ hai phương thức nhập chương trình:
- Nhập trực tiếp mã nguồn trên trình soạn thảo tích hợp.
- Tải lên chương trình có phần mở rộng **.c**.

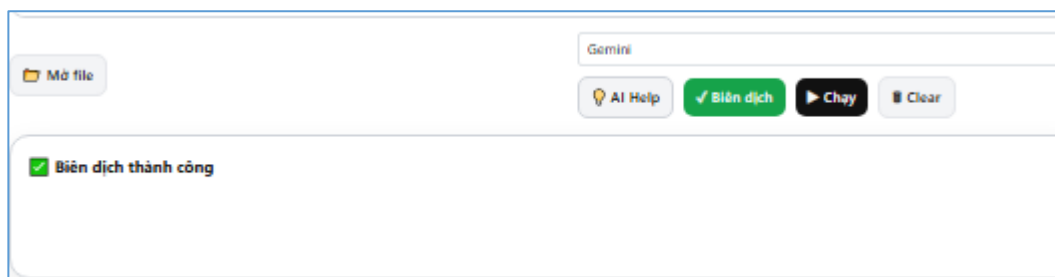
Đối với phương thức nhập trực tiếp, học sinh có thể chỉnh sửa chương trình ngay trên trình duyệt mà không cần sử dụng phần mềm lập trình bên ngoài. Điều này giúp giảm thời gian thao tác và tạo sự thuận tiện trong quá trình học tập.

Trong trường hợp chương trình đã được viết sẵn bằng các môi trường như Visual Studio Code hoặc Code::Blocks, học sinh chỉ cần lựa chọn tệp nguồn và tải lên hệ thống. Backend sẽ tự động đọc nội dung tệp và hiển thị trong trình soạn thảo để tiếp tục thực hiện các thao tác tiếp theo.

Ngoài khu vực nhập mã nguồn, giao diện còn cho phép học sinh lựa chọn mô hình AI sẽ được sử dụng khi cần hỗ trợ phân tích lỗi logic.

3.4.3 Biên dịch chương trình

Sau khi hoàn thành chương trình, học sinh nhấn nút **Biên dịch** để kiểm tra lỗi cú pháp.



Hình 3.11 Chức năng biên dịch

Khi nhận được yêu cầu, Backend Flask sẽ tạo một tệp mã nguồn tạm thời và sử dụng trình biên dịch GCC để tiến hành biên dịch chương trình. Trong quá trình này, hệ thống sẽ ghi nhận toàn bộ thông báo do GCC trả về.

Nếu chương trình tồn tại lỗi cú pháp, hệ thống sẽ hiển thị chi tiết các thông báo lỗi, bao gồm vị trí dòng xảy ra lỗi và nguyên nhân gây lỗi. Điều này giúp học sinh nhanh chóng xác định và sửa lỗi trước khi chuyển sang bước thực thi chương trình.

Trong trường hợp chương trình được biên dịch thành công, hệ thống sẽ tạo tệp thực thi và thông báo trạng thái thành công để học sinh tiếp tục bước kiểm thử với Test Case.

Việc tách riêng bước biên dịch giúp người học phân biệt rõ giữa lỗi cú pháp và lỗi logic, tránh nhầm lẫn trong quá trình sửa chương trình.

3.4.4 Thực thi chương trình và kiểm thử bằng Test Case

Sau khi biên dịch thành công, học sinh lựa chọn chức năng Chạy chương trình để kiểm tra kết quả thực hiện.

Backend sẽ truy xuất toàn bộ Test Case của đề bài từ cơ sở dữ liệu, sau đó lần lượt truyền dữ liệu đầu vào của từng Test Case vào chương trình thông qua luồng nhập chuẩn.

3.4.5 Hỗ trợ phân tích bằng AI

Một trong những chức năng nổi bật của hệ thống là AI Help, được xây dựng nhằm hỗ trợ học sinh trong quá trình tìm và sửa lỗi logic.

Khi học sinh nhấn nút AI Help, Backend sẽ tổng hợp các thông tin gồm:

- Nội dung đề bài
- Mã nguồn chương trình

- Kết quả biên dịch
- Kết quả thực hiện Test Case

Các thông tin này được xây dựng thành Prompt và gửi đến mô hình AI thông qua API.

Sau khi nhận được phản hồi, hệ thống sẽ hiển thị:

- Phân tích nguyên nhân gây lỗi.
- Các lỗi logic có thể tồn tại trong chương trình.
- Giải thích vì sao chương trình chưa đạt yêu cầu.
- Gợi ý hướng khắc phục.
- Đề xuất cách cải thiện thuật toán.

Khác với các trình biên dịch truyền thống chỉ phát hiện lỗi cú pháp, AI có khả năng phân tích ngữ cảnh của chương trình và đánh giá thuật toán dựa trên yêu cầu của đề bài. Nhờ đó, học sinh không chỉ biết chương trình sai mà còn hiểu được nguyên nhân và cách sửa lỗi.

Các gợi ý do AI đưa ra chỉ mang tính chất định hướng, không cung cấp trực tiếp lời giải hoàn chỉnh. Điều này giúp học sinh rèn luyện khả năng tư duy và tự hoàn thiện chương trình của mình.

Thông qua các chức năng trên, học sinh có thể hoàn thành toàn bộ quá trình thực hành lập trình C trên cùng một hệ thống mà không cần sử dụng thêm các công cụ hỗ trợ khác. Đây cũng là điểm khác biệt nổi bật của hệ thống so với các môi trường biên dịch trực tuyến thông thường, khi kết hợp giữa kiểm thử tự động bằng Test Case và khả năng hỗ trợ học tập của trí tuệ nhân tạo.

3.5 KIỂM THỬ HỆ THỐNG

Chọn một đề bài ngẫu nhiên để kiểm thử

Chọn đề bài

Chọn đề bài để làm

Tính tổng các số chẵn từ 1 đến N

Đã sẵn sàng

ĐỘ KHÓ

Đễ

TESTCASE

10 mẫu

TIÊU ĐỀ

Tính tổng các số chẵn từ 1 đến N

MÔ TẢ

Viết chương trình nhập vào một số nguyên dương N, sau đó tính tổng tất cả các số chẵn trong đoạn từ 1 đến N (bao gồm cả N nếu N là số chẵn).

YÊU CẦU

Nhập một số nguyên dương N. Tính tổng các số chẵn từ 1 đến N. In ra tổng vừa tính được Đầu vào Một số nguyên dương N. Đầu ra Một số nguyên là tổng các số chẵn từ 1 đến N. Ràng buộc $1 \leq N \leq 100000$.

Hình 3.12 Chọn đề bài

Nhập mã nguồn bị sai lỗi logic

Mã C

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d",&n);

    int sum = 0;

    for(int i = 2; i < n; i += 2)
        sum += i;

    printf("%d",sum);

    return 0;
}
```

Mở file

Gemini

AI Help

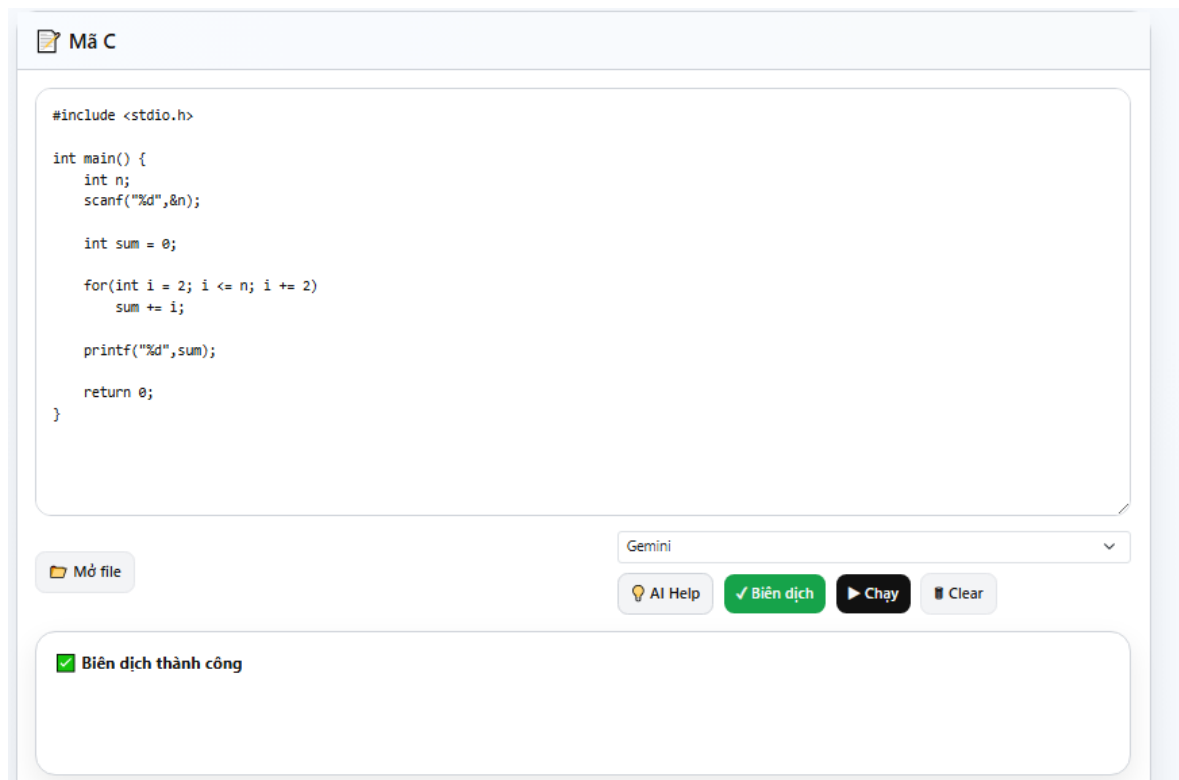
Biên dịch

Chạy

Clear

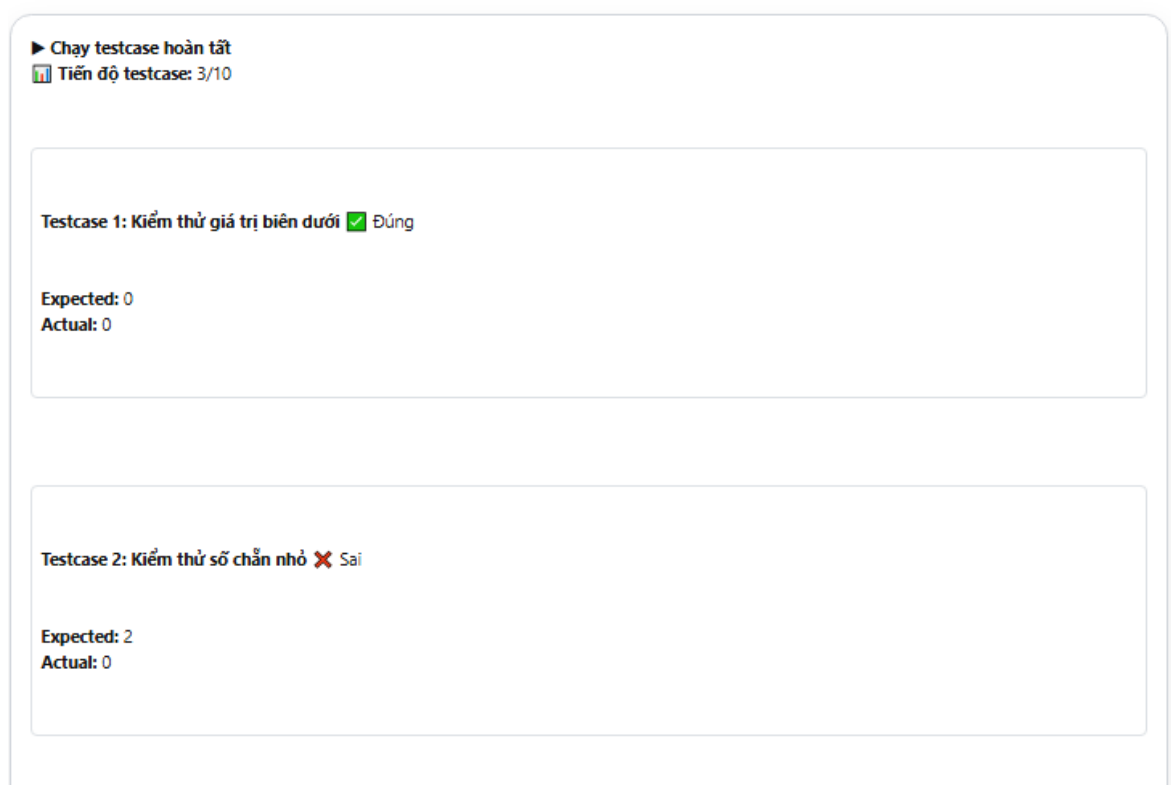
Hình 3.13 Nhập mã nguồn

Do mã nguồn bị sai lỗi logic nên kết quả biên dịch sẽ là thành công.



Hình 3.14 Kết quả biên dịch

Chạy TestCase để kiểm thử, đoạn mã chỉ đạt 3/10 TestCase.



Hình 3.15 Kết quả TestCase

Kết quả phân tích từ AI như sau:

Gợi ý từ AI:

****1) Xác định lỗi chính****

- Vòng lặp 'for' chỉ cộng các số chẵn **đến trước n** ($i < n$). Khi 'n' là số chẵn, giá trị 'n' không được cộng vào tổng.
- Kết quả nên là tổng các số chẵn **đến n (kể cả n)** , nhưng chương trình hiện tại bỏ qua 'n' nếu 'n' chẵn.

****2) Giải thích tại sao sai****

- Định nghĩa "tổng các số chẵn từ 2 tới n" thường hiểu là bao gồm cả 'n' nếu 'n' là số chẵn.
- Khi điều kiện $i < n$ được dùng, vòng lặp dừng khi 'i' đạt 'n-1'. Nếu 'n' là 10, vòng lặp chạy với $i = 2, 4, 6, 8$ và bỏ qua 10.
- Do đó, khi người dùng nhập '10', kết quả trả về là '20' ($2+4+6+8$) thay vì '30' ($2+4+6+8+10$).

Hình 3.16 Kết quả phân tích từ AI

CHƯƠNG 4 KẾT LUẬN VÀ KIẾN NGHỊ

4.1 KẾT LUẬN

Trong bối cảnh trí tuệ nhân tạo ngày càng được ứng dụng rộng rãi trong lĩnh vực giáo dục và hỗ trợ lập trình, việc xây dựng một hệ thống giúp người học phát hiện và khắc phục lỗi trong chương trình là một hướng nghiên cứu có tính ứng dụng cao. Đề tài “Xây dựng hệ thống xphân tích và gợi ý sửa lỗi logic mã nguồn C” được thực hiện với mục tiêu hỗ trợ học sinh và giáo viên trong quá trình dạy và học lập trình ngôn ngữ C thông qua môi trường Web kết hợp với công nghệ trí tuệ nhân tạo.

Trong quá trình thực hiện đề tài, em đã nghiên cứu các kiến thức liên quan đến ngôn ngữ lập trình C, đặc điểm của lỗi logic trong chương trình, kỹ thuật biên dịch và thực thi chương trình bằng trình biên dịch GCC, phương pháp xây dựng ứng dụng Web với Flask, quản lý dữ liệu bằng SQLite cũng như tích hợp các mô hình trí tuệ nhân tạo thông qua API để hỗ trợ phân tích mã nguồn.

Từ những cơ sở lý thuyết đã nghiên cứu, hệ thống đã được phân tích, thiết kế và xây dựng hoàn chỉnh theo mô hình Client–Server. Backend được phát triển bằng Flask, chịu trách nhiệm xử lý các yêu cầu từ người dùng, quản lý cơ sở dữ liệu, thực hiện biên dịch chương trình, chạy các bộ Test Case và giao tiếp với dịch vụ AI. Frontend được xây dựng bằng HTML, CSS và JavaScript nhằm cung cấp giao diện trực quan, thân thiện và dễ sử dụng.

Qua quá trình kiểm thử, hệ thống hoạt động ổn định đối với các chức năng chính. Kết quả biên dịch và thực thi chương trình được thực hiện chính xác, các Test Case được xử lý tự động và hiển thị kết quả rõ ràng. Chức năng AI Help có khả năng hỗ trợ người học phân tích nguyên nhân gây lỗi và đưa ra các gợi ý phù hợp dựa trên nội dung đề bài, mã nguồn và kết quả kiểm thử.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế. Khả năng phân tích của AI phụ thuộc vào chất lượng phản hồi từ mô hình được sử dụng, do đó độ chính xác của các gợi ý có thể thay đổi trong một số trường hợp. Ngoài ra, hệ thống hiện mới hỗ trợ ngôn ngữ lập trình C và chưa đáp ứng được nhu cầu đối với các ngôn ngữ lập trình khác.

Nhìn chung, các mục tiêu đề ra ban đầu đã được hoàn thành. Hệ thống không chỉ hỗ trợ kiểm tra và đánh giá chương trình thông qua Test Case mà còn kết hợp trí tuệ nhân tạo để phân tích và gợi ý sửa lỗi logic. Đây là một công cụ hỗ trợ học tập hữu ích, góp phần nâng cao hiệu quả học lập trình và giảm khối lượng công việc cho giáo viên trong quá trình hướng dẫn và đánh giá bài tập.

4.2 HƯỚNG PHÁT TRIỂN

Mặc dù hệ thống đã đáp ứng được các mục tiêu của đề tài, vẫn còn nhiều hướng có thể tiếp tục nghiên cứu và phát triển nhằm nâng cao hiệu quả sử dụng cũng như mở rộng phạm vi ứng dụng.

Thứ nhất, hệ thống có thể mở rộng hỗ trợ thêm nhiều ngôn ngữ lập trình khác như C++, Java, Python hoặc JavaScript. Việc hỗ trợ đa ngôn ngữ sẽ giúp hệ thống phù hợp hơn với nhiều môn học và đối tượng người dùng khác nhau.

Thứ hai, có thể nâng cấp chức năng phân tích của AI bằng cách xây dựng Prompt chuyên biệt cho từng dạng bài hoặc từng chủ đề lập trình. Đồng thời, hệ thống có thể tích hợp các mô hình AI mới có khả năng xử lý mã nguồn tốt hơn nhằm nâng cao độ chính xác của các gợi ý sửa lỗi.

Thứ ba, hệ thống có thể bổ sung chức năng đánh giá chất lượng mã nguồn, bao gồm phân tích độ phức tạp thuật toán, phát hiện đoạn mã trùng lặp, kiểm tra quy tắc đặt tên biến và đánh giá khả năng tối ưu của chương trình. Những thông tin này sẽ giúp người học không chỉ sửa lỗi mà còn cải thiện kỹ năng lập trình.

Thứ tư, có thể xây dựng cơ chế thống kê và phân tích kết quả học tập. Giáo viên sẽ có thể theo dõi tiến độ học tập, số lượng bài hoàn thành, tỷ lệ vượt qua Test Case, các lỗi thường gặp và mức độ cải thiện của từng học sinh theo thời gian. Những dữ liệu này sẽ hỗ trợ quá trình đánh giá và điều chỉnh phương pháp giảng dạy.

Thứ năm, hệ thống có thể phát triển chức năng quản lý lớp học, cho phép giáo viên tạo lớp, phân công bài tập theo từng lớp hoặc từng nhóm học sinh, quy định thời hạn nộp bài và theo dõi kết quả thực hiện của từng thành viên. Điều này giúp hệ thống trở thành một nền tảng hỗ trợ giảng dạy lập trình hoàn chỉnh.

Ngoài ra, hệ thống cũng có thể được triển khai trên nền tảng điện toán đám mây nhằm tăng khả năng mở rộng, hỗ trợ nhiều người dùng truy cập đồng thời và

nâng cao tính ổn định. Việc tối ưu hiệu năng thực thi chương trình và tăng cường các cơ chế bảo mật cũng là những hướng phát triển cần được quan tâm trong tương lai.

Với những định hướng trên, hệ thống không chỉ dừng lại ở vai trò là công cụ hỗ trợ phát hiện và gợi ý sửa lỗi logic trong chương trình C mà còn có thể phát triển thành một nền tảng hỗ trợ giảng dạy và học tập lập trình thông minh, góp phần nâng cao chất lượng đào tạo trong lĩnh vực công nghệ thông tin.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] H. Keuning, J. T. Jeuring, and B. Heeren, ‘A Systematic Literature Review of Automated Feedback Generation for Programming Exercises’, *ACM Trans. Comput. Educ.*, vol. 19, no. 1, pp. 1–43, 2018, doi: 10.1145/3231711.
- [2] S. MacNeil *et al.*, ‘Decoding Logic Errors: A Comparative Study on Bug Detection by Students and Large Language Models’, in *Proceedings of the 26th Australasian Computing Education Conference*, in ACE 2024. New York, NY, USA: Association for Computing Machinery, 2024, pp. 11–18. doi: 10.1145/3636243.3636245.
- [3] H. Heickal and A. S. Lan, ‘Generating Feedback-Ladders for Logical Errors in Programming using Large Language Models’, in *Proceedings of the 17th International Conference on Educational Data Mining*, in EDM 2024. Atlanta, GA, USA, 2024. [Online]. Available: <https://educationaldatamining.org/edm2024/proceedings/2024.EDM-posters.114/>
- [4] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd edn. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.
- [5] K. J. Millman and M. Aivazis, ‘Python for Scientists and Engineers’, *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 9–12, Mar. 2011, doi: 10.1109/MCSE.2011.36.
- [6] X.-K. Wu *et al.*, ‘LLM Fine-Tuning: Concepts, Opportunities, and Challenges’, *Big Data Cogn. Comput.*, vol. 9, no. 4, p. 87, Apr. 2025, doi: 10.3390/bdcc9040087.
- [7] E. A. Santos and B. A. Becker, ‘Not the Silver Bullet: LLM-enhanced Programming Error Messages are Ineffective in Practice’, in *Proceedings of the 2024 Conference on United Kingdom & Ireland Computing Education Research*, in UKICER 2024. New York, NY, USA: Association for Computing Machinery, 2024. doi: 10.1145/3689535.3689554.
- [8] G. Allen and M. Owens, *The Definitive Guide to SQLite*, 2nd edn. Berkeley, CA, USA: Apress, 2010. doi: 10.1007/978-1-4302-3226-1.

PHỤ LỤC

BỘ ĐỀ 1

Tiêu đề: Tính tổng các số chẵn từ 1 đến N

Mô tả

Viết chương trình nhập vào một số nguyên dương N. Hãy tính tổng tất cả các số chẵn trong đoạn từ 1 đến N (bao gồm N nếu N là số chẵn) và in kết quả ra màn hình.

Yêu cầu

- Nhập một số nguyên dương N.
- Tính tổng các số chẵn từ 1 đến N.
- In ra tổng các số chẵn.
- Chương trình phải xử lý đúng với mọi giá trị của N trong phạm vi cho phép.

Độ khó: Dễ

Mã chuẩn:

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int sum = 0;

    for (int i = 2; i <= n; i += 2) {
        sum += i;
    }

    printf("%d", sum);

    return 0;
}
```

Danh sách Test Case

Tên Test Case	Input	Output
Test Case 1: Kiểm thử giá trị biên dưới	1	0
Test Case 2: Kiểm thử số chẵn nhỏ nhất	2	2
Test Case 3: Kiểm thử số lẻ nhỏ	5	6
Test Case 4: Kiểm thử trường hợp thông thường	4	6
Test Case 5: Kiểm thử nhiều số chẵn	10	30
Test Case 6: Kiểm thử dữ liệu trung bình	20	110
Test Case 7: Kiểm thử dữ liệu lớn	100	2550
Test Case 8: Kiểm thử số chẵn liên tiếp	8	20
Test Case 9: Kiểm thử số lẻ lớn	9	20
Test Case 10: Kiểm thử giá trị lớn	50	650

BỘ ĐỀ 2

Tiêu đề: Tìm giá trị lớn nhất trong mảng

Mô tả: Viết chương trình nhập vào một số nguyên N và một mảng gồm N số nguyên. Hãy tìm và in ra giá trị lớn nhất trong mảng.

Yêu cầu

- Nhập số nguyên N là số lượng phần tử của mảng.
- Nhập N số nguyên.
- Tìm phần tử có giá trị lớn nhất trong mảng.
- In ra giá trị lớn nhất tìm được.
- Chương trình phải xử lý đúng với các trường hợp mảng có số âm, số dương và các phần tử trùng nhau.

Độ khó: Dễ

Mã chuẩn

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    int max = a[0];

    for (int i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
    }

    printf("%d", max);

    return 0;
}
```

Danh sách Test Case

Tên Test Case	Input	Output
Test Case 1: Kiểm thử mảng chỉ có một phần tử	1 99	99
Test Case 2: Kiểm thử mảng tăng dần	5 1 2 3 4 5	5
Test Case 3: Kiểm thử mảng giảm dần	5 5 4 3 2 1	5
Test Case 4: Kiểm thử mảng toàn số âm	4 -1 -2 -3 -4	-1
Test Case 5: Kiểm thử các phần tử bằng nhau	6 8 8 8 8 8 8	8
Test Case 6: Kiểm thử mảng có giá trị 0	5 -5 0 -2 -1 -9	0
Test Case 7: Kiểm thử giá trị lớn nhất ở đầu mảng	3 100 50 99	100

Test Case 8: Kiểm thử giá trị lớn nhất ở giữa mảng	4 7 9 1 8	9
Test Case 9: Kiểm thử mảng toàn số âm lớn	5 -100 -50 -20 - 10 -1	-1
Test Case 10: Kiểm thử mảng gồm hai phần tử	2 999 1000	1000

BỘ ĐỀ 3

Tiêu đề: Kiểm tra số nguyên tố

Mô tả: Viết chương trình nhập vào một số nguyên N. Hãy kiểm tra xem N có phải là số nguyên tố hay không. Nếu là số nguyên tố, in ra YES, ngược lại in ra NO.

Yêu cầu

- Nhập một số nguyên N.
- Kiểm tra xem N có phải là số nguyên tố hay không.
- Nếu N là số nguyên tố, in YES.
- Nếu N không phải số nguyên tố, in NO.
- Chương trình phải xử lý đúng với các trường hợp số âm, số 0, số 1 và các số nguyên dương.

Độ khó: Trung bình

Mã chuẩn:

```
#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);

    if (n < 2) {
        printf("NO");
        return 0;
    }

    for (int i = 2; i <= sqrt(n); i++) {
```

```

        if (n % i == 0) {
            printf("NO");
            return 0;
        }

    printf("YES");

    return 0;
}

```

Danh sách Test Case

Tên Test Case	Input	Output
Test Case 1: Kiểm thử số nguyên tố nhỏ nhất	2	YES
Test Case 2: Kiểm thử số nguyên tố nhỏ	3	YES
Test Case 3: Kiểm thử hợp số chẵn	4	NO
Test Case 4: Kiểm thử hợp số lẻ	9	NO
Test Case 5: Kiểm thử giá trị biên dưới	1	NO
Test Case 6: Kiểm thử giá trị bằng 0	0	NO
Test Case 7: Kiểm thử số âm	-7	NO
Test Case 8: Kiểm thử số nguyên tố lớn	17	YES
Test Case 9: Kiểm thử số chính phương	25	NO
Test Case 10: Kiểm thử số nguyên tố lớn hơn	97	YES

BỘ ĐỀ 4

Tiêu đề: Đếm số phần tử chia hết cho 3 trong mảng

Mô tả: Viết chương trình nhập vào một số nguyên N và một mảng gồm N số nguyên. Hãy đếm số lượng phần tử trong mảng chia hết cho 3 và in kết quả ra màn hình.

Yêu cầu

- Nhập số nguyên N là số lượng phần tử của mảng.
- Nhập N số nguyên.
- Đếm số phần tử chia hết cho 3.

- In ra số lượng phần tử chia hết cho 3.
- Chương trình phải xử lý đúng với các trường hợp có số âm, số 0 và không có phần tử nào chia hết cho 3.

Độ khó: Trung bình

Mã chuẩn

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int count = 0;

    for (int i = 0; i < n; i++) {
        int x;
        scanf("%d", &x);

        if (x % 3 == 0) {
            count++;
        }
    }

    printf("%d", count);

    return 0;
}
```

Danh sách Test Case

Tên Test Case	Input	Output
Test Case 1: Kiểm thử nhiều số chia hết cho 3	5 3 6 9 1 2	3
Test Case 2: Kiểm thử không có số chia hết cho 3	4 1 2 4 5	0
Test Case 3: Kiểm thử có giá trị bằng 0	3 0 3 6	3
Test Case 4: Kiểm thử mảng có số âm	5 -3 -6 1 2 3	3
Test Case 5: Kiểm thử mảng chỉ có một phần tử	1 9	1

Test Case 6: Kiểm thử dữ liệu hỗn hợp	6 1 3 5 6 8 12	3
Test Case 7: Kiểm thử chỉ có một số chia hết cho 3	4 -1 -2 -3 -4	1
Test Case 8: Kiểm thử không có phần tử hợp lệ	2 7 8	0
Test Case 9: Kiểm thử tất cả phần tử chia hết cho 3	5 15 18 21 24 27	5
Test Case 10: Kiểm thử phần tử chia hết ở giữa mảng	5 2 4 6 8 10	1

BỘ ĐỀ 5

Tiêu đề: Xếp loại học sinh theo điểm trung bình

Mô tả: Viết chương trình nhập vào điểm trung bình của một học sinh (thang điểm 10). Dựa trên điểm trung bình, hãy xác định và in ra xếp loại học lực theo các tiêu chí sau:

Xuất sắc: Điểm ≥ 9.0

Giỏi: $8.0 \leq \text{Điểm} < 9.0$

Khá: $6.5 \leq \text{Điểm} < 8.0$

Trung bình: $5.0 \leq \text{Điểm} < 6.5$

Yếu: Điểm < 5.0

Yêu cầu

- Nhập một số thực là điểm trung bình của học sinh.
- Xác định xếp loại dựa trên các mức điểm đã cho.
- In ra đúng tên xếp loại.
- Chương trình phải xử lý chính xác các giá trị biên của từng mức xếp loại.

Độ khó: Trung bình

Mã chuẩn

```

#include <stdio.h>

int main() {
    float diem;
    scanf("%f", &diem);

    if (diem >= 9.0)
        printf("Xuat sac");
    else if (diem >= 8.0)
        printf("Gioi");
    else if (diem >= 6.5)
        printf("Kha");
    else if (diem >= 5.0)
        printf("Trung binh");
    else
        printf("Yeu");

    return 0;
}

```

Danh sách Test Case

Tên Test Case	Input	Output
Test Case 1: Kiểm thử mức Xuất sắc	9.5	Xuat sac
Test Case 2: Kiểm thử giá trị biên mức Xuất sắc	9.0	Xuat sac
Test Case 3: Kiểm thử mức Giỏi	8.5	Gioi
Test Case 4: Kiểm thử giá trị biên mức Giỏi	8.0	Gioi
Test Case 5: Kiểm thử mức Khá	7.0	Kha
Test Case 6: Kiểm thử giá trị biên mức Khá	6.5	Kha
Test Case 7: Kiểm thử mức Trung bình	5.5	Trung binh
Test Case 8: Kiểm thử giá trị biên mức Trung bình	5.0	Trung binh
Test Case 9: Kiểm thử mức Yếu	4.9	Yeu
Test Case 10: Kiểm thử giá trị nhỏ nhất	0	Yeu